



中国研究生创新实践系列大赛
“华为杯”第二十一届中国研究生
数学建模竞赛

学 校

西南交通大学

参赛队号

24106130096

队员姓名

1. 易曾瑞

2. 闵心怡

3. 杨采琪

中国研究生创新实践系列大赛

“华为杯”第二十一届中国研究生

数学建模竞赛

题 目： 多变量复杂环境下的磁芯损耗建模

摘 要：

磁性元件的磁芯损耗精准预测在工业生产中有着重要的意义，但传统模型通常面临精度不足或适用场景受限等问题，无法在复杂的工况中实现有效评估。针对这个问题，本文提出了以下数据驱动的建模方法：

针对问题一，分析磁通密度的分布特征和形状特征，并利用其特征构建分类模型。首先通过划分形状特征（阶跃数）阈值进行分类，准确率能达到 98.5%，但是自动化程度低。进而，通过描述性分析提取了 5 维时域分布特征（均值、方差、峰度、偏度和信息熵），建立 SVM 分类模型，达到了 100% 的准确率。考虑到现实场景存在噪声，模拟噪声环境来测试模型的抗噪性，其精度下降了约 26%。因此，本文设计了融合 Hlibert — Huang 变换和双谱变换时频域统计特征的多域融合特征。通过实验分析，基于多域特征的准确率较时域特征提升了 10%，且在无噪条件下同样能达到 100% 的精度，说明多域特征能更有效地描述不同波形间的细微差异。最后，所提模型在测试集上的部分识别结果见表 5.7。

针对问题二，在 SE 方程的基础上进行修正，以适应不同温度 T 变化下的磁芯损耗预测。本文首先用 NAdam 算法求解原方程探究预测效果，其相对误差有 0.36。随后，本文基于对数法将方程变为线性的形式结构（简称 LSE），并用多元线性回归和 NAdam 分别进行求解。实验结果表明，两种方法的解相同，且 LSE 的相对误差降低到了 0.31。因此，本文考虑了两种能保持这种线性形式结构的修正模型，即在 SE 方程上乘以 γT 或 $e^{\gamma T}$ ，用多元线性回归进行求解，其相对误差分别为 0.17 和 0.18。考虑到修正后的模型不能动态调整以适应数据变化，本文提出了一种基于深度学习的自表示模型（称为 LSE_{DSR} ），可以根据温度自适应地调整待估参数。通过 NAdam 对模型求解， LSE_{DSR} 的相对误差降低至 0.13。

针对问题三，本文通过多因素方差分析研究单一外界条件及两两之间交互作用对磁芯损耗的影响，其中温度、励磁波形、材料与励磁波形和材料的交互作用均有显著影响，使用单因素方差分析量化各自的影响程度，选用 F 统计量作为指标评估，最后通过独立样本非参数检验寻找让磁芯损耗最小的外界条件，将同一个温度、励磁波形、和磁芯材料作为一个样本集，采用遍历的方法对比任意两组之间磁芯损耗之间的显著性差异，得到让磁芯

损耗最小的参数组为为温度 70 摄氏度、正弦波、材料 4；温度 90 摄氏度、正弦波、材料 2 和温度 90 摄氏度、正弦波、材料 4。

针对问题四，建立能跨越不同材料类型与工况条件的磁芯损耗预测模型。本文研究了分离模型、经验模型和深度模型。对于分离模型，本文探究了传统分离模型性能，尝试了深度自表示法和引入温度信息，通过实验分析，其相对误差分别有 0.78、0.46 和 1.1。对于经验模型，本文首先基于对数法分析了 MSE 方程的线性形式结构，并进行了如下的逐步改进：采用深度自表示法、引入温度信息、磁性材料信息和励磁波形信息，最终构建了 WMTLMSE_{DSR} 模型。其实验结果表明，通过逐步的改进，模型的相对误差从 0.51 逐渐降低至了 0.02。对于深度模型，本文基于深度特征嵌入建立了编码器模型，并探讨了其与所提模型 WMTLMSE_{DSR} 之间的内在联系。最后，所提模型在测试集上的部分预测结果见表 5.36。

针对问题五，建立优化模型以分析什么条件下能达到最小的磁芯损耗且具有最大的传输磁能。本文通过减法和除法的形式构建了两带约束条件的优化模型，并引入了正则项系数 λ ，最后用遗传算法和粒子群优化算法对优化模型进行求解。其实验结果表明，在 $\lambda = 1$ 时，频率 $f = 7862.85$ ，等效正弦波频率 $f_{\sin.eq} = 2.2E - 3$ ，磁通密度峰值 $B_m = 9.6E - 3$ ，磁芯材料为材料 1，温度 $T = 90$ ，正弦波时磁芯损耗最小，传输磁能最大： $P = 1449.52$, $\mathcal{P} = 3664.84$ 。在 $\lambda = 10$ 时能达到最小的磁芯损耗和最大的传输磁能的最优参数组合具体为：频率 $f = 50000$ ，等效正弦波频率 $f_{\sin.eq} = 8.25E - 4$ ，磁通密度峰值 $B_m = 1.99E - 1$ ，磁芯材料为材料二，温度 $T = 90$ ，正弦波时磁芯损耗最小，传输磁能最大： $P = 79740.54$, $\mathcal{P} = 18659.54$ 。

关键词： 磁芯损耗预测 多于融合特征 支持向量机 深度学习 智能优化算法

目录

1	问题重述	5
1.1	引言	5
1.2	问题的提出	5
2	模型的假设	6
3	符号说明	6
4	问题分析	7
4.1	问题一分析	7
4.2	问题二分析	7
4.3	问题三分析	8
4.4	问题四分析	8
4.5	问题五分析	8
5	模型的建立与求解	9
5.1	问题一的模型建立与求解	9
5.1.1	数据预处理和特征分析	9
5.1.2	基于支持向量机的分类模型	13
5.1.3	模拟噪声场景下的分类模型	16
5.1.4	多域融合特征分类	17
5.2	问题二的模型建立与求解	22
5.2.1	基于 NAdam 的 SE 方程求解	22
5.2.2	修正 1: LSE — 基于 $\ln$ 的 SE 方程	27
5.2.3	修正 2: TLSE — 考虑温度后的 LSE 方程	31
5.2.4	修正 3: $TLSE_{DSR}$ — 基于深度自表示的 TLSE 方程	33
5.2.5	总结	36
5.3	问题三的模型建立与求解	36
5.3.1	多因素方差分析判别显著影响条件	37
5.3.2	单因素方差分析量化显著影响条件	37
5.3.3	独立样本非参数检验寻找最优条件	41
5.4	问题四的模型建立与求解	43

5.4.1	分离模型: DM — 常规的分离模型	43
5.4.2	分离模型改进 1: DM_{SDR} — 基于深度自表示的 DM 模型	46
5.4.3	分离模型改进 2: TDM — 考虑温度的 DM 模型	47
5.4.4	经验模型改进 1: LMSE — 基于 ln 的修正 SE 方程	49
5.4.5	经验模型改进 2: $LMSE_{SDR}$ — 基于深度自表示的 LMSE 方程	51
5.4.6	经验模型改进 3: $TLMSE_{SDR}$ — 考虑温度的 $TLMSE_{SDR}$ 方程	53
5.4.7	经验模型改进 4: $MTLMSE_{SDR}$ — 考虑材料类型的 $TLMSE_{SDR}$ 方程	55
5.4.8	经验模型改进 5: $WMTLMSE_{SDR}$ — 考虑波形的 $MTLMSE_{SDR}$ 方程	57
5.4.9	探讨: 基于深度学习和 IGSE 方程的模型	59
5.4.10	总结	60
5.5	问题五的模型建立与求解	61
5.5.1	多目标优化模型	61
5.5.2	利用遗传算法求解优化模型	64
5.5.3	利用粒子群优化算法求解优化模型	67
6	模型评价与推广	69
6.1	模型优点	69
6.2	模型不足	69
6.3	模型推广	69
	参考文献	70
	附录 A MATLAB 源程序	72
A.1	问题 1: 计算多域融合特征	72
	附录 B Python 源程序	74
B.1	问题 1: 基于多域特征的分类	74
B.2	问题 2: $TLSE_{DSR}$ 方程求解	79
B.3	问题 4: DM_{DSR} 方程求解	84
B.4	问题 4: $WMTLMSE_{DSR}$ 方程求解	89
B.5	问题 5: 遗传算法-减法	96
B.6	问题 5: 粒子群优化-减法	100

1 问题重述

1.1 引言

随着通信以及电子信息技术的发展，电力电子技术取得了显著的进步，其中电能变换在电力领域如新能源和信息通信中起着重要的作用。电能转换技术如今被广泛应用于通讯电源、轨道交通、智能电网等多个方面。如今半导体技术愈发成熟，功率变换器产品也向着高频、高功率、高可靠性的方向发展。在功率转换器中，磁性元件（如变压器和电感）是不可或缺的，它们负责磁能的传递、存储和滤波，对设备的体积、重量、损耗和成本有重要影响。磁性元件想要具有高效率和高功率密度不仅需要满足电气参数，还需要保持低损耗。因此，研究和分析磁性元件的损耗特性变得至关重要。损耗分为绕组损耗和磁芯损耗，其中可以通过有限元仿真准确获得铜导体的绕组损耗，而由于磁性材料微观结构复杂，其损耗与工作频率、磁通密度、励磁波形、工作温度及磁芯材料相关，如高频、较高的磁通密度会增加磁滞损耗、不同材料具有不同的损耗特性、高温会改变材料特性从而影响损耗。由于这些因素之间存在复杂相关关系和相互作用，目前的磁芯损耗预测模型如分离模型和经验计算模型与实际情况仍存在较大差距。

1.2 问题的提出

围绕如何通过温度、频率、励磁波形和磁通密度准确评估磁芯损耗，本文依次提出如下问题：

问题一：由于励磁波形塑造了磁芯内部磁通的动态行为，因此不同的励磁波形会导致不同的磁芯损耗特性，进而对磁芯性能造成影响。励磁波形的影响主要体现在磁通密度上，使得磁通密度的时间序列呈现不同的波动。请通过附件一中不同的磁通密度分布特征，提取出励磁波形的形状特征，并通过这些形状特征建立起波形分类模型。最后通过构建的分类模型将附件二中样本的励磁波形识别出来并将结果填入附件四。

问题二：由于传统的斯坦麦茨方程（SE 方程）作用在正弦波形上，对于非正弦波形的情况有着较大的误差。并且 SE 方程并未考虑温度以及不同磁芯材料的影响，请在传统 SE 方程的基础上考虑温度对磁芯损耗的影响，以提出一种修正方程。最后通过附录一材料 1 的正弦波形数据对修正的 SE 方程和传统的 SE 方程进行对比，分别计算出两种方程磁芯损耗的预测误差。

问题三：磁芯损耗的主要影响因素较为常见的有温度、励磁波形和磁芯材料，现在需要深度评估这三种因素独立作用以及两两交互作用对磁性损耗的影响及其影响程度，并考虑如何通过设置这三种因素使得磁芯损耗最小化。

问题四：目前众多传统的磁芯损耗预测模型普遍面临着预测精度低的问题，业界希望能够开发出一种预测模型能够应对不同的外界条件和磁芯材料。请根据附件一中的数据，设计出准确的磁芯损耗预测模型并对其精度和泛化能力进行评估，最后对附件三中的数据

进行预测并将磁芯损耗预测结果填入附件四。

问题五：在实际工程问题中，除了磁芯损耗以外还有多个磁性元件的评价指标，传输磁能就是其中之一。现在将问题四中提出的模型作为目标函数，在此目标函数之外还需要考虑传输磁能。请建立最优化模型找出磁芯损耗尽可能小的同时传输磁能尽可能大所需要的外界条件。

2 模型的假设

为了便于问题的研究，对于题目中某些条件进行以下简化及合理的假设：

1. 假设样本之间相互独立及其磁芯损耗具有可比性；
2. 假设数据对各种变量划分的标准是可靠的；
3. 假设个别数据的缺失不会对计算结果造成大的影响；
4. 假设磁芯损耗与磁通密度变化率有关的物理性质；
5. 假设测量磁芯损耗时可能存在误差；
6. 假设磁通密度的测量可能存在噪声干扰。

3 符号说明

本文的符号说明如下：

符号	含义
n_{step}	阶跃数
n	样本数量
P	磁芯损耗
f, f_{sin}	频率及其修正量
$B, B_m, \Delta B$	磁通密度及其峰值和峰峰值
T	温度
W	励磁波形
M	材料
g_t, m_t, θ_t	t 时刻的梯度、动量以及参数
$k, \alpha, \beta, \gamma, \zeta$	(修正) SE 方程中的待估参数
w, b	多元回归方程参数
$\mathcal{L}$	损失函数
$\mathcal{P}$	传输磁能

4 问题分析

4.1 问题一分析

根据问题重述，不同的励磁波形磁通密度呈现出不同的增长、衰减或波动模式，本文的目标是分析不同波形下磁通密度的分布特征及其波形形状特征，并在此基础上进一步找出有表征性的特征，并基于这些特征变量对不同励磁波形进行精确分类。首先，对数据进行预处理，包括缺失值处理、数据标准化等步骤，选择合适的处理方法以确保数据适合后续的模型分析。为了更好地描述磁通密度的分布特征和形状特征，对不同工况下的磁通密度采样数据点进行数据分析，包括可视化分析和描述性统计等，直观地展示正弦波、三角波和梯形波三类励磁波形之间的特征差异，选择合适的特征。

接下来，利用适当的机器学习算法对所选择的特征构建分类模型，常用的机器学习算法包括支持向量机、决策树、随机森林等，我们希望选择的算法既能简单高效地完成分类任务，又能保证较高的准确性。为了评估所构建模型的效果，将附件一中的训练数据划分为训练集和验证集。使用训练集对模型进行训练，以学习模型参数；在验证集上，使用常见的分类评估指标（如准确率、召回率、F1 分数等）对模型进行评估。

考虑到现实生活中进行磁通密度数据采集时会遇到干扰产生噪声，故尝试在原始数据的基础上添加噪声，再次计算特征进行分类并评估分类精度。在波形域上进行特征提取有时会遇到一些挑战，尤其是在波形特征不明显的情况下，因此也将同时考虑频域特征，以增强特征的表征性。在此基础上，对特征进行筛选和降维，以确保用于分类的特征是最相关且最具表征性的。最后对比使用不同特征在是否添加噪声情况下的分类精度。选择最优的模型对附件 2 的测试数据进行分类。

4.2 问题二分析

如问题二的重述所说，传统的斯坦麦茨方程（SE 方程）有较大的局限性，其仅为正弦波形设计，并且一些会影响磁芯损耗的外界因素如磁芯材料和工作温度在 SE 方程中未考虑到。在这里希望引入温度这个因素来修正 SE 方程，以提高正弦波形磁芯损耗的预测效果。针对 SE 方程考虑因素不足的问题，颜冲等人^[1]在文章中考虑了直流偏置对磁芯损耗的影响，他们通过在 SE 方程上乘一个修正系数进行修正，其中修正系数是直流磁通密度和交流磁通密度幅值的函数。除此之外，飞利浦公司研究了 3F3 材料在正弦波下温度对磁芯损耗的影响，给出修正后的 SE 方程同样是乘以一个修正系数，其中修正系数是温度的函数。

本文同样考虑在经典 SE 方程上乘以温度的函数作为修正系数，在此之前需要先确定参数估计方式，主要有两种方式分别是 NAdam 优化算法迭代求解析解和最小二乘方法求闭式解。在比较两种求解方式的优劣后，再考虑引入温度进入 SE 方程并用适当的求解方式求解参数。并且除了将方程中的参数视为常数外，还可以考虑参数受外界条件影响的情

况，此时参数可能不是一个恒定值。

4.3 问题三分析

如问题三的重述所说，要研究一些外界条件如温度、励磁波形和磁芯材料对磁芯损耗的影响程度，除了要考虑单个条件对磁芯损耗的影响，还需要考虑两两条件协同作用对磁芯损耗的影响。可以使用多因素方差分析找出哪些外界条件及交互作用对磁芯损耗有显著的影响，并将识别出的条件及交互作用分别进行单因素方差分析以衡量各自的影响程度。

最后要找出磁芯损耗能达到最小时所对应的外界条件，可以将同一个温度、励磁波形和磁芯材料的样本视作一个样本集，由于温度、励磁波形和磁芯材料分别具有 4 个、3 个和 4 个水平，共有 48 组样本集。依次对 48 组样本集进行独立样本非参数检验即可找出其中平均磁通密度显著最小的样本集与之对应的外界条件。

4.4 问题四分析

在磁芯损耗的相关研究中，研究者们已经提出了有许多的预测评估模型，能在不同的环境下表现出一定的效果，但是通常都存在使用条件受限制或预测精度不够等问题，例如在不同的材料、温度和其他工作环境下，材料的磁芯损耗可能存在较大的差异。为解决这些问题，本文将设计一种新型的磁芯损耗预测模型，能在不同材料和工况条件下（包括温度、频率、励磁波形等因素）实现精准高效的预测。一个高精度的预测模型可以为磁性元件设计等领域的生产打下坚实的基础。在本文的具体问题中，将通过附件一的实验数据建立磁芯损耗预测模型并分析其预测的性能。

4.5 问题五分析

在磁性元件的设计与优化过程中，磁芯损耗和传输磁能是两个至关重要的评价指标，所以需要同时考虑，其中采用频率与磁通密度峰值的乘积来简化衡量传输磁能的大小，这一指标虽然不能全面反映传输磁能的复杂性，但足以作为优化设计的参考依据。为了实现磁性元件整体性能的最优化，使得磁芯损耗最小的同时传输磁能最大，需要构建一个多目标优化模型。

该模型以磁芯损耗预测模型为基础，同时将传输磁能作为另一个目标函数，分析在各种条件下（如温度、频率、等效正弦波频率、波形、磁通密度峰值及磁芯材料）磁性元件的性能表现，在给定约束条件下通过对模型的多次优化迭代，能够找到最优的设计参数组合。由于此优化问题设计到的参数条件范围宽泛，且问题四中得到的磁芯损耗预测模型是高度非凸的，传统的优化算法难以应对此类复杂优化问题，考虑利用智能优化算法求解，例如如遗传算法、粒子群优化算法等在求解优化问题时能够跳出局部最优解，探索全局解空间，从而找到全局最优解或者接近全局最优解的解决方案。

5 模型的建立与求解

5.1 问题一的模型建立与求解

5.1.1 数据预处理和特征分析

(1) 数据预处理

数据预处理是构建可靠分析模型的关键步骤之一，原始数据的质量和准确性对于研究结果具有重要的影响。由于波形和幅值大小无关，所以在波形分类任务中可以做 0-1 标准化，在提取特征前将原始磁通密度数据进行层标准化使每个样本数据均转化为无量纲、无数量级差异的标准化数值，将数据转换到相同的尺度上，防止部分特征因为数值大而对模型产生过大的影响，从而使结果更具有参考意义。

极差标准化法是一种常用的数据标准化方法，它将数据缩放到一个固定的区间 $[0, 1]$ 范围内。能够保留数据的相对比例，并且易于理解和实施。对于每个样本，设 $\tilde{x}_{max}$ 和 $\tilde{x}_{min}$ 分别为 $\tilde{x}$ 的最小值和最大值，将其一个原始值 $\tilde{x}$ 通过 $min-max$ 标准化映射成在区间 $[0, 1]$ 中的值 x ，其公式为：

$$x = \frac{\tilde{x} - \tilde{x}_{min}}{\tilde{x}_{max} - \tilde{x}_{min}} \quad (5.1)$$

(2) 时域可视化分析

可视化分析是一种利用图形或图像来展示和解释数据的方法，它可以辅助直观理解数据的形状特征。形状特征主要指数据在图表中的视觉表现形式，具体来说包括每个样本磁通密度数据形成的曲线形状、趋势、波峰波谷的分布、平滑性。从正弦波、三角波和梯形波这三种励磁类型的全部样本中，分别随机抽取了一个代表样本，并对其各自的 1024 个采样点的磁通密度数据绘制了波形图，如图 5.1(a) 所示。

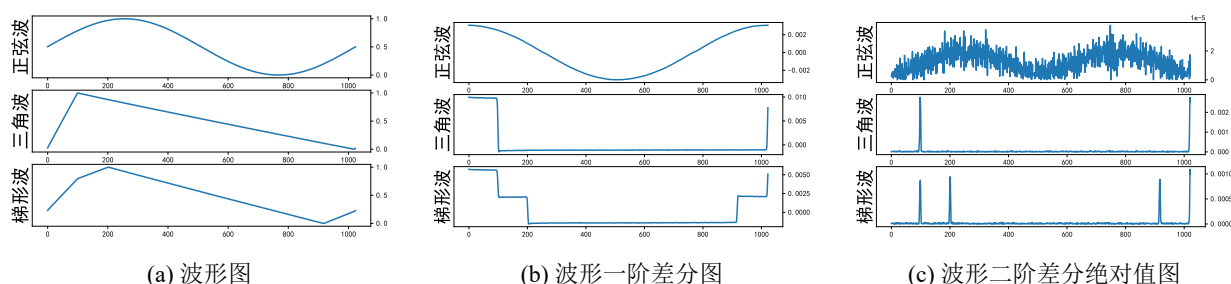


图 5.1 波形及差分特征图

图 5.1(a) 展示了正弦波、三角波和梯形波三种励磁波形的典型形状，从上至下分别是正弦波、三角波和梯形波。通过视觉对比直观发现不同波形的特点和区别。正弦波呈现出周期性的波动，形状类似正弦函数，曲线波动平稳没有明显的拐点，其峰值和谷值相对平滑；三角波则呈现为斜向上升和下降的趋势，波形是折线而非曲线，有明显的拐点，上升

和下降的斜率恒定，和横轴围成的图形形状类似三角形；而梯形波则是分两阶段上升一阶段下降，上升的斜率不同，在第一上升阶段和下降阶段较为陡峭，第二上升阶段较为平缓，波形是折线而非曲线，和横轴围成的图形形状类似梯形，有明显的拐点，且拐点数量相较于三角波更多。

那么针对形状特征能够较为明显地反映三种类型励磁波形的情况，本文考虑利用他们曲线的平滑程度和拐点的数量建立分类模型，例如导数和微分反应的是连续函数的值的变化，而差分反映了离散函数的值的变化，所以对磁通密度离散数据采样点做一阶差分，观察样本值的变化，如图 5.1(b)。具体来说，一阶差分是指对一个样本 1024 个磁通密度数据中的每个元素，计算其与其前一个元素之差，具体公式为：

$$\Delta x_t = x_t - x_{t-1} \quad (5.2)$$

观察图 5.1(b)，正弦波的一阶差分结果也是一条平滑的曲线，表示正弦波的变化率是连续且均匀的。三角波一阶差分结果这种波形由一系列线段组成，是一个阶梯函数，表示三角波的变化率是间断的，只在拐点处发生突变。梯形波与三角波类似，梯形波的一阶差分结果也是一个阶梯函数，但是相比三角波，它的台阶高度更小，台阶数更多表示梯形波的变化率相对更为平缓。一阶差分后三类波差异更明显，主要体现在斜率突变数量即图形阶梯的数量上，以图 5.1(b) 为例，由于正弦波图像没有突变点，图形的突变阶段数量为 0；三角波图形的突变阶段数为 1；而梯形波的图形的突变阶段数为 3。

因此，本文定义了阶跃数 n_{step} 刻画突变阶段的数量，突变阶段临近数据点之间的差异较大，可以通过差分较为直观地计算。所以对一阶差分的 n_{step} 进行差分计算，即原始数据的二阶差分。二阶差分绝对值后可可视化结果如图 5.1(c) 所示，一阶差分阶跃对应于二阶差分绝对值图上峰值处，判定阶跃的标准为二阶差分绝对值超过阈值即视为阶跃，阈值的选择会影响阶跃检测的结果。较小的阈值可能导致检测到更多的小波动，较大的阈值可能错过一些真实的阶跃，所以本文阈值的设定依赖于每个样本幅值，选用 $\frac{1}{4}$ 幅值作为阈值。超过阈值的区间个数即为 n_{step} ，对每个样本计算一阶差分的 n_{step} ，根据 n_{step} 的不同将样本划分为不同的类别。 $\frac{1}{4}$ 这个比例是人为选择的超参数。根据阶跃数 n_{step} 所在区间范围对各样本进行分类，本文所设定的分类标准如下表所示：

表 5.1 阶跃数分类标准

励磁波形类别	阶跃数 n_{step}	励磁波形类别
正弦波	$n_{step} \geq 10$	
三角波	$n_{step} < 3$	
梯形波	$3 \leq n_{step} < 10$	

通过上文中的阶跃数分类标准,对训练数据中的各样本进行分类,使用准确率 (Accuracy)、精确率 (Precision)、召回率 (Recall)、F1-score 这四个指标对模型的分类精度进行评估。

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (5.3)$$

$$Precision = \frac{TP}{TP + FP} \quad (5.4)$$

$$Recall = \frac{TP}{TP + FN} \quad (5.5)$$

$$F1\text{-score} = \frac{2 \times Precision \times Recall}{Precision + Recall} \quad (5.6)$$

各指标计算结果分别为准确率 0.985；精确率 0.985；召回率 0.985；F1-score 0.985，故利用阶跃数作为特征对样本分类总体具有较好的结果。更进一步通过混淆矩阵可视化分析算法性能，混淆矩阵能够可视化分类任务算法具体在每一个类别上的实际性能。其中每一列代表了预测类别，每一列的总数表示预测为该类别的数据的数目，每一行代表了数据的真实归属类别，每一行的数据总数表示该类别的数据实例的数目；每一列的数值表示真实数据被预测为该类的数目。在这里实际显示时为了更为直观地判断分类结果的优劣，每个位置上的数值分别除以它的真实类别所含样本的总数，从而每一行的和即为 1。利用阶跃数进行分类的混淆矩阵如图 5.2。

正弦波	96.32	0.39	3.28
三角波	0.0	99.96	0.04
梯形波	0.0	0.94	99.06
	正弦波	三角波	梯形波

图 5.2 阶跃数混淆矩阵

从混淆矩阵中可以看出，三角波和梯形波的预测精度较高，而正弦波的预测精度相对较低。考虑到阶跃数是基于磁通密度数据的形状特征来进行分类的，因此在预测过程中可

能出现偏差。此外，利用阶跃数作为分类依据时，需要人为设定阈值大小，这一过程不仅增加了操作的复杂性，还降低了分类任务的自动化水平。为进一步提升分类性能，考虑从数据的分布特征入手，使用能够自动适应波形变化的特征提取方法，进行更深入的分析。

(3) 描述性统计分析

描述性统计分析^[2]通过定量的方法对数据进行整理总结，挖掘数据的分布特征，分布特征通常指的是数据在数值范围内的分布情况。分布特征核心在于计算各种统计指标，包括中心趋势度量、离散程度变量、分布形态描述。中心趋势度量用于反映数据的集中趋势，包括均值、中位数、众数等；离散程度度量反映数据互相之间的离散程度，包括方差、标准差、四分位距等；分布形态包括峰度和偏度用于描述数据分布的尖峭程度和对称性。

为了全面描述数据的分布特征和波形之间的细微差异，本文在时域上选择了均值、方差、峰度、偏度和信息熵这几个关键指标。均值反映了数据集的中心位置，方差则量化了数据点围绕均值的离散程度，峰度和偏度分别用于描述分布的尖峭程度以及对称性，高峰度意味着数据集中度更高，而低峰度则意味着数据更加分散。而信息熵^[3]则衡量了数据集的不确定性或随机性。信息熵作为一个衡量不确定性的统计量，在描述性统计中可以用来量化数据的不确定性或者随机性。信息熵越高，说明数据的不确定性越大。

对于一个随机变量，数学期望即这里的均值定义为 $E(X)$ ，其方差定义为：

$$D(X) = E\{[X - E(X)]^2\} \quad (5.7)$$

峰度定义为：

$$Kurt(X) = \frac{E[X - E(X)]^4}{[E[X - E(X)]^2]^2} - 3 \quad (5.8)$$

偏度定义为：

$$Skew(X) = \frac{E[X - E(X)]^3}{[E[X - E(X)]^2]^{3/2}} \quad (5.9)$$

信息熵定义为：

$$H(X) = - \sum_i p(x_i) \log(p(x_i)) \quad (5.10)$$

计算每个样本的上述五个指标作为时域分布特征，为了观察这些时域分布特征对类别的关系，绘制了四个图例如图 5.3，均值-方差散点图、均值-峰度散点图、均值-偏度散点图、均值-信息熵散点图。

从图 5.3 中发现，不同励磁波形在均值维度上差异相对并不明显，在方差、峰度、信息熵三个维度上差异显著，这些特征能反映不同励磁波形类别磁通密度分布特征。利用这些特征变量建立分类模型，识别出励磁的三种波形是合理的。为了更详细地表征不同波形之间的细微差异，在接下来的分类中同时使用均值、方差、峰度、偏度和信息熵五个时域特征。

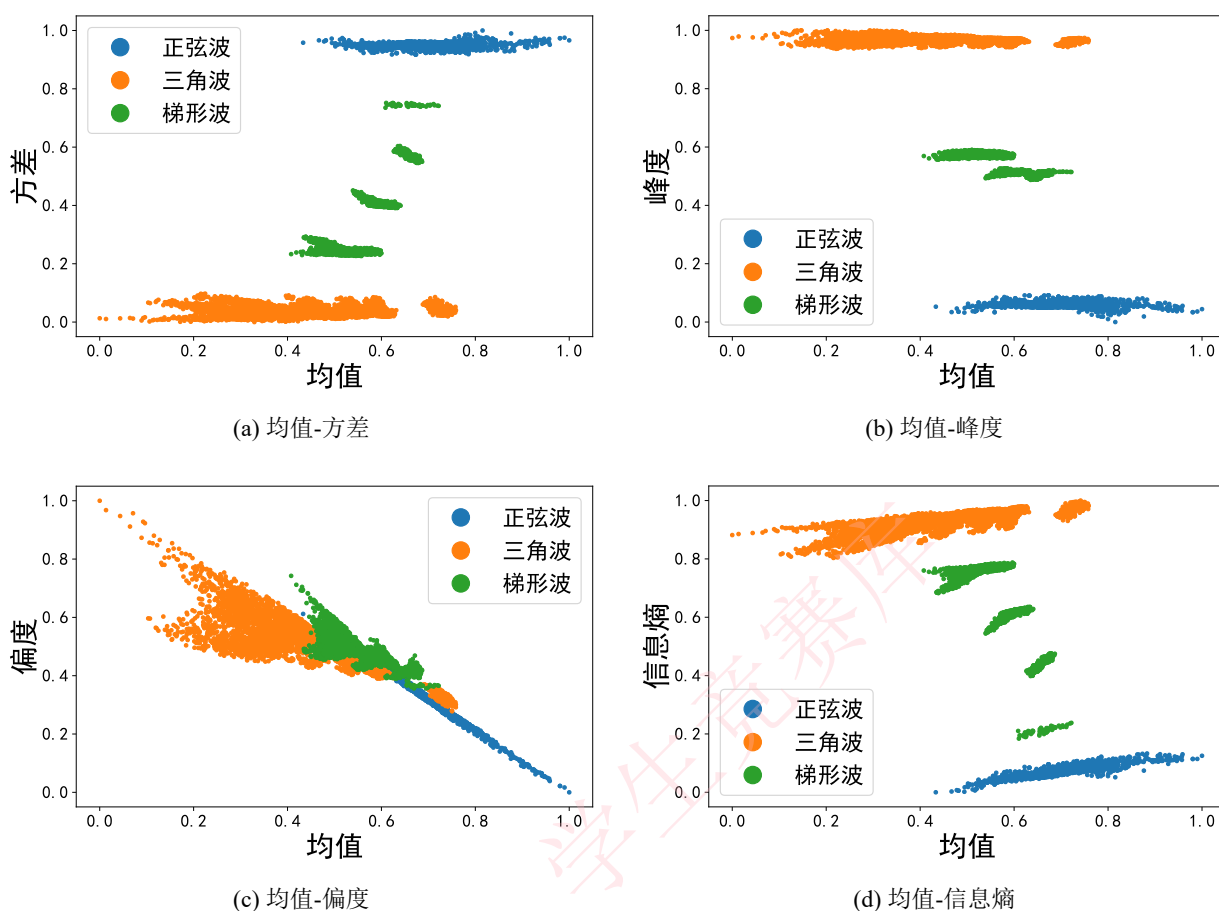


图 5.3 时域分布特征间的散点图

5.1.2 基于支持向量机的分类模型

从问题的性质和数据的特点出发，本文在研究过程中期望所选模型具备以下特性：较强的解释性，以便提供清晰可理解的预测结果；高效的分类性能及高精度；同时，倾向于使用较为简单的模型以达到良好的精度，避免过高复杂度带来的额外负担。支持向量机 (SVM) 模型能够满足上述要求，故决定采用支持向量机进行分类分析。

(1) 批量标准化

为了加速训练过程并提高模型的稳定性和性能，消除量纲对分类精度的影响，本文首先对从时域融合中提取出的样本的五个特征进行了批量标准化^[4]。具体而言，对每个特征下的数据分别进行了极差标准化。批量标准化技术通过对每个特征的输入数据进行归一化处理，进而减少了内部协变量偏移。这一技术不仅能够加快训练速度，还能够提高模型的收敛性，进一步提升模型的整体表现。

(2) 支持向量机

支持向量机^[4] (Support Vector Machines, SVM) 是一类按监督学习方式对数据进行二元分类的广义线性分类器，可以将问题化为一个求解凸二次规划的问题^[5]。并且可以将低

维不可分的数据通过核函数映射到高维使其线性可分，对小样本、非线性及高维问题具有独特的优势。

假设有样本数据 $(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N), (x_i, y_i) \in R, i \in N$ ，其中 x_i 是第 i 个样本， y_i 是与 x_i 对应的标签。SVM 需找到一个超平面将两类数据划分开并使得两类样本到超平面的距离最大，设分类的超平面方程为 $f(x) = w^T x + b$ ，得到以下优化问题：

$$\arg \min \frac{1}{2} w^T w + C \sum_{i=1}^l (\xi_i) \quad (5.11)$$

$$s.t. \quad y_i(w^T x_i + b) \geq 1 - \xi_i, i = 1, 2, \dots, l, \xi_i \geq 0$$

其中 $w^T w$ 表示结构风险， ξ 为松弛变量， C 为惩罚因子。运用拉格朗日乘子法：

$$(w, b, \xi, a, \tau) = \frac{1}{2} \|w\|^2 + C \sum_{i=1}^l \xi_i - \sum_{i=1}^l a_i (y_i (w^T x_i + b) - 1 + \xi_i) - \sum_{i=1}^l \tau_i \xi_i \quad (5.12)$$

其中 a_i, τ_i 为拉格朗日乘子。对 L 分别求偏导数，令其为零，得到：

$$w = \sum a_i y_i x_i, \quad 0 = \sum a_i y_i, \quad C = a_i + \tau_i \quad (5.13)$$

将式 5.13 代入式 5.12，得到：

$$\begin{aligned} \max_a L(a) &= \sum_{i=1}^l a_i - \frac{1}{2} \sum_{i,j=1}^l a_i a_j y_i y_j x_i^T x_j \\ s.t. \quad &\sum a_i y_i = 0, \quad 0 \leq a_i \leq C, \quad i = 1, 2, \dots, l \end{aligned} \quad (5.14)$$

将优化问题转化为求最值的问题。

支持向量机还包括核技巧，这使它成为实质上的非线性分类器。SVM 本质上是基于核的统计学习方法，参数以及核函数的选择对 SVM 的泛化能力有重要影响。核函数的主要作用就是将样本从原始空间映射到一个更高维的特征空间，使得样本在这个特征空间内线性可分。不同的核函数会导致模型性能的显著差异，因此必须根据数据特性和具体问题的需求来挑选合适的核函数。常用的核函数有线性核、多项式核及高斯核等，其中高斯核可以将原始空间映射到无穷维度的高维空间，以下是一些常用的核函数：

A. 高斯核

$$K(x, z) = \exp \left(-\frac{\|x - z\|^2}{2\sigma^2} \right) \quad (5.15)$$

其中 $\sigma > 0$ 为高斯核的带宽。

B. 线性核函数

$$K(x, z) = x \bullet z + 1 \quad (5.16)$$

C. Sigmoid 核函数

$$K(x, z) = \tanh(\beta x \bullet z + \theta) \quad (5.17)$$

D. Poly 多项式核函数

$$K(x, z) = (x \bullet z + 1)^p \quad (5.18)$$

其中 $p \geq 1$ 为多项式的次数。

核函数的选择通常需要通过交叉验证来确定，从而找到在具体问题上表现最好的核函数和其对应的超参数。本文选择对上述四种核函数均进行应用并且评估，比较在不同核函数上分类的效果。

(3) 模型求解及分析

A. 实验环境说明

本文实验的硬件环境为 Intel(R) Core(TM) i7-13700K@3.40 GHz，64G DDR5 内存及 GeForce RTX 4090 D（24GB 显存）的 64 位计算机。软件平台为 Python 3.11 及 Pytorch 2.0.1+cu118。

B. 参数设置

本文以附件一材料 1、材料 2、材料 3 和材料 4 中的所有波形样本共 12400 个样本为研究对象，并将其随机分为 80% 的训练集和 20% 的验证集。对于高斯核的支持向量机，惩罚因子 C 设置为 2，

C. 结果与分析

使用支持向量机基于样本的均值、方差、峰度、偏度和信息熵五个时域分布特征进行分类，评估在验证集上不同的核函数下的分类精度见表 5.2。当使用高斯核、线性核和 Poly

表 5.2 支持向量机基于时域分布特征分类精度

核函数	准确率 ↑	精确率 ↑	召回率 ↑	F1-score ↑
高斯核	1.000	1.000	1.000	1.000
线性核	1.000	1.000	1.000	1.000
Sigmoid 核	0.5419	0.5079	0.5419	0.5231
Poly 核	1.000	1.000	1.000	1.000

核时，所有指标（准确率、精确率、召回率和 Recall）都达到了 1.000，表明这些核函数在当前数据集上取得了完美的分类效果。然而，当使用 Sigmoid 核时，各项指标较低，与其它核相比有所下降，进一步绘制混淆矩阵，如图 5.4 所示。

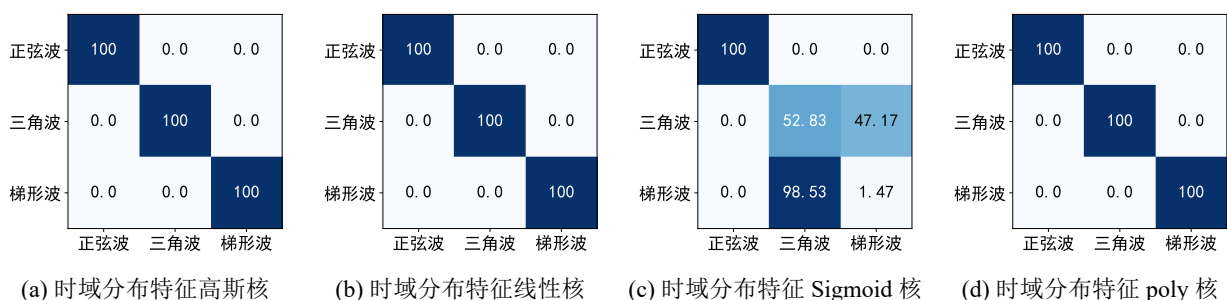


图 5.4 时域分布特征混淆矩阵

从混淆矩阵中可以看出，使用高斯核、线性核和多项式核均能利用时域特征对三类励磁波形实现准确预测，而 Sigmoid 核对于正弦波和梯形波的预测精度较高，三角波的预测精度较低。表明 Sigmoid 核函数不适用于本文的数据。通过上述评估，选择支持向量机模型对励磁波进行分类是合理且有效的。

5.1.3 模拟噪声场景下的分类模型

(1) 噪声影响

考虑到在现实场景测量磁通密度时可能存在误差，或受到其他干扰，影响磁通密度数据，真实的磁通密度数据可能含有噪声。故选择对原始数据添加噪声，这里添加信噪比为 5dB 的噪声，加噪后的磁通密度波形图如图 5.5 所示。

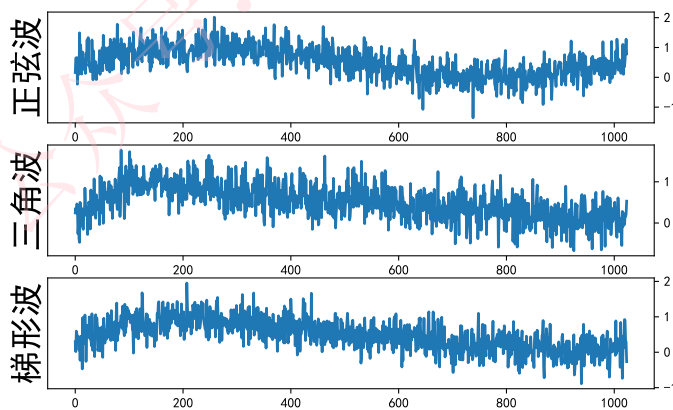


图 5.5 加噪后的波形图

(2) 加噪后的实验结果

A. 实验环境和参数设置说明

该模型的实验环境和参数设置与前一节相同。

B. 结果与分析

对磁通密度数据添加噪声后提取前文所述五种时域分布特征，利用支持向量机模型进行分类，验证集上分类精度见表 5.3，混淆矩阵如图 5.6 所示。从实验结果显示，使用高斯

核、线性核、Sigmoid 核和 Poly 核各指标均较低，且对于各类波分类精度均较低，尤其是梯形波，且观察到核函数用 Sigmoid 核进行分类时，将所有波均预测为了三角波。故在磁通密度数据添加噪声后使用时域分布特征建立模型进行分类的性能较差，时域特征已经不能完全刻画加噪后磁通密度数据的分布，为了更好地对加噪后数据分类，以适应真实环境，考虑进一步从加入频域特征协同分析。

表 5.3 加噪后基于时域分布特征分类精度

核函数	准确率 ↑	精确率 ↑	召回率 ↑	F1-score ↑
高斯核	0.717	0.698	0.717	0.702
线性核	0.723	0.710	0.723	0.715
Sigmoid 核	0.399	0.159	0.399	0.228
Poly 核	0.718	0.699	0.718	0.703

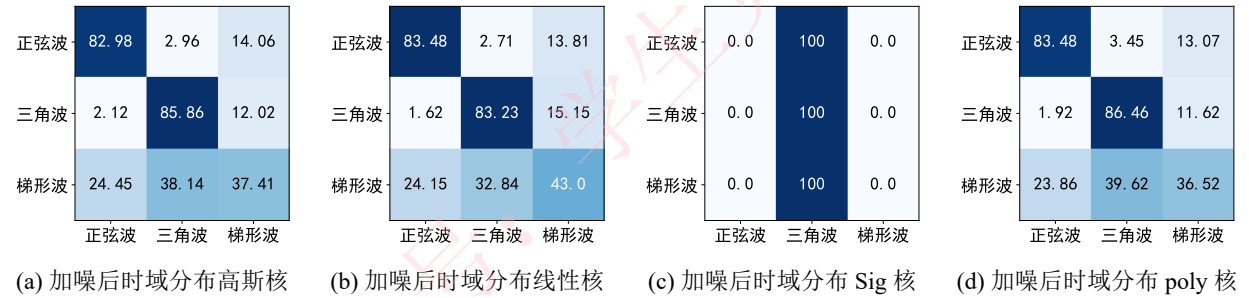


图 5.6 加噪后时域分布特征混淆矩阵

5.1.4 多域融合特征分类

(1) 多域融合

使用单一的时域或者频域分析方法提取磁通密度数据的特征难以全面地表示不同波形磁通密度波形的细微差别，存在局限性。所以希望能够同时结合时域和频域的信息，挖掘最能表示波形细微差异的特征，从而实现对加噪后的数据也能准确分类的目的。对于波形类别识别辐射源领域有很多方法，从辐射源识别进行迁移，本文采取多域融合特征提取方法应用于磁通密度数据特征提取^[6, 7, 8]。对磁通密度数据分别进行 Hilbert - Huang 变换和双谱变换，计算 Hilbert 边际谱和双谱投影，再分别提取变换域五个典型分布特征均值、方差、峰度、偏度和信息熵，最终拼接成 10 维的特征，Hilbert 边际谱和双谱投影作为时频谱同时包含有时域和频域的信息，对他们提取分布特征能够更有效刻画磁通密度数据本质特性，降低噪声对特征的干扰，有利于后续对加噪后磁通密度数据分类。

A. Hilbert - Huang 变换

Hilbert - Huang 变换是指将经验模态分解 (Empirical Mode Decomposition, EMD) 得到的内涵模态分量 (Intrinsic Mode Functions, IMF) 通过 Hilbert 变换最终得到信号瞬时频率和瞬时幅值。直接对解析信号做瞬时属性的分析时, 瞬时相位对时间的导数就是瞬时频率, 但是通过这个方法计算出的瞬时频率会出现负频率, 是没有物理意义的, 而 Hilbert - Huang 变换可以解决这一问题, 对非稳态信号做时频分析。Hilbert - Huang 变换相较于傅里叶变换和小波变换等传统的时频分析方法, 更适用于非线性和非平稳信号的分析, 它是对 EMD 分解后的 IMF 分量进行 Hilbert 变换来提供时频分布, 这个过程可以表示为:

$$s^k(t) = \sum_{i=1}^n c_i(t) + r_n(t) \quad (5.19)$$

原始信号 $s^k(t)$ 可以表示为 IMF 分量 $c_i(t)$ 和余项 $r_n(t)$ 的组合, 然后对每个 IMF 分量进行 Hilbert 变换:

$$\tilde{c}_i(t) = \frac{1}{\pi} \int_{-\infty}^{\infty} \frac{c_i(\tau)}{t - \tau} d\tau \quad (5.20)$$

构造解析函数

$$Z_i(t) = c_i(t) + j\tilde{c}_i(t) = a_i(t)e^{j\phi_i(t)} \quad (5.21)$$

其中 $a_i(t)$ 、 $\phi_i(t)$ 和 $\omega_i(t)$ 分别表示瞬时振幅、相位和频率, 即

$$a_i(t) = \sqrt{c_i(t)^2 + \tilde{c}_i(t)^2} \quad (5.22)$$

$$\phi_i(t) = \arctan\left(\frac{\tilde{c}_i(t)}{c_i(t)}\right) \quad (5.23)$$

$$\omega_i(t) = \frac{d\phi_i(t)}{dt} \quad (5.24)$$

进而 Hilbert 谱可以表示为

$$H(\omega, t) = \text{Re}\left(\sum_{i=1}^n a_i(t)e^{j\int \omega_i(t)dt}\right) \quad (5.25)$$

HHT 边际谱由 HHT 时频谱沿时间轴进行积分得到:

$$J(\omega) = \int_0^T H(\omega, t)dt \quad (5.26)$$

B. 双谱变换

双谱是最低阶的高阶谱, 通过双谱变换可以有效地抑制一定程度的高斯噪声, 使得辐射源的细微特征充分地反映在双谱信息中^[9]。双谱的定义为:

$$B(\omega_1, \omega_2) = \sum_{\tau_1=-\infty}^{\infty} \sum_{\tau_2=-\infty}^{\infty} c_{3x}(\tau_1, \tau_2) \exp[-j2\pi(\omega_1\tau_1 + \omega_2\tau_2)] \quad (5.27)$$

其中 $c_{3x}(\tau_1, \tau_2) = E\{x(t)x(t+\tau_1)(t+\tau_2)\}$ ，表示信号的三阶累积量。

由于双谱系数是对称的，因此定义双谱投影为：

$$P = \sum_i B(\omega_i, \omega_2) = \sum_i B(\omega_1, \omega_i) \quad (5.28)$$

C. 特征提取

对 Hilbert - Huang 变换后的 Hilbert 边际谱和双谱变换后的双谱投影，分别计算均值、方差、峰度、偏度和信息熵五个特征，共计十个特征。

(2) T-SNE 降维

为了预先分析数据十维特征的可分性，考虑采用 T-SNE 降维可视化分析^[10]，T-SNE 是一种非线性降维技术，适用于在二维或三维的低维空间中表征高维数据。通过对降维数据的可视化，可以了解到高维数据的分布以及其可分性。与其他降维算法（如 PCA）相比，T-SNE 创建了一个缩小的特征空间，相似的样本由附近的点建模，不相似的样本由高概率的远点建模。核心思想是定义两个概率分布，一个基于高维数据，另一个基于低维嵌入空间中的数据。通过最小化这两个分布之间的差异来找到一个合适的低维映射。

给定高维数据集 $X = \{x_1, x_2, \dots, x_N\}$ ，寻找一个低维表示 $Y = \{y_1, y_2, \dots, y_N\}$ ，使得 X 中的相似度尽可能地反映在 Y 中。

高维空间中的相似度 $p_{j|i}$ 通过以下方式计算：

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)} \quad (5.29)$$

低维空间中的相似度 q_{ij} 由 Student-t 分布给出：

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}} \quad (5.30)$$

KL 散度用来衡量 p 和 q 之间的差异

$$D_{KL}(P||Q) = \sum_{i,j} p_{ij} \log \frac{p_{ij}}{q_{ij}} \quad (5.31)$$

T-SNE 通过梯度下降法来最小化 KL 散度 $D_{KL}(P||Q)$ ，从而得到最终的低维嵌入 Y 。本文用 T-SNE 方法降维，将原始数据标准化后降至二维后进行可视化，其分布如图 5.7 所示。从图中观察到，不同类别的样本点区分度很高，这一定程度上说明原始数据线性可分，使用多域融合特征得到的十类特征用于分类是合理的。

(3) 模型求解及分析

A. 实验环境说明和参数设置

该模型的实验环境和参数设置与前一节相同。在测试样本分类部分的研究以附件二材料 1、材料 2、材料 3 和材料 4 中的所有波形样本共 80 个样本为研究对象。

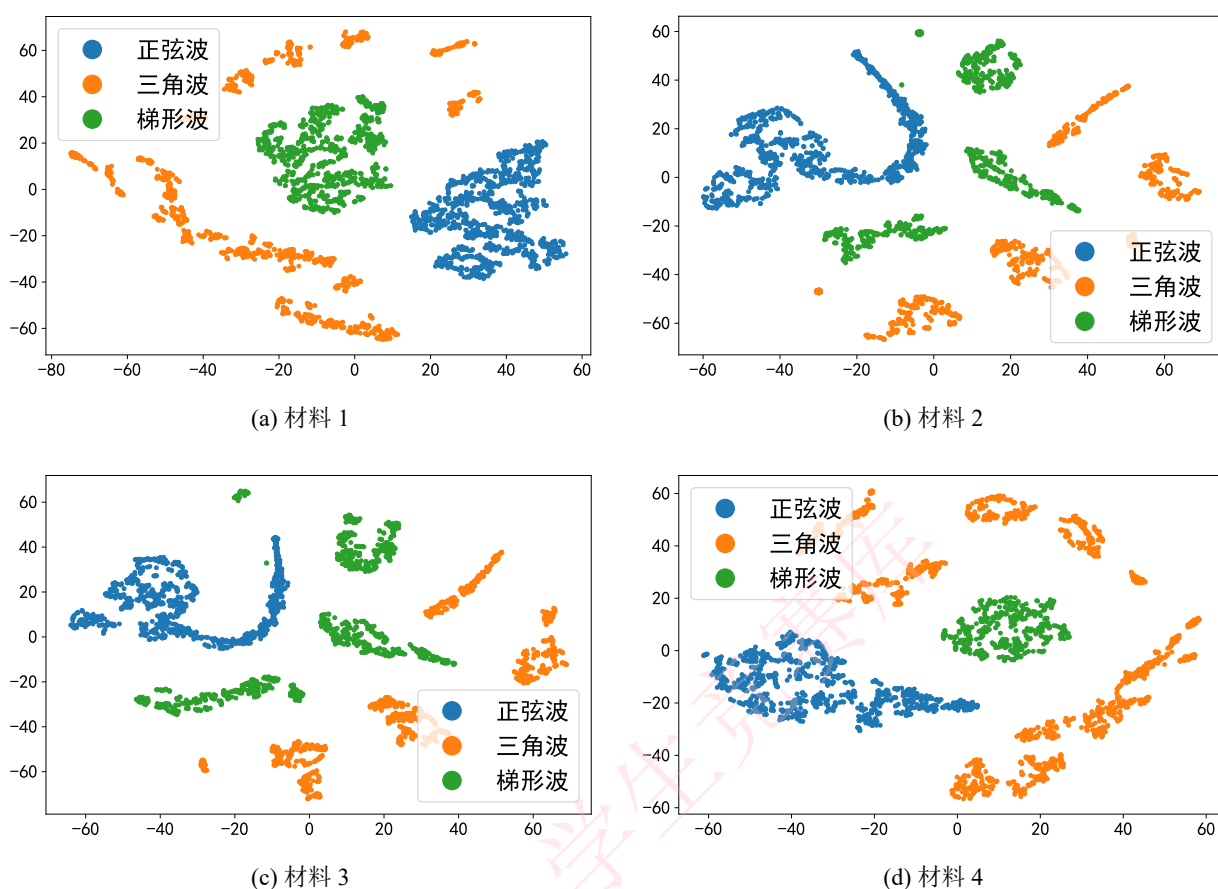


图 5.7 T-SNE 降维后的数据分布

B. 结果与分析

使用支持向量机基于十种多域融合特征对加噪后的磁通密度进行分类，评估在验证集上不同的核函数下的分类精度，见表 5.4。当使用高斯核、线性核和 Poly 核时，所有指标

表 5.4 加噪后时域分布特征和多域融合特征分类精度对比

核函数	准确率		精确率		召回率		F1-score	
	时域分布	多域融合	时域分布	多域融合	时域分布	多域融合	时域分布	多域融合
高斯核	0.717	0.795	0.698	0.797	0.717	0.795	0.702	0.789
线性核	0.723	0.773	0.710	0.780	0.723	0.773	0.715	0.770
Sigmoid 核	0.399	0.164	0.159	0.126	0.399	0.164	0.228	0.142
Poly 核	0.718	0.810	0.699	0.810	0.718	0.810	0.703	0.804

(准确率、精确率、召回率和 *Recall*) 均高于 0.75，有较好的分类精度；使用 Sigmoid 核时，各项指标较低。进一步绘制混淆矩阵，如图 5.8 所示。对比加噪后基于多域融合特征分类精度与基于时域分布特征分类精度，如表 5.4 所示，各类波分类精度均有所提升，所以频

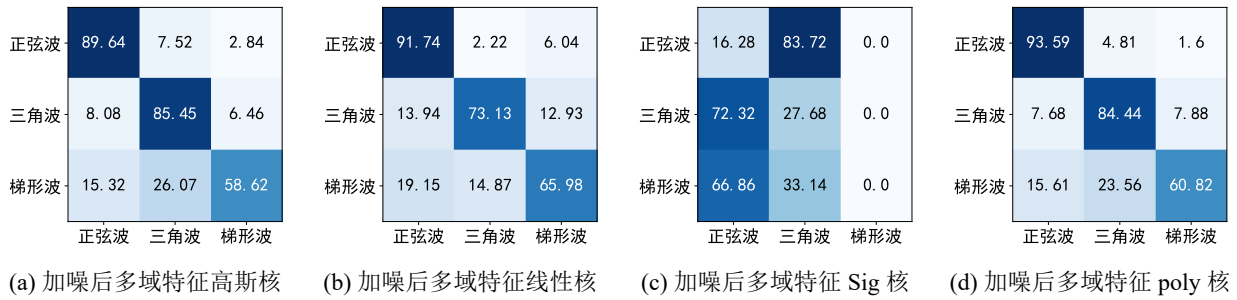


图 5.8 加噪后多域特征混淆矩阵

表 5.5 加噪后基于多域融合特征分类精度

核函数	准确率 ↑	精确率 ↑	召回率 ↑	F1-score ↑
高斯核	0.795	0.797	0.795	0.789
线性核	0.773	0.780	0.773	0.770
Sigmoid 核	0.164	0.126	0.164	0.142
Poly 核	0.810	0.810	0.810	0.804

域特征对分类任务有利的特征，多域特征能够很好地刻画加噪后磁通密度数据的分布，从而更准确地分类，在实际生活应用中考虑使用多域特征。

同时为了检验本文使用多域融合特征提取方法提取的特征对原始不加噪声磁通密度数据的表征能力，使用支持向量机利用原始数据多域融合特征进行分类和评估，对比其与利用时域特征进行分类的性能，对比结果如表 5.6 所示。对比结果显示在线性核上使用

表 5.6 无噪声时域分布特征和多域融合特征分类精度对比

核函数	准确率		精确率		召回率		F1-score	
	时域分布	多域融合	时域分布	多域融合	时域分布	多域融合	时域分布	多域融合
高斯核	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
线性核	1.000	0.999	1.000	0.999	1.000	0.999	1.000	0.999
Sigmoid 核	0.5419	0.617	0.5079	0.722	0.5419	0.617	0.523	0.602
Poly 核	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000

域分布特征的分类精度比使用多域融合特征时高 0.001，而在 sigmoid 核上使用多域融合特征的分类精度比使用时域分布特征时高 0.1。所以本文选择的多域融合特征能够适应理想数据和现实数据两种情况，有无噪声时均能够根据磁通密度数据实现高效高精度分类。通过上述的研究，本文能对励磁波形准确分类，对后续探究不同励磁波形对磁芯损耗奠定基础，此外自动化无需人为设置的方法也为实际落地应用提供可能。

C. 测试样本分类

本文使用基于时域分布特征和基于多域融合特征的支持向量机模型对附件二 80 个测试样本分类，二者分类结果一致。最终得到附件二中三种波形的各自数量为正弦波：20，三角波：44，梯形波：16。其中样本序号为：1、5、15、25、35、45、55、65、75、80 的分类结果如表 5.7 所示。为了直观观察分类是否正确，本文绘制了这十个测试样本的磁通密度波形，如图 5.9 所示，可以发现这十个样本均分类正确，反映出本文模型在测试样本上同样表现优异，模型具有较强的泛化能力。

表 5.7 测试样本分类

序号	1	5	15	25	35	45	55	65	75	80
预测类别	三角波	三角波	正弦波	三角波	梯形波	梯形波	三角波	三角波	三角波	正弦波

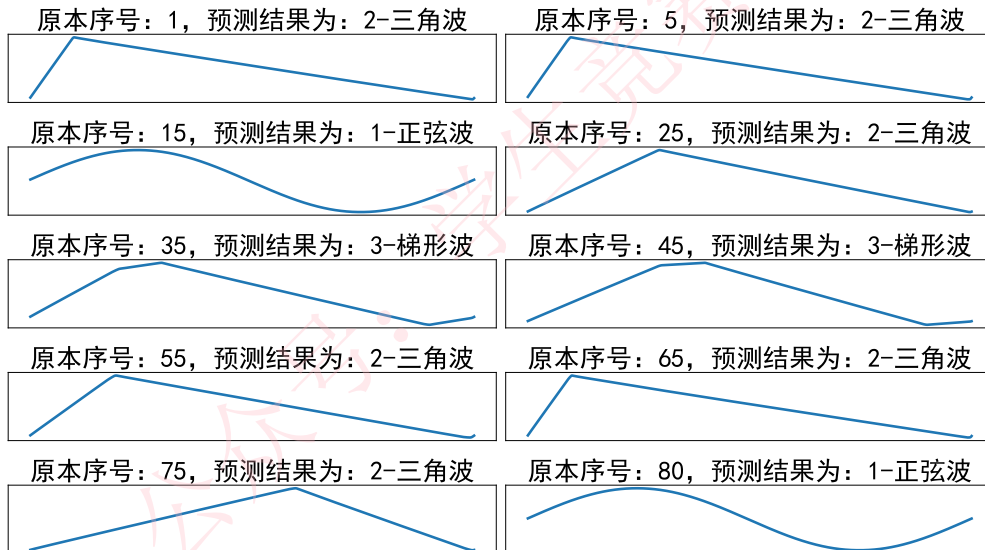


图 5.9 测试集预测结果图

5.2 问题二的模型建立与求解

在研究如何修正斯坦麦茨方程之前，需要对传统斯坦麦茨方程的参数估计方式及结果、预测精度进行分析。以便确定修正后的 SE 方程的参数估计方式，并与修正后的预测结果进行对比。

5.2.1 基于 NAdam 的 SE 方程求解

由于斯坦麦茨方程（Steinmetz Equation, SE）

$$P = k_1 \cdot f^{\alpha_1} \cdot B_m^{\beta_1} \quad (5.32)$$

为非线性方程，其中待估参数 k_1 、 α_1 和 β_1 无法通过常见的参数估计方式求解，因此考虑使用 NAdam 优化算法^[11, 12]（Nesterov-accelerated Adaptive Moment Estimation）迭代求解参数。其中 NAdam 优化算法的原理如下文所述。

(1) NAdam 优化算法

梯度下降算法^[13, 14]通过迭代的方式求解最优参数，其主要思想是找到损失函数下降最快的方向以确定梯度的方向。NAdam 算法是在 Adam 算法^[15]的基础上引入 NAG 算法^[16]形成的，它通过将 Adam 算法中的经典动量项 m_t 替换为 NAG 动量以提高优化算法的性能。

传统的迭代方法是通过当前迭代的梯度 g_t 以及前一轮迭代的动量 m_{t-1} 来更新当前的动量 m_t ，从而进一步更新下一轮迭代的参数 θ_{t+1} ，具体地说

$$\begin{aligned} g_t &= \nabla_{\theta_t} \mathcal{L}(\theta_t) \\ m_t &= \gamma m_{t-1} + \eta g_t \\ \theta_{t+1} &= \theta_t - m_t \end{aligned}$$

其中 $\mathcal{L}$ 是目标损失函数， γ 是动量衰减项， η 是步长。而 NAG 在计算梯度之前，通过使用动量更新参数，从而在梯度方向上执行更精确的步骤。但此过程需要两次应用动量更新参数，因此更简洁的方式是

$$\begin{aligned} g_t &= \nabla_{\theta_t} \mathcal{L}(\theta_t) \\ m_t &= \gamma m_{t-1} + \eta g_t \\ \theta_{t+1} &= \theta_t - (\gamma m_t + \eta g_t) \end{aligned}$$

Adam 是将 RMSprop^[17] 和动量法相结合，使用动量进行梯度下降的参数更新，并能够自适应调整学习率，具体地说

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} (h_1 \hat{m}_{t-1} + \frac{(1 - h_1)g_t}{1 - h_1^t}) \quad (5.33)$$

其中 $\hat{m}_{t-1} = m_{t-1}/(1 - h_1^t)$ 。现在将 Nesterov 动量添加进 Adma（式 5.33）中，简单地说就是用当前动量 $\hat{m}_t$ 的偏差修正估计替换之前的动量 $\hat{m}_{t-1}$ 的偏差修正估计，得到 Nadam 更新规则为

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} (\beta_1 \hat{m}_t + \frac{(1 - \beta_1)g_t}{1 - \beta_1^t}) \quad (5.34)$$

(2) 模型的建立

在训练集上使用 NAdam 算法 (式 5.34) 通过迭代使得损失函数

$$\begin{aligned} \mathcal{L} &= \mathbb{E}\{(P - \hat{P})^2\} \\ &= \mathbb{E}\{(P - \hat{k}_1 \cdot f^{\hat{\alpha}_1} \cdot B_m^{\hat{\beta}_1})^2\} \end{aligned} \quad (5.35)$$

最小化，其中 P 为真实磁芯损耗， $\hat{P}$ 为磁芯损耗预测值， $\hat{k}_1$ ， $\hat{\alpha}_1$ 和 $\hat{\beta}_1$ 是迭代求解出的最优参数。

(3) 评价指标

为了衡量模型的预测效果，这里主要使用七种统计指标进行度量，具体如下：

A. R^2

R^2 也称作决定系数，代表预测值对 P 的方差的解释比例，它经常被用来衡量回归模型的拟合程度，并且计算公式为

$$R^2 = 1 - \frac{\sum_{i=1}^n (P_i - \hat{P}_i)^2}{\sum_{i=1}^n (P_i - \bar{P})^2} \quad (5.36)$$

其中 $\bar{P}$ 为所有样本 P 值的均值。通常 R^2 的取值都在 0 到 1 之间， R^2 越接近 1 代表模型的拟合程度越好，反之越接近 0 代表模型不能很好解释 P 的变化情况。

B. MRE

MRE 是平均相对误差，代表 P 预测值与真实值之间的相对偏差的平均水平，它的计算公式为

$$MRE = \frac{1}{n} \sum_{i=1}^n \left| \frac{P_i - \hat{P}_i}{P_i} \right| \quad (5.37)$$

MRE 的值越小，代表模型预测精度越高，拟合效果越好。

C. MAE

MAE 是平均绝对误差，代表 P 预测值与真实值之间的绝对偏差的平均水平，它的计算公式为

$$MAE = \frac{1}{n} \sum_{i=1}^n |P_i - \hat{P}_i| \quad (5.38)$$

MAE 的值越小，代表模型预测精度越高，拟合效果越好。

D. MSE

MSE 是均方误差，代表 P 预测值与真实值之间的差异程度，它的计算公式为

$$MSE = \frac{1}{n} \sum_{i=1}^n (P_i - \hat{P}_i)^2 \quad (5.39)$$

MSE 的值越小，代表模型预测精度越高，拟合效果越好。有许多优化算法如 NAdam 都将 MSE 作为损失函数。

E. $RMSE$

$RMSE$ 是均方根误差，简单来说就是 MSE 的幂，它的计算公式为

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (P_i - \hat{P}_i)^2} \quad (5.40)$$

$RMSE$ 的值越小，代表模型预测精度越高，拟合效果越好。

F. $MSLE$

$MSLE$ 是均方对数误差，它通过使用对数转换使得误差度量更为稳定，计算公式为

$$MSLE = \frac{1}{n} \sum_{i=1}^n \left(\log(1 + P_i) - \log(1 + \hat{P}_i) \right)^2 \quad (5.41)$$

$MSLE$ 的值越小，代表模型预测精度越高，拟合效果越好。

G. EVS

EVS 为解释方差得分，作用和 R^2 类似，它代表着模型能解释数据方差的比例，计算公式为

$$EVS = 1 - \frac{\sum_{i=1}^n (\hat{P}_i - \bar{P})^2}{\sum_{i=1}^n (P_i - \bar{P})^2} \quad (5.42)$$

通常 EVS 的取值都在 0 到 1 之间， EVS 越接近 1 代表模型的拟合程度越好，反之越接近 0 代表模型不能很好解释数据方差。

(4) 模型的求解及分析

A. 实验环境说明

本文实验的硬件环境为 Intel(R) Core(TM) i7-13700K@3.40 GHz，64G DDR5 内存及 GeForce RTX 4090 D (24GB 显存) 的 64 位计算机。软件平台为 Python 3.11 及 Pytorch 2.0.1+cu118。

B. 参数设置

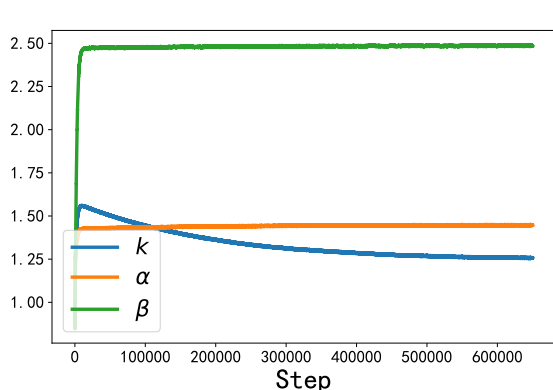
本文以附件一材料 1 中的正弦波形共 1067 个样本为研究对象，并将其随机分为 80% 的训练集和 20% 的测试集。并且给定三种未知参数 k_1 ， α_1 和 β_1 的初始值均为 1，初始学习率设置为 0.001，批处理大小设置为 32，迭代次数设置为 25000 次。

C. 结果与分析

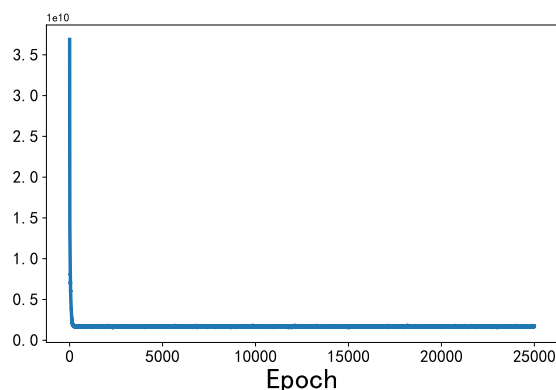
损失函数值以及待估参数随迭代次数和批处理数目增加的变化趋势如下图 5.10 所示。图 5.10b 中横轴 Epoch 与图 5.10a 中横轴 Step 分别代表迭代次数与其对应的批处理数目。由上图 5.10 可以看出，损失函数值随着迭代次数的增加而迅速降低，并在迭代次数增加到 1000 次时已经趋于稳定，而参数 k_1 先随着批处理数目的增加而迅速上升而后缓慢降低并收敛到常数 1.26，参数 α_1 首先迅速升高并趋于一个常数 1.45，参数 β_1 则是在开始时先迅速降低然后迅速升高并收敛到常数 2.49。其中造成参数迅速改变方向的原因可能是在训练开始时损失还在持续降低的过程中。

因此使用 NAdam 优化算法求解的最优 SE 方程为

$$P = 1.26 \cdot f^{1.45} \cdot B_m^{2.49} \quad (5.43)$$



(a) 参数变化趋势



(b) 损失函数值变化趋势

图 5.10 使用 NAdam 求解 SE 方程过程

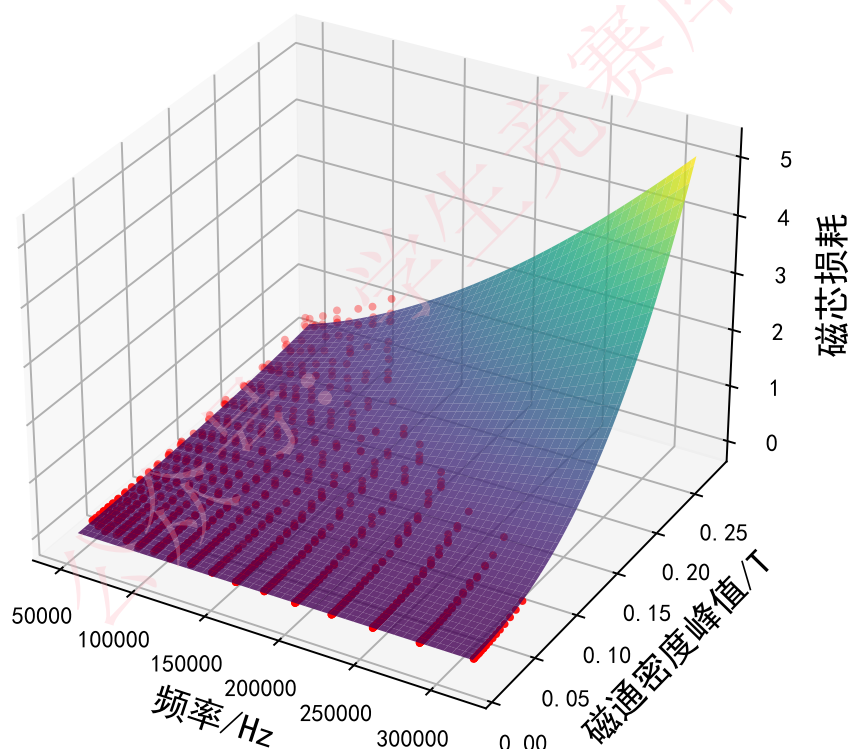


图 5.11 原始空间上的拟合结果

其中参数满足一般性条件 $1 < \alpha_1 = 1.45 < 3$ 和 $2 < \beta_1 = 2.49 < 3$ 。下面通过绘图以更加直观体现拟合效果，其中图 5.11 展示了所拟合出的 SE 方程（式 5.43）对应的曲面与原始空间上样本点的对应关系。

另外在训练集和测试集上的预测结果如下表 5.8 所示，其中各性能度量参数的计算等式见式 5.36 至 5.42。从表 5.8 可以看出，拟合结果有较高的拟合优度 (R^2) 以及解释方差得分 (EVS)，但平均相对误差 (MRE) 在 0.35 附近并且均方根误差 ($RMSE$) 较大，这

些指标说明模型的拟合效果较为一般。

表 5.8 使用 NAdam 求解 SE 方程的预测性能

	$R^2 \uparrow$	$MRE \downarrow$	$MAE \downarrow$	$MSE \downarrow$	$RMSE \downarrow$	$MSLE \downarrow$	$EVS \uparrow$
训练集	0.946	0.345	2.096E+4	1.682E+9	4.102E+4	0.138	0.946
测试集	0.939	0.358	1.955E+4	1.473E+9	3.838E+4	0.143	0.939

5.2.2 修正 1: LSE — 基于 $\ln$ 的 SE 方程

另一种 SE 方程参数的估计方式是对等式左右两边取对数，从而将非线性方程 SE 转换为对数 SE 方程（Ln Steinmetz Equation, LSE）

$$\ln(P) = \ln(k_1) + \alpha_1 \cdot \ln(f) + \beta_1 \cdot \ln(B_m) \quad (5.44)$$

此时对于多元线性方程^[18]的参数求解，不仅可以使用 NAdam 优化算法求得其数值解，还能通过最小二乘法求得其闭式解。其中最小二乘法^[19]的原理如下文所述。

(1) 最小二乘法

多元线性回归模型

$$f(\mathbf{x}) = \mathbf{w}\mathbf{x} + b$$

试图确定未知参数 $\mathbf{w}$ 和 b 的值，使得在有 n 个样本的数据集 $D = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$ 上有 $f(\mathbf{x}_i) \simeq y_i$ 。由于回归问题经常使用均方误差（MSE）作为性能度的重要指标，因此将均方误差作为损失函数，使其最小化以确定未知参数 $\mathbf{w}$ 和 b 的估计值 $\hat{\mathbf{w}}$ 和 $\hat{b}$ ，即

$$\begin{aligned} (\hat{\mathbf{w}}, \hat{b}) &= \arg \min_{(\mathbf{w}, b)} \sum_{i=1}^n (f(\mathbf{x}_i) - y_i)^2 \\ &= \arg \min_{(\mathbf{w}, b)} \sum_{i=1}^n (y_i - \mathbf{w}\mathbf{x}_i - b)^2 \\ &=: \arg \min_{(\mathbf{w}, b)} \mathcal{L}_{(\mathbf{w}, b)} \end{aligned}$$

这种将均方误差最小化来求解参数的方法被称为最小二乘法，其核心思想就是回归一条直线，使得各个样本点到直线距离之和最小化。

要想求得损失函数 $\mathcal{L}_{(\mathbf{w}, b)}$ 最小时对应的参数值，可以分别对损失函数中的 $\mathbf{w}$ 和 b 求偏导并另偏导为零，即

$$\begin{aligned} \frac{\partial \mathcal{L}_{(\mathbf{w}, b)}}{\partial \mathbf{w}} &= 2 \left(\mathbf{w} \sum_{i=1}^n \mathbf{x}_i^2 - \sum_{i=1}^n (y_i - b) \mathbf{x}_i \right) = 0 \\ \frac{\partial \mathcal{L}_{(\mathbf{w}, b)}}{\partial b} &= 2 \left(mb - \sum_{i=1}^n (y_i - \mathbf{w}\mathbf{x}_i) \right) = 0 \end{aligned} \quad (5.45)$$

求解式 5.45 即可得到 $\mathbf{w}$ 和 b 的闭式解为

$$\begin{aligned}\hat{\mathbf{w}} &= \frac{\sum_{i=1}^n y_i (\mathbf{x}_i - \bar{\mathbf{x}})}{\sum_{i=1}^n \mathbf{x}_i^2 - \frac{1}{n} (\sum_{i=1}^n \mathbf{x}_i)^2} \\ \hat{b} &= \frac{1}{n} \sum_{i=1}^n (y_i - \hat{\mathbf{w}} \mathbf{x}_i)\end{aligned}\quad (5.46)$$

其中 $\bar{\mathbf{x}} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$ 为样本协变量 $\mathbf{x}$ 的均值。

(2) 模型的建立

在训练集上使用 NAdam 算法或最小二乘法使得损失函数

$$\begin{aligned}\mathcal{L} &= \mathbb{E}\{(\ln P - \ln \hat{P})^2\} \\ &= \mathbb{E}\{(\ln P - \ln \hat{k}_1 - \hat{\alpha}_1 \cdot \ln f - \hat{\beta}_1 \cdot \ln B_m)^2\}\end{aligned}\quad (5.47)$$

最小化，其中 P 为真实磁芯损耗， $\hat{P}$ 为磁芯损耗预测值， $\hat{k}_1$ ， $\hat{\alpha}_1$ 和 $\hat{\beta}_1$ 是根据式 5.46 求解出的最优参数。

(3) 模型的求解及分析

A. 实验环境和参数设置说明

该模型的实验环境和参数设置中除迭代次数设置为 10000 以外其余均与前一节相同

B. 结果与分析 (NAdam 方法)

与 5.2.1 节类似，使用 NAdam 优化算法求解 LSE 方程中的待估参数。在训练集上使用 NAdam 算法通过迭代使得损失函数最小化，损失函数值以及待估参数随迭代次数与其对应的批处理量增加的变化趋势分别如下图 5.12 所示。

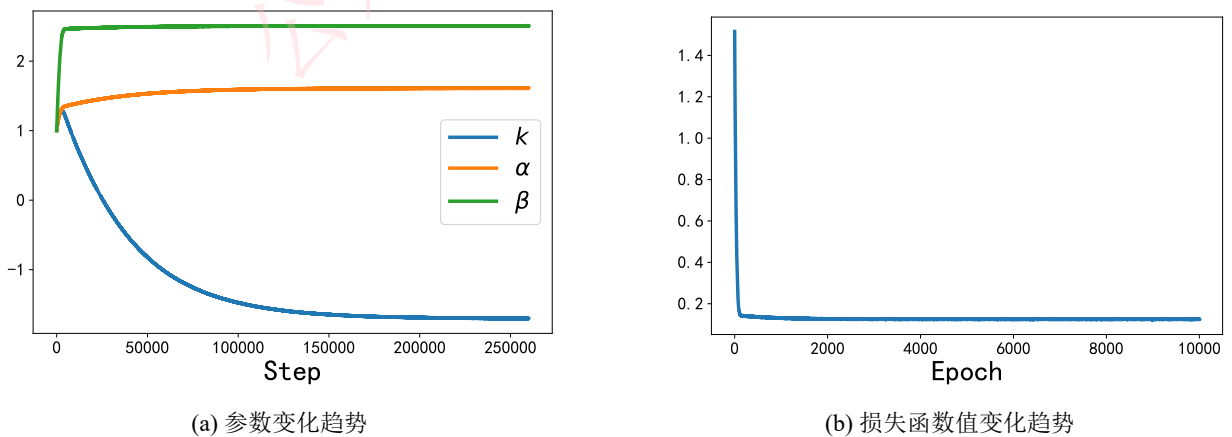


图 5.12 使用 NAdam 求解 LSE 方程过程

图 5.12b 中横轴 Epoch 与图 5.12a 中横轴 Step 分别代表迭代次数与其对应的批处理数目。图 5.12 中可以容易看出损失函数值随着迭代次数的增加而迅速降低，并在迭代次数增

加到 1000 次时已经趋近于稳定，而参数 $\ln(k_1)$ 先随着批处理量的增加上升而后缓慢降低并收敛到常数 -1.70 ，参数 α_1 和 β_1 则是迅速升高并分别收敛到常数 1.61 和 2.51 。

因此使用 NAdam 优化算法求解得到最优的 LSE 方程的表达式为

$$\ln(P) = -1.7 + 1.61 \cdot \ln(f) + 2.51 \cdot \ln(B_m) \quad (5.48)$$

将位于对数空间上的 LSE 方程映射到原始数据空间得到对应的 SE 方程的表达式为

$$P = 0.18 \cdot f^{1.61} \cdot B_m^{2.51} \quad (5.49)$$

其中参数满足一般性条件 $1 < \alpha_1 = 1.61 < 3$ 和 $2 < \beta_1 = 2.51 < 3$ 。下面通过绘图以更加直观体现拟合效果，其中图 5.13 表示所拟合出的 LSE 方程（式 5.48）对应的平面与对数空间上样本的对应关系。

另外在训练集和测试集上的预测结果如表 5.9 所示 从表 5.9 可以看出，拟合结果有较高的拟合优度 (R^2) 以及解释方差得分 (EVS)，且平均相对误差 (MRE) 在 0.30 附近，这些指标说明模型的拟合效果较为一般，但相较于 5.2.1 节中 SE 方程的预测效果更好。

C. 结果与分析（最小二乘法）

在训练集上通过分别令损失函数 $\mathcal{L}$ 关于 $\ln(k_1)$ 、 α_1 和 β_1 各自的偏导为零，最终得到 LSE 方程的参数拟合结果见下表 5.10 所示。

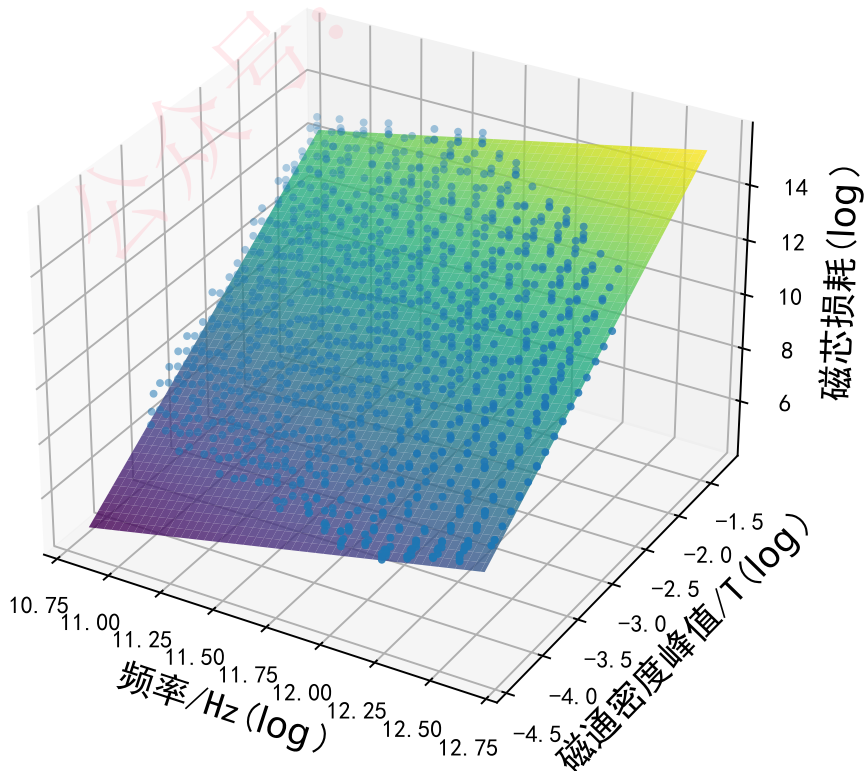


图 5.13 对数空间上的拟合结果

表 5.9 使用 NAdam 求解 LSE 方程的预测性能

	$R^2 \uparrow$	MRE↓	MAE↓	MSE↓	RMSE↓	MSLE↓	EVS↑
训练集	0.937	0.290	2.017E+4	1.944E+9	4.409E+4	0.125	0.939
测试集	0.931	0.305	1.847E+4	1.654E+9	4.067E+4	0.132	0.932

表 5.10 使用最小二乘法求解 LSE 方程的拟合结果

	系数	标准误差	T 统计量	p-value	区间下界	区间上界
常数项 $\ln(k_1)$	-1.70	0.277	-6.166	0.000	-2.248	-1.162
参数项 α_1	1.61	0.025	64.946	0.000	1.562	1.659
参数项 β_1	2.51	0.016	160.063	0.000	2.476	2.537

由表 5.10 可以看出 LSE 方程拟合结果的三个系数检验统计量对应的 p 值均小于显著性水平 0.05，拒绝参数显著为零的原假设，因此可以得到 LSE 方程的表达式为

$$\ln(P) = -1.7 + 1.61 \cdot \ln(f) + 2.51 \cdot \ln(B_m) \quad (5.50)$$

并将在对数空间上的 LSE 方程还原到原始数据空间中，得到对应 SE 方程的表达式为

$$P = 0.18 \cdot f^{1.61} \cdot B_m^{2.51} \quad (5.51)$$

其中参数满足一般性条件 $1 < \alpha_1 = 1.61 < 3$ 和 $2 < \beta_1 = 2.51 < 3$ 。

另外在训练集和测试集上的预测性能如下表 5.11 所示。

表 5.11 使用最小二乘法求解 LSE 方程的预测性能

	$R^2 \uparrow$	MRE ↓	MAE ↓	MSE ↓	RMSE ↓	MSLE ↓	EVS ↑
训练集	0.939	0.294	2.021E+4	1.898E+9	4.102E+4	0.125	0.940
测试集	0.932	0.310	1.866E+4	1.640E+9	3.838E+4	0.132	0.932

从表 5.11 可以看出，拟合结果同样有较高的拟合优度 (R^2) 以及解释方差得分 (EVS)，但平均相对误差 (MRE) 在 0.30 附近并且均方根误差 (RMSE) 较大，这些指标说明模型的预测性能较为一般。

使用最小二乘拟合和使用 NAdam 优化算法迭代求解 LSE 方程参数相比，两者的预测效果类似，但闭式解相较于数值解的求解计算量以及训练时间更少，因此接下来引入温度 T 的函数进入 SE 方程的拟合方式主要采用最小二乘法。

5.2.3 修正 2: TLSE — 考虑温度后的 LSE 方程

颜冲等人^[1]介绍了直流偏置磁场对磁芯损耗的影响, 由于 SE 方程中并没有考虑这个因素, 于是他们采用了一种最简单的方法就是在 SE 方程中乘一个直流偏置修正系数 k_B , 也就是说

$$P = k_B \cdot k_1 \cdot f^{\alpha_1} \cdot B_m^{\beta_1}$$

其中修正系数 k_B 是关于直流偏置的函数。另外飞利浦公司研究了温度对 3F3 材料磁芯损耗的影响, 和 [18] 类似, 他们同样考虑在 SE 方程上乘一个温度修正系数 k_T , 具体为

$$P = k_T \cdot k_1 \cdot f^{\alpha_1} \cdot B_m^{\beta_1}$$

其中修正系数 $k_T = c_1 - c_2T + c_3T$ 是关于温度 T 的一元一次函数。

因此这里也考虑构建新的温度修正系数 k_T , 并通过将其乘到 SE 方程上以引入温度这个因素。也就是说构建的修正 SE 方程和式有一致的结构。由于 5.2.2 节中对数空间上的样本点显然服从同平面结构 (见图 5.13), 为了使等式两边取 $\ln$ 后保持方程线性结构一致性, 本文考虑以下三种温度 T 的函数作为修正系数, 包括 $k_T = T^{\gamma_1}$ 、 $k_T = e^{\gamma_1 T}$ 和 $k_T = e^{\gamma_1 T + c_1}$ 。

(1) TLSE₁ — 温度修正模型一

第一种方法是采用 $k_T = T^{\gamma_1}$ 作为修正系数, 修正后的 TSE₁ 方程为

$$P = k_1 \cdot f^{\alpha_1} \cdot B_m^{\beta_1} \cdot T^{\gamma_1} \quad (5.52)$$

为了使用最小二乘法估计方程中的待估参数 k_1 、 α_1 、 β_1 和 γ_1 , 需要将 TSE₁ 方程转化为多元线性方程, 具体的 TLSE₁ 方程写作

$$\ln(P) = \ln(k_1) + \alpha_1 \cdot \ln(f) + \beta_1 \cdot \ln(B_m) + \gamma_1 \cdot \ln(T) \quad (5.53)$$

在训练集上使用最小二乘法使得 TLSE₁ 方程的损失函数最小化, 通过分别令损失函数 J 关于 $\ln(k_1)$ 、 α_1 、 β_1 和 γ_1 各自的偏导为零, 最终得到此多元线性模型的拟合结果如表 5.12 所示。由表 5.12 可以看出 TLSE₁ 方程拟合结果的四个系数检验统计量对应的 p 值均小于显著性水平 0.05, 拒绝参数显著为零的原假设, 因此可以得到 TLSE₁ 方程的表达式为

$$\ln(P) = 0.49 + 1.64 \cdot \ln(f) + 2.52 \cdot \ln(B_m) - 0.61 \cdot \ln(T) \quad (5.54)$$

并将在对数空间上的 TLSE₁ 方程还原到原始数据空间中, 得到原本 TSE₁ 方程的表达式为

$$P = 1.62 \cdot f^{1.64} \cdot B_m^{2.52} \cdot T^{-0.61} \quad (5.55)$$

其中参数满足一般性条件 $1 < \alpha_1 = 1.64 < 3$ 和 $2 < \beta_1 = 2.52 < 3$ 。

表 5.12 使用最小二乘法求解 TLSE₁ 方程的拟合结果

	系数	标准误差	T 统计量	p-value	区间下界	区间上界
常数项 $\ln(k_1)$	0.49	0.160	3.034	0.002	0.171	0.799
参数项 α_1	1.64	0.014	119.855	0.000	1.609	1.663
参数项 β_1	2.52	0.009	292.823	0.000	2.508	2.542
参数项 γ_1	-0.61	0.014	-44.291	0.000	-0.641	-0.587

表 5.13 使用最小二乘法求解 TLSE₁ 方程的预测性能

	$R^2 \uparrow$	$MRE \downarrow$	$MAE \downarrow$	$MSE \downarrow$	$RMSE \downarrow$	$MSLE \downarrow$	$EVS \uparrow$
训练集	0.976	0.151	1.098E+4	7.296E+8	2.701E+4	0.038	0.976
测试集	0.979	0.168	9.828E+3	5.168E+8	2.273E+4	0.049	0.979

另外在训练集和测试集上的预测结果如表 5.13 所示。从表 5.13 中可以看出拟合优度 (R^2) 以及解释方差得分 (EVS) 非常接近 1, 且平均相对误差 (MRE) 仅在 0.15 附近, 这些指标说明模型的拟合效果较改进前更好。

(2) TLSE₂ — 温度修正模型二

第二种方法是采用 $k_T = e^{\gamma_1 T}$ 作为修正系数, 修正后的 TSE₂ 方程为

$$P = k_1 \cdot f^{\alpha_1} \cdot B_m^{\beta_1} \cdot e^{\gamma_1 T} \quad (5.56)$$

为了使用最小二乘法估计方程中的待估参数 k_1 、 α_1 、 β_1 和 γ_1 , 需要将 TSE₂ 方程转化为多元线性方程, 具体的 TLSE₂ 方程写作

$$\ln(P) = \ln(k_1) + \alpha_1 \cdot \ln(f) + \beta_1 \cdot \ln(B_m) + \gamma_1 \cdot T \quad (5.57)$$

在训练集上使用最小二乘法使得 TLSE₂ 方程的损失函数最小化, 通过分别令损失函数 $\mathcal{L}$ 关于 $\ln(k_1)$ 、 α_1 、 β_1 和 γ_1 各自的偏导为零, 最终得到此多元线性模型的拟合结果如表 5.14 所示。

由表 5.14 可以看出 TLSE₂ 方程拟合结果的四个系数检验统计量对应的 p 值均小于显著性水平 0.05, 拒绝参数显著为零的原假设, 因此可以得到 TLSE₂ 方程的表达式为

$$\ln(P) = -1.22 + 1.63 \cdot \ln(f) + 2.53 \cdot \ln(B_m) - 0.01 \cdot T \quad (5.58)$$

并将在对数空间上的 TLSE₂ 方程还原到原始数据空间中, 得到原本 TSE₂ 方程的表达式为

$$P = -1.22 \cdot f^{1.63} \cdot B_m^{2.53} \cdot e^{-0.01T} \quad (5.59)$$

表 5.14 使用最小二乘法求解 TLSE₂ 方程的拟合结果

	系数	标准误差	T 统计量	p-value	区间下界	区间上界
常数项 $\ln(k_1)$	-1.22	0.158	-7.760	0.000	-1.533	-0.914
参数项 α_1	1.63	0.014	115.833	0.000	1.607	1.663
参数项 β_1	2.53	0.009	283.297	0.000	2.509	2.544
参数项 γ_1	-0.01	0.000	-42.188	0.000	-0.013	-0.012

其中参数满足一般性条件 $1 < \alpha_1 = 1.63 < 3$ 和 $2 < \beta_1 = 2.53 < 3$ 。

另外在训练集和测试集上的预测结果如下表 5.15 所示。

表 5.15 使用最小二乘法求解 TLSE₂ 方程的预测性能

	$R^2 \uparrow$	$MRE \downarrow$	$MAE \downarrow$	$MSE \downarrow$	$RMSE \downarrow$	$MSLE \downarrow$	$EVS \uparrow$
训练集	0.979	0.155	1.148E+4	6.602E+8	2.569E+4	0.040	0.979
测试集	0.979	0.177	1.044E+4	5.020E+8	2.241E+4	0.056	0.979

从表 5.15 中可以看出拟合优度 (R^2) 以及解释方差得分 (EVS) 同样非常接近 1, 且平均相对误差 (MRE) 仅在 0.16 附近, 这些指标说明模型的拟合效果较好。

(3) TLSE₃ — 温度修正模型三

第三种方法是采用 $k_T = e^{\gamma_1 T + c_1}$ 作为修正系数, 修正后的 TSE₃ 方程为

$$P = k_1 \cdot f^{\alpha_1} \cdot B_m^{\beta_1} \cdot e^{\gamma_1 T + c_1} \quad (5.60)$$

为了使用最小二乘法估计方程中的待估参数 k_1 、 α_1 、 β_1 、 γ_1 和 c_1 , 需要将 TSE₃ 方程转化为多元线性方程, 具体的 TLSE₃ 方程写作

$$\ln(P) = \ln(k_1) + \alpha_1 \cdot \ln(f) + \beta_1 \cdot \ln(B_m) + \gamma_1 \cdot T + c_1 \quad (5.61)$$

值得注意的是, 式中 $\ln(k_1) + c_1$ 可以合并为一项, 这意味着在原始空间中可以将 $k_1 \cdot e^{c_1}$ 视作整体, 此时 TLSE₃ 方程与 TLSE₂ 方程等价, 进而预测结果也会与 TLSE₂ 方程保持一致, 故此处不再进行过多赘述。

5.2.4 修正 3: TLSE_{DSR} — 基于深度自表示的 TLSE 方程

在 5.2.3 节三种温度修正方程 TLSE₁、TLSE₂ 和 TLSE₃ 的预测性能都较为理想, 其中 TLSE₁ 略优于其它两种, 此处将表现最优的 TLSE₁ 方程推广到更为复杂的情况。

(1) 模型的建立

假设 TSE_1 方程中的参数 k_1 、 α_1 、 β_1 和 γ_1 均与频率 f 、磁通密度的峰值 B_m 和温度 T 有关，得到基于深度自表示的 TSE 方程（Deep Self-Reputation Steinmetz Equation with Temperature, TSE_{DSR} ）表达式为

$$P = \ddot{k}_1 \cdot f^{\ddot{\alpha}_1} \cdot B_m^{\ddot{\beta}_1} \cdot T^{\ddot{\gamma}_1} \quad (5.62)$$

其中参数 $\ddot{k}_1$ 、 $\ddot{\alpha}_1$ 和 $\ddot{\beta}_1$ 均为关于 f 、 B_m 和 T 的函数。 TSE_1 方程映射到对数空间上得到对应的深度自表示的 TLSE 方程（Deep Self-Reputation Ln Steinmetz Equation with Temperature, TLSE_{DSR} ）表达式为

$$\ln P = \ln \ddot{k}_1 + \ddot{\alpha}_1 \cdot \ln f + \ddot{\beta}_1 \cdot \ln B_m + \ddot{\gamma}_1 \cdot \ln T \quad (5.63)$$

由于函数结构未知，因此使用深度全连接网络^[20]（其拓扑结构如图 5.14 所示）习得 TLSE_{DSR} 方程中的参数，可以近似表示为

$$\ln \ddot{k}_1 = w_{k12} \cdot \text{Tanh}(w_{k12} \cdot \text{Concat}(f, B_m, T)) \quad (5.64)$$

$$\ddot{\alpha}_1 = w_{\alpha12} \cdot \text{Tanh}(w_{\alpha12} \cdot \text{Concat}(f, B_m, T)) \quad (5.65)$$

$$\ddot{\beta}_1 = w_{\beta12} \cdot \text{Tanh}(w_{\beta12} \cdot \text{Concat}(f, B_m, T)) \quad (5.66)$$

其中， $\text{Concat}(f, B_m, T) \in \mathbb{R}^{3 \times 1}$ 表示将三个元素相拼接， $w_1 \in \mathbb{R}^{Latent \times 2}$ 和 $w_2 \in \mathbb{R}^{1 \times Latent}$ 是可学习的参数， Tanh 为激活函数。通过这种方式计算得到的参数是来自于 f 、 B_m 和 T 自身，因此本文将其称之为基于深度自表示^[21]的方法。此模型的损失函数可定义为

$$\begin{aligned} \mathcal{L} &= \mathbb{E}\{(\ln P - \ln \hat{P})^2\} \\ &= \mathbb{E}\{(\ln P - \ln \ddot{k}_1 - \ddot{\alpha}_1 \cdot \ln f - \ddot{\beta}_1 \cdot \ln B_m - \ddot{\gamma}_1 \cdot \ln T)^2\} \end{aligned} \quad (5.67)$$

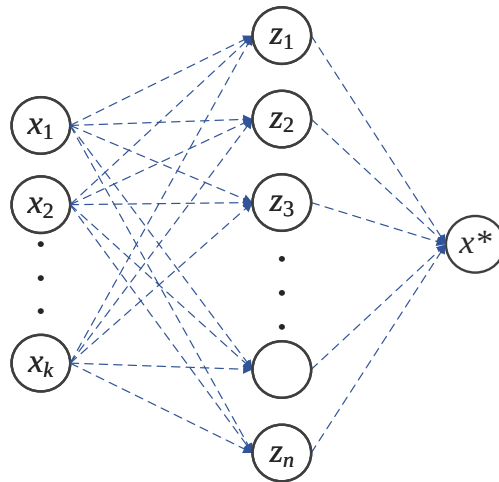


图 5.14 网络拓扑结构示意图

(2) 模型的求解及分析

A. 实验环境和参数设置说明

该模型的实验环境与前一节相同。除了设置 $\mathbb{R}^{3 \times 1}$ 大小的输入层，维度为 $Latent = 128$ 的单隐含层， $\mathbb{R}^{1 \times 1}$ 大小的输出层以及迭代次数为 2500 次外其余参数设置与前一节相同。

B. 结果与分析

与 5.2.1 节类似，使用 NAdam 优化算法求解 $TLSE_{DSR}$ 方程中的参数。在训练集上使用 NAdam 算法通过迭代使得损失函数最小化，损失函数值以及待估参数随迭代次数与其对应的批处理量增加的变化趋势如下图 5.15 所示。

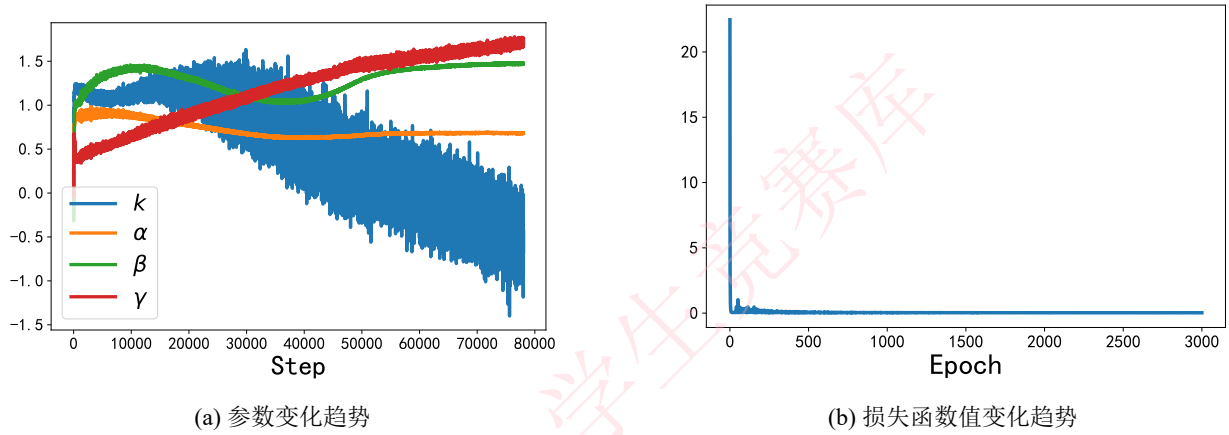


图 5.15 使用 NAdam 求解 $TLSE_{DSR}$ 方程时参数变化趋势

图 5.15b 中横轴 Epoch 与图 5.15a 中横轴 Step 分别代表迭代次数与其对应的批处理数目。容易看出损失函数值随着迭代次数的增加而迅速降低，并在迭代次数增加到 500 次时已经趋近于稳定，而参数 $\ln \ddot{k}_1$ 先随着批处理量的增加迅速上升后缓慢上升随即缓慢降低，参数 $\ddot{\alpha}_1$ 在迅速上升后缓慢下降并相对趋于稳定，参数 $\ddot{\beta}_1$ 在迅速上升后呈正弦波动随即相对趋于稳定，最后参数 $\ddot{\gamma}_1$ 则是迅速升高后持续缓慢上升。图 5.15 中四个参数都呈现着不同程度的震荡，这是因为参数受到其内部变量 f 、 B_m 和 T 的控制而不断变化，因此无法得到 $TLSE_{DSR}$ 方程的显式表达式。

截止至批处理量为 80000 时，在训练集和测试集上的预测性能如表 5.16 所示。

表 5.16 使用 NAdam 求解 $TLSE_{DSR}$ 方程的预测性能

	$R^2 \uparrow$	$MRE \downarrow$	$MAE \downarrow$	$MSE \downarrow$	$RMSE \downarrow$	$MSLE \downarrow$	$EVS \uparrow$
训练集	0.995	0.112	6.040E+3	1.629E+8	1.276E+4	0.023	0.995
测试集	0.996	0.128	5.151E+4	1.081E+8	1.040E+4	0.030	0.996

从表 5.16 可以看出，拟合的拟合优度 (R^2) 以及解释方差得分 (EVS) 非常接近 1，且平

均相对误差 (MRE) 仅在 0.12 附近, 这些指标说明模型有着较好的拟合效果。

5.2.5 总结

5.2.1 至 5.2.4 节中主要研究了四种 SE 方程, 包括 SE 方程、LSE 方程、TLSE 方程和 $TLSE_{DSR}$ 方程, 其中 LSE 方程采用了两种参数求解方式, 以及 TLSE 方程中关于温度 T 的函数研究了三种形式。以上各方程的预测性能汇总如表 5.17 所示。

表 5.17 各 SE 方程的预测性能

		$R^2 \uparrow$	$MRE \downarrow$	$MAE \downarrow$	$MSE \downarrow$	$RMSE \downarrow$	$MSLE \downarrow$	$EVS \uparrow$
SE	训练集	0.946	0.345	2.096E+4	1.682E+9	4.102E+4	0.138	0.946
	测试集	0.939	0.358	1.955E+4	1.473E+9	3.838E+4	0.143	0.939
LSE_1	训练集	0.937	0.290	2.017E+4	1.944E+9	4.409E+4	0.125	0.939
	测试集	0.931	0.305	1.847E+4	1.654E+9	4.067E+4	0.132	0.932
LSE_2	训练集	0.939	0.294	2.021E+4	1.898E+9	4.102E+4	0.125	0.940
	测试集	0.932	0.310	1.866E+4	1.640E+9	3.838E+4	0.132	0.932
$TLSE_1$	训练集	0.976	0.151	1.098E+4	7.296E+8	2.701E+4	0.038	0.976
	测试集	0.979	0.168	9.828E+3	5.168E+8	2.273E+4	0.049	0.979
$TLSE_{2/3}$	训练集	0.979	0.155	1.148E+4	6.602E+8	2.569E+4	0.040	0.979
	测试集	0.979	0.177	1.044E+4	5.020E+8	2.241E+4	0.056	0.979
$TLSE_{DSR}$	训练集	0.995	0.112	6.040E+3	1.629E+8	1.276E+4	0.023	0.995
	测试集	0.996	0.128	5.151E+4	1.081E+8	1.040E+4	0.030	0.996

由表 5.17 可以看出, 将经典的 SE 方程线性化为 LSE 方程有更低的平均相对误差 (MRE), 并且 LSE 方程分别通过 NAdam 优化算法和最小二乘法拟合得到的预测性能基本一致。为了延续 LSE 预测性能, 减少计算复杂度, 并引入温度 T , 这里研究了三种不同的温度修正系数并使用最小二乘法求解 TLSE 方程中的未知参数, 其中第二与第三种方法经过分析发现等价。结果显示 TLSE 方程相较于 LSE 方程显著提高 R^2 并且有更低的平均相对误差 (MRE), 说明将温度能够对磁芯损耗造成较大的影响, 其中 $TLSE_1$ 方程的表现最好。此外本文还研究了 $TLSE_1$ 方程的参数会随频率 f 、磁通密度峰值 B_m 和温度 T 改变的情况, 并将其称之为 $TLSE_{DSR}$ 方程并使用全连接神经网络学习参数, 最后 $TLSE_{DSR}$ 方程的 R^2 已经接近 1 并且平均相对误差 (MRE) 仅有 0.12 左右。

5.3 问题三的模型建立与求解

问题三的主要思路即是先通过多因素方差分析确定哪些外界条件及其协同作用会对磁芯损耗造成影响, 并将会显著影响磁芯损耗的条件各自进行单因素方差分析以量化各自

的影响程度。最后使用独立样本非参数检验遍历各个条件水平寻出最低的磁芯损耗所对应的外界条件。

5.3.1 多因素方差分析判别显著影响条件

(1) 多因素方差分析

多因素方差分析^[22] 研究多个分类变量的不同水平下，响应的均值是否存在显著性差异。它不仅可以用于分析单个变量对响应的影响（主效应），也可以分析多个变量协同作用对响应的影响（交互效应）。

以本题中三因素方差分析为例，需要考虑两两之间变量的交互作用。设定一个包含主效应和交互作用的线性模型。对于三个变量温度 T 、励磁波形 W 和材料 M ，模型表示为

$$P = \mu + a_i + b_j + c_k + (ab)_{ij} + (ac)_{ik} + (bc)_{jk} + \epsilon_{ijkl} \quad (5.68)$$

其中磁芯损耗 P 为响应， μ 为总体均值， a_i 、 b_j 和 c_k 分别是 T 、 W 和 M 的效应， $(ab)_{ij}$ 、 $(ac)_{ik}$ 和 $(bc)_{jk}$ 分别是 T 与 W 、 T 与 M 和 W 与 M 的交互作用效应。

多因素方差分析提出的原假设为各变量在不同水平下观测到总体的均值无显著差异，也就是

$$H_0 : a_i = b_j = c_k = (ab)_{ij} = (ac)_{ik} = (bc)_{jk} = 0$$

使用 F 检验来评估各效应的显著性，计算 F 统计量的值并与临界值比较，以决定是否拒绝原假设 H_0 。

(2) 模型的求解及分析

A. 实验环境

本文实验的硬件环境为 Intel(R) Core(TM) i7-13700K@3.40 GHz，64G DDR5 内存及 GeForce RTX 4090 D（24GB 显存）的 64 位计算机。软件平台为 SPSS 27。

B. 结果与分析

使用附件一中的样本数据对上述多因素方差分析求解，得到主效应及交互效应显著性的结果如表 5.18 所示。由表 5.18 可以看出三种主效应 a_i 、 b_j 和 c_k 与三种交互效应 $(ab)_{ij}$ 、 $(ac)_{ik}$ 和 $(bc)_{jk}$ 求得 F 统计量对应的 p 值依次为 0.001、0.001、0.001、0.105、0.604、0.001，其中温度、励磁波形、材料与励磁波形和材料的交互作用的 p 值均小于显著性水平 0.05，说明这四种因素均对磁芯损耗有显著影响。

5.3.2 单因素方差分析量化显著影响条件

(1) 单因素方差分析

为了衡量四种因素中温度、励磁波形、材料与励磁波形和材料的交互作用各自对磁芯损耗的影响程度，可以使用单因素方差^[23] 衡量单个因素的影响。

表 5.18 多因素方差分析效应检验

	III 类平方和	自由度	均方	F 统计量	p-value
修正	1.416E+14	29	4.882E+12	37.403	< 0.001
截距	4.370E+14	1	4.370E+14	3348.111	0.000
温度	4.370E+12	3	2.940E+12	22.527	< 0.001
励磁波形	7.541E+13	2	3.770E+13	288.891	< 0.001
材料	3.690E+13	3	1.230E+13	94.240	< 0.001
温度 + 励磁波形	1.371E+12	6	2.285E+11	1.751	0.105
温度 + 材料	9.552E+11	9	1.061E+11	0.813	0.604
励磁波形 + 材料	1.224E+13	6	2.041E+12	15.636	< 0.001
误差	1.614E+15	12370	1.305E+11		
总计	2.247E+15	12400			
修正总计	1.756E+15	12399			

以温度 T 进行单因素方差分析为例，设定一个包含效应 a_i 的线性模型，具体表示为：

$$P = \mu + a_i + \epsilon_{ij} \quad (5.69)$$

其中磁芯损耗 P 为响应， μ 为总体均值。单因素方差分析提出的原假设为各变量在不同水平下观测到总体的均值无显著差异，也就是

$$H_0 : a_i = 0$$

单因素方差分析通过比较组间和组内的离差平方和来判断组均值是否存在显著差异。组间离差平方和（ SSA ）是各组均值与总体均值之间差异的平方和，反映了因子水平对因变量的影响，组内离差平方和（ SSE ）是每个观测值与其所在组的均值之间差异的平方和，反映了组内个体的随机离差，总离差平方和（ SST ）是所有观测值与总体均值之间差异的平方和，这三个指标之间满足

$$SST = SSA + SSE \quad (5.70)$$

并且总自由度、组间自由度和组内自由度分别为 $n - 1$ 、 $k - 1$ 和 $n - k$ ，其中 n 为样本数， k 为组数。随即使用 F 检验来评估效应 a_i 的显著性，计算 F 统计量

$$F = \frac{SSA/(k - 1)}{SSE/(n - k)}$$

的取值并与临界值比较，以决定是否拒绝原假设 H_0 。

(2) 模型的求解及分析

A. 实验环境

该模型的实验环境与前一节相同。

B. 结果与分析

以温度作为因素进行单因素方差分析并进行求解，得到效应 a_i 的显著性结果如下表 5.19 所示

表 5.19 温度单因素方差分析效应检验

	III 类平方和	自由度	均方	F 统计量	p-value
修正	7.513E+12	3	2.504E+12	17.755	< 0.001
截距	4.875E+14	1	4.875E+14	3456.213	0.000
温度	7.513E+12	3	2.504E+12	17.755	< 0.001
误差	1.748E+15	12396	1.411E+11		
总计	2.247E+15	12400			
修正总计	1.756E+15	12399			

由表 5.19 可以看出效应 a_i 的 F 统计量值为 3456.213，对应的 p 值为 0.001 小于显著性水平 0.05，故温度对磁芯损耗有非常显著的影响。

以励磁波形作为因素进行单因素方差分析并进行求解，得到效应 b_j 的显著性结果如下表 5.20 所示

表 5.20 励磁波形单因素方差分析效应检验

	III 类平方和	自由度	均方	F 统计量	p-value
修正	7.520E+13	2	3.760E+13	277.335	< 0.001
截距	4.790E+14	1	4.790E+14	3533.169	0.000
励磁波形	7.520E+13	2	3.760E+13	277.335	< 0.001
误差	1.681E+15	12397	1.356E+11		
总计	2.247E+15	12400			
修正总计	1.756E+15	12399			

由表 5.20 可以看出效应 b_j 的 F 统计量值为 277.335，对应的 p 值为 0.001 小于显著性水平 0.05，故励磁波形对磁芯损耗也有非常显著的影响。

以材料作为因素进行单因素方差分析并进行求解，得到效应 c_k 的显著性结果如下表 5.21 所示

表 5.21 材料单因素方差分析效应检验

	III 类平方和	自由度	均方	F 统计量	p-value
修正	4.114E+13	3	1.371E+13	99.121	< 0.001
截距	4.789E+14	1	4.789E+14	3461.567	0.000
材料	4.114E+13	3	1.371E+13	99.121	< 0.001
误差	1.715E+15	12396	1.383E+11		
总计	2.247E+15	12400			
修正总计	1.756E+15	12399			

由表 5.21 可以看出效应 c_k 的 F 统计量值为 99.121，对应的 p 值为 0.001 小于显著性水平 0.05，故材料对磁芯损耗也有非常显著的影响。

以励磁波形和材料的协同作为因素进行单因素方差分析并进行求解，得到效应 $(bc)_{jk}$ 的显著性结果如下表 5.22 所示

表 5.22 励磁波形和材料的协同单因素方差分析效应检验

	III 类平方和	自由度	均方	F 统计量	p-value
修正	1.300E+14	11	1.182E+13	90.036	< 0.001
截距	4.395E+14	1	4.395E+14	3348.349	0.000
励磁波形 + 材料	1.300E+14	11	1.182E+13	90.036	< 0.001
误差	1.626E+15	12388	1.313E+11		
总计	2.247E+15	12400			
修正总计	1.756E+15	12399			

由表 5.22 可以看出效应 $(bc)_{jk}$ 的 F 统计量值为 90.036，对应的 p 值为 0.001 小于显著性水平 0.05，故励磁波形和材料的协同对磁芯损耗也有非常显著的影响。

上述四种因素温度、励磁波形、材料以及励磁波形和材料协同作用的效应的 P 值均为 0.001，都对磁芯损耗有着非常显著的影响。为了进一步刻画这四种因素对磁芯损耗的影响强弱，可以参考对应的 F 统计量，F 统计量的值越大，说明组内方差与组间方差的差异越大，也就是说该因素对响应的影响越显著。由于温度、励磁波形、材料以及励磁波形和材料协同作用的 F 统计量的值依次为 17.755、277.335、99.121 和 90.036，说明励磁波形对磁芯损耗的影响程度最大，材料次之，而温度对磁芯损耗的影响程度最小。

5.3.3 独立样本非参数检验寻找最优条件

由于温度有四个水平（25，50，70，90）、励磁波形有三个水平（正弦波，三角波，梯形波）、材料有四个水平（材料 1，材料 2，材料 3，材料 4），因此附件一中 12400 个样本可根据不同的温度、励磁波形和材料被分为 48 组。接下来要找出此 48 组中磁芯损耗最低的组别，并确定该组对应的外界条件。

这里采用遍历的方法对比任意两组之间的磁芯损耗之间是否有显著性差异。由于分组后样本不一定满足正态性与方差齐性，因此无法使用独立样本均值检验^[24]，本文考虑使用一种非参数方法 Mann-Whitney U 检验^[25] 进行评估。

(1) Mann-Whitney U 检验

Mann-Whitney U 检验的目标是判断两组独立样本是否具有相同的分布，它仅要求样本之间相互独立以及数据具有可比性。Mann-Whitney U 具体通过比较两组样本的排序来评估其分布差异，它首先将来自两组样本的所有数据合并，并按照大小排序，计算每组样本中数据的秩之和。假设有两个样本组 1 和组 2，那么可以分别得到两组的 U 统计量

$$U_1 = n_1 \times n_2 + \frac{n_1(n_1 + 1)}{2} - R_1$$
$$U_2 = n_1 \times n_2 + \frac{n_2(n_2 + 1)}{2} - R_2$$

其中 n_1 和 n_2 分别为组 1 和组 2 中的样本个数， R_1 和 R_2 分别为组 1 和组 2 的秩和。从 U_1 和 U_2 中选取较小的值与标准的 U 分布表进行对比，以确定是否拒绝原假设。

(2) 模型的建立

这里将温度为 25 摄氏度、励磁波形为正弦波、材料为材料 1 的样本集记为组 1，温度为 25 摄氏度、励磁波形为正弦波、材料为材料 2 的样本集记为组 2，...，温度为 25 摄氏度、励磁波形为三角波、材料为材料 1 的样本集记为组 3，以此类推共计 48 组样本集。而后固定一个样本集 A，使用下一个样本集 B 与之进行 Mann-Whitney U 检验，若 AB 两组之间有显著性差异且 B 的平均秩更小则固定样本集 B，使用样本集 C 与 B 进行 Mann-Whitney U 检验，反之则进行下一轮，使用样本集 C 与 A 进行 Mann-Whitney U 检验。此算法只需进行 47 次 Mann-Whitney U 检验即可找出使磁芯损耗最小的外界条件。

(3) 模型的求解及分析

A. 实验环境

该模型的实验环境与前一节相同。

B. 结果与分析

首先固定组 1，并将其同组 2 进行 Mann-Whitney U 检验，检验结果中 U 统计量值为 47812，对应的 p 值为 0.993 大于显著性水平 0.05，说明组 1 与组 2 的磁芯损耗没有显著性差异。将组 1 同组 3 进行 Mann-Whitney U 检验，检验结果中 U 统计量值为 39090，对应的 p 值为 0.700 大于显著性水平 0.05，说明组 1 与组 3 的磁芯损耗同样没有显著性差异。将

组 1 同组 4 进行 Mann-Whitney U 检验，检验结果中 U 统计量值为 23992，对应的 p 值为 0.002 小于显著性水平 0.05，说明组 1 与组 4 的磁芯损耗有显著性差异，并且由下表 5.23 可以看出组 4 中样本的秩平均值小于组 1，因此在组 4 的外界条件下有着更小的磁芯损耗。

表 5.23 组 1 与组 4 中样本的秩

	样本个数	秩平均值	秩的总和
组 1	271	259.47	70316
组 4	212	219.67	46570
总计	483		

固定组 4 并将其同组 5 进行 Mann-Whitney U 检验，检验结果中 U 统计量值为 20826，对应的 p 值为 0.001 小于显著性水平 0.05，说明组 4 与组 5 的磁芯损耗有显著性差异，并且由下表 5.24 可以看出组 4 中样本的秩平均值小于组 5，因此不需要更新固定的样本集。组 4 与组 6 至组 27 进行 Mann-Whitney U 检验均未更新此固定的样本集，直到组 4 与组 28

表 5.24 组 4 与组 5 中样本的秩

	样本个数	秩平均值	秩的总和
组 4	272	204.74	43404
组 5	370	341.21	126249
总计	582		

进行 Mann-Whitney U 检验，检验结果中 U 统计量值为 20348，对应的 p 值为 0.018 小于显著性水平 0.05，说明组 4 与组 28 的磁芯损耗有显著性差异，并且由下表 5.25 可以看出组 28 中样本的秩平均值小于组 4，因此在组 28 的外界条件下有着更小的磁芯损耗。

表 5.25 组 4 与组 28 中样本的秩

	样本个数	秩平均值	秩的总和
组 4	212	231.52	49082
组 28	221	203.07	44879
总计	433		

固定组 28 后，与往后所有组（组 29 至组 48）进行 Mann-Whitney U 检验均无法更新固定的样本集，说明在组 28 的外界条件下磁芯损耗能够达到最小。除此之外，组 28 与组

38 和组 40 进行 Mann-Whitney U 检验检验结果中 U 统计量值对应的 p 值均大于显著性水平 0.05，说明在组 38 与组 40 的外界条件下磁芯损耗同样能够达到最小。综上所述，组 28、38 和 40 分别对应外界条件为温度 70 摄氏度、正弦波、材料 4；温度 90 摄氏度、正弦波、材料 2 和温度 90 摄氏度、正弦波、材料 4。

5.4 问题四的模型建立与求解

在模型二中，本文对同一种磁芯材料、正弦波形下和不同温度时的磁芯损耗进行了建模预测，修正了 SE 方程，达到了良好的预测精度。因此，本文将考虑三类模型（研究及改进路线如图5.16 所示）：损耗分离模型、经验计算模型和深度学习模型，并在模型二和这两种模型的基础上做进一步改进，以适应特殊环境中的磁芯损耗精准预测。

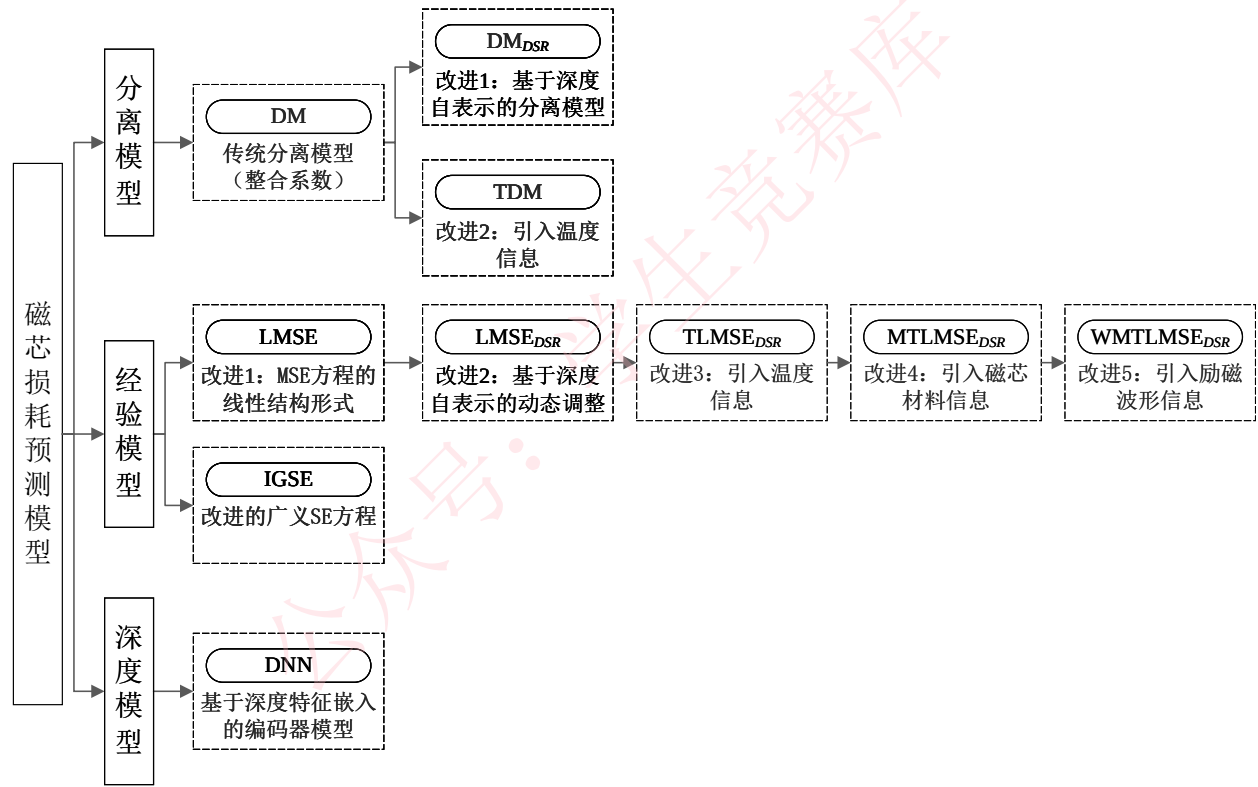


图 5.16 模型四研究及改进路线总览

5.4.1 分离模型：DM — 常规的分离模型

(1) 模型的建立

分离模型^[26]（本文简称为 Detached Model, DM）通常是将磁芯损耗 P 分为磁滞损耗 P_h 、涡流损耗 P_{cl} 和剩余损耗 P_e 这三个部分，即：

$$P = P_h + P_{cl} + P_e \quad (5.71)$$

磁滞损耗是指磁芯材料内部的磁畴展现出不同的响应机制，计算公式如下：

$$P_h = k_h \cdot f \cdot B_m^\beta \quad (5.72)$$

其中， f 和 B_m 分别表示频率和磁通密度的峰值， k_h 和 β 是待估的参数。

涡流损耗是指磁通交变时磁芯中由电流变化所产生的能量消耗，计算公式如下：

$$P_{cl} = k_{cl} \cdot f^2 \cdot B_m^2 \quad (5.73)$$

其中， f 和 B_m 分别表示频率和磁通密度的峰值， k_{cl} 是一个与被测磁芯截面积和电阻率有关的数值。

剩余损耗是指除了磁滞损耗和涡流损耗以外的损耗统称为剩余损耗，与材料的固有属性有关，计算公式如下：

$$P_e = 8 \cdot \sqrt{\sigma \cdot S \cdot G \cdot V_0} \cdot f^{1.5} \cdot B_m^{1.5} \quad (5.74)$$

其中， σ 和 S 分别表示电导率和磁芯的有效截面积， G 是一个常数 0.1356， V_0 是待估参数。在本文所给的数据中，磁芯材料的 σ 和 S 未能给出，因此可以将其系数部分看做一个整体的 k_e ，与 V_0 是一同作为待估参数。那么，磁芯损耗的分离模型可以表示为：

$$P = k_h \cdot f \cdot B_m^\beta + k_{cl} \cdot f^2 \cdot B_m^2 + k_e \cdot f^{1.5} \cdot B_m^{1.5} \quad (5.75)$$

其中 k_h 、 k_{cl} 、 k_e 和 β 是待估参数。对于这个非线性方程的拟合，可以将模型二中引入的 NAdam 算法用于求解，其损失函数为：

$$\begin{aligned} \mathcal{L} &= \mathbb{E}\{(P - \hat{P})^2\} \\ &= \mathbb{E}\{(P - k_h \cdot f \cdot B_m^\beta - k_{cl} \cdot f^2 \cdot B_m^2 - k_e \cdot f^{1.5} \cdot B_m^{1.5})^2\} \end{aligned} \quad (5.76)$$

(2) 模型的求解与分析

A. 实验环境说明

本文实验的硬件环境为 Intel(R) Core(TM) i7-13700K@3.40 GHz，64G DDR5 内存及 GeForce RTX 4090 D (24GB 显存) 的 64 位计算机。软件平台为 Python 3.11 及 Pytorch 2.0.1+cu118。

B. 参数设置

本文以附件一材料 1~4 中的正弦波形共 12400 个样本为研究对象，将其随机分为 80% 的训练集和 20% 的验证集。并且给定四种未知参数 k_h 、 k_{cl} 、 k_e 和 β 的初始值为 200、1、1 和 1，初始学习率设置为 0.001，批处理大小设置为 256，迭代次数设置为 10000 次。

C. 结果与分析

由 NAdam 求解参数的参数值变化曲线和损失函数值曲线如图 5.17 所示。图 5.17 中横

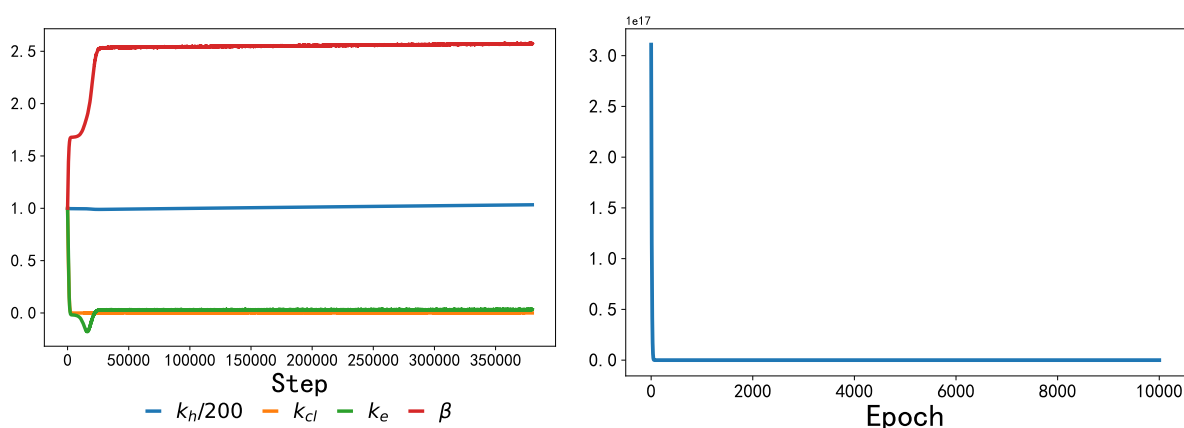


图 5.17 使用 NAdam 求解 DM 模型时参数值（左）和损失函数值（右）变化趋势

轴 Epoch 与 Step 分别代表迭代次数与其对应的批处理数目。由上图可以看出，损失函数随着迭代次数的增加而迅速降低，并在迭代次数增加到 1000 次时已经趋近于稳定，而参数 β 随着批处理数目的增加而迅速上升后收敛到 2.57，参数 k_{cl} 和 k_e 首先迅速下降并趋于 0.00021 和 0.033，参数 k_h 则是相对稳定在 206.73。因此，使用 NAdam 算法求解的最优分离模型为：

$$P = k_h \cdot f \cdot B_m^{2.57} + 0.00021 \cdot f^2 \cdot B_m^2 + 0.033 \cdot f^{1.5} \cdot B_m^{1.5} \quad (5.77)$$

为了更客观地分析 DM 模型的预测性能，表 5.26 给出了模型在不同材料下的指标值。从表 5.26 中可以看出，模型的拟合优度在训练集和验证集上都能达到 0.9 以上，但是在验

表 5.26 DM 模型的预测性能

材料	数据集	$R^2 \uparrow$	$MRE \downarrow$	$MAE \downarrow$	$MSE \downarrow$	$RMSE \downarrow$	$MSLE \downarrow$	$EVS \uparrow$
材料 1	训练集	0.913	1.001	5.681E+04	9.764E+09	9.881E+04	0.550	0.921
	验证集	0.921	0.946	5.654E+04	1.007E+10	1.003E+05	0.513	0.927
材料 2	训练集	0.955	0.829	4.989E+04	7.832E+09	8.850E+04	0.438	0.956
	验证集	0.938	0.831	4.783E+04	8.128E+09	9.015E+04	0.442	0.939
材料 3	训练集	0.942	0.674	5.148E+04	1.243E+10	1.115E+05	0.344	0.944
	验证集	0.940	0.617	5.377E+04	1.378E+10	1.174E+05	0.309	0.943
材料 4	训练集	0.924	0.724	2.852E+04	3.603E+09	6.002E+04	0.392	0.928
	验证集	0.918	0.729	2.597E+04	3.264E+09	5.713E+04	0.392	0.921
训练集均值		0.940	0.812	4.737E+04	8.593E+09	9.270E+04	0.434	0.940
验证集均值		0.935	0.784	4.682E+04	9.022E+09	9.498E+04	0.416	0.935

证集上的相对误差 MRE 达到了 0.78+，说明模型的泛化性能差，预测精度低。

5.4.2 分离模型改进 1: DM_{SDR} — 基于深度自表示的 DM 模型

(1) 模型的建立

与模型二类似，常规的分离模型 DM 把待估参数设置为确定的数值，最终可能导致模型的泛化能力差，预测的精度低。因此，本文考虑将模型改进为基于深度自表示的分离模型（Detached Model with Deep Self-Reputation, DM_{SDR} ），用基于神经网络来提取各个属性之间的深度特征，这种深度学习的方法可以自适应数据属性内部间的分布规律，从而动态地调整系数，达到更精确的预测性能。 DM_{SDR} 的形式可表示为：

$$P = \ddot{k}_h \cdot f \cdot B_m^{\ddot{\beta}} + \ddot{k}_{cl} \cdot f^2 \cdot B_m^2 + \ddot{k}_e \cdot f^{1.5} \cdot B_m^{1.5} \quad (5.78)$$

其中， $\ddot{k}_h$ 、 $\ddot{k}_{cl}$ 、 $\ddot{k}_e$ 和 $\ddot{\beta}$ 是通过深度全连接网络（其拓扑结构如图5.14 所示）得到的，可以近似地表示为：

$$\ddot{k}_h = w_{k_h2} \cdot \text{Tanh}(w_{k_h1} \cdot \text{Concat}(f, B_m)) \quad (5.79)$$

$$\ddot{k}_{cl} = w_{k_{cl}2} \cdot \text{Tanh}(w_{k_{cl}1} \cdot \text{Concat}(f, B_m)) \quad (5.80)$$

$$\ddot{k}_e = w_{k_e2} \cdot \text{Tanh}(w_{k_e1} \cdot \text{Concat}(f, B_m)) \quad (5.81)$$

$$\ddot{\beta} = w_{\beta2} \cdot \text{Tanh}(w_{\beta1} \cdot \text{Concat}(f, B_m)) \quad (5.82)$$

其中， $\text{Concat}(f, B_m)^T \in \mathbb{R}^{2 \times 1}$ 表示将 2 个元素相拼接， $w_1 \in \mathbb{R}^{Latent \times 2}$ 和 $w_2 \in \mathbb{R}^{1 \times Latent}$ 是可学习的参数， $Latent$ 为隐藏层的维度， Tanh 为激活函数。通过这种方式计算得到的系数是来自于 f 和 B_m 自身，因此本文将其称之为基于深度自表示的方法^[21]。因此，模型的损失函数可定义为：

$$\begin{aligned} \mathcal{L} &= \mathbb{E}\{(P - \hat{P})^2\} \\ &= \mathbb{E}\{(P - \ddot{k}_h \cdot f \cdot B_m^{\ddot{\beta}} - \ddot{k}_{cl} \cdot f^2 \cdot B_m^2 - \ddot{k}_e \cdot f^{1.5} \cdot B_m^{1.5})^2\} \end{aligned} \quad (5.83)$$

(2) 模型的求解与分析

A. 实验环境说明

该模型的实验环境与前一节相同。

B. 参数设置

本文以附件一材料 1~4 中的正弦波形共 12400 个样本为研究对象，将其随机分为 80% 的训练集和 20% 的验证集。并且初始学习率设置为 0.001，隐藏层维度 $Latent = 128$ ，批处理大小设置为 256，迭代次数设置为 10000 次。

C. 结果与分析

通过 NAdam 求解参数的参数值变化曲线和损失函数值曲线如图5.18 所示。图5.18 中横轴 Epoch 与 Step 分别代表迭代次数与其对应的批处理数目，从图中可以看出，损失函数随着迭代次数的增加而迅速降低，并在迭代次数增加到 1000 次时已经趋近于稳定，而对

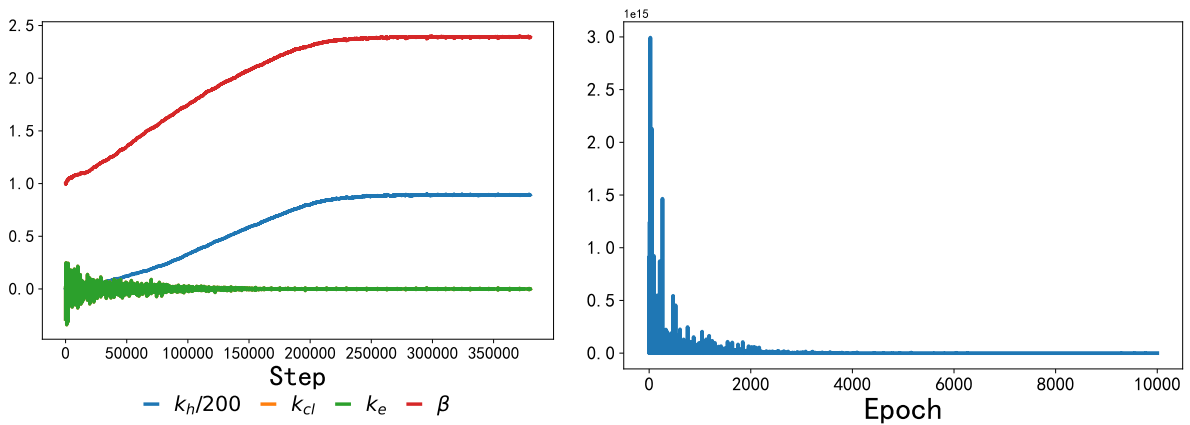


图 5.18 使用 NAdam 求解分离模型 DM_{SDR} 时参数值（左）和损失函数值（右）变化趋势

应的参数 k_h 、 k_{cl} 、 k_e 和 β 最终分别趋近于 178.58、0.00028、0.00028 和 2.39。此外，参数最终并不是一个固定的数值，而是在某个均值处震荡，这是因为模型有对输入数据自适应调整参数的能力。

为了更客观地分析 DM_{SDR} 模型的预测性能，表5.27 给出了模型在不同材料下的指标值。从表5.27 中可以看出，模型的拟合优度在训练集和验证集上都能达到 0.9 以上，但是

表 5.27 DM_{SDR} 模型的预测性能

材料	数据集	$R^2 \uparrow$	$MRE \downarrow$	$MAE \downarrow$	$MSE \downarrow$	$RMSE \downarrow$	$MSLE \downarrow$	$EVS \uparrow$
材料 1	训练集	0.922	0.655	5.257E+04	8.759E+09	9.359E+04	0.343	0.924
	验证集	0.927	0.622	5.294E+04	9.307E+09	9.647E+04	0.324	0.928
材料 2	训练集	0.949	0.452	4.704E+04	8.972E+09	9.472E+04	0.216	0.949
	验证集	0.929	0.445	4.547E+04	9.421E+09	9.706E+04	0.218	0.930
材料 3	训练集	0.926	0.351	5.380E+04	1.580E+10	1.257E+05	0.162	0.933
	验证集	0.927	0.338	5.544E+04	1.699E+10	1.303E+05	0.159	0.934
材料 4	训练集	0.904	0.402	2.960E+04	4.507E+09	6.714E+04	0.252	0.917
	验证集	0.899	0.412	2.654E+04	4.032E+09	6.349E+04	0.263	0.909
训练集均值		0.932	0.470	4.636E+04	9.668E+09	9.832E+04	0.245	0.933
验证集均值		0.927	0.458	4.582E+04	1.013E+10	1.006E+05	0.242	0.929

在验证集上的相对误差 MRE 达到了 0.45+，说明模型具有一定的泛化性能，虽然性能相较于 DM 模型有所改进，尤其对材料 2 的预测相对误差较低，但预测精度仍然低。

5.4.3 分离模型改进 2：TDM — 考虑温度的 DM 模型

(1) 模型的建立

由模型二可以看出，磁芯损耗对温度是较为敏感的，因此本文考虑将温度引入模型

DM 或者 DM_{SDR} ，其形式可表示为：

$$P = P_h \cdot T^{\gamma_1} + P_{cl} \cdot T^{\gamma_2} + P_e \cdot T^{\gamma_3} \quad (5.84)$$

其中， T 表示温度， γ_i 表示对应的待估参数。但是，通过实验发现，该模型引入温度的方式不适用于 DM_{SDR} 。因为，模型所增加的项是幂指数形式 a^x ，在使用 NAdam 求解时，误差反向传播的过程中对 x 求导则是 $a^x \ln a$ ，再加上 x 是由网络计算得到的，容易出现梯度消失或爆炸，导致无法继续学习。因此，本文还是考虑将温度因素引入至模型 DM，即：

$$P = k_h \cdot f \cdot B_m^\beta \cdot T^{\gamma_1} + k_{cl} \cdot f^2 \cdot B_m^2 \cdot T^{\gamma_2} + k_e \cdot f^{1.5} \cdot B_m^{1.5} \cdot T^{\gamma_3} \quad (5.85)$$

其中 k 、 γ 和 β 为待估参数。因此，模型的损失函数可定义为：

$$\begin{aligned} \mathcal{L} &= \mathbb{E}\{(P - \hat{P})^2\} \\ &= \mathbb{E}\{(P - k_h \cdot f \cdot B_m^\beta \cdot T^{\gamma_1} - k_{cl} \cdot f^2 \cdot B_m^2 \cdot T^{\gamma_2} - k_e \cdot f^{1.5} \cdot B_m^{1.5} \cdot T^{\gamma_3})^2\} \end{aligned} \quad (5.86)$$

(2) 模型的求解与分析

A. 实验环境说明

该模型的实验环境与前一节相同。

B. 参数设置

本文以附件一材料 1~4 中的正弦波形共 12400 个样本为研究对象，将其随机分为 80% 的训练集和 20% 的验证集。并且给定七种未知参数 k_h 、 k_{cl} 、 k_e 、 β 、 α_1 、 α_2 和 α_3 的初始值为 200、1、1、1、0.1、0.1 和 0.1，初始学习率设置为 0.001，批处理大小设置为 256，迭代次数设置为 10000 次。

C. 结果与分析

通过 NAdam 求解参数的参数值变化曲线和损失函数值曲线如图 5.19 所示。图 5.19 中横轴 Epoch 与 Step 分别代表迭代次数与其对应的批处理数目，从图中可以看出，损失函数随着迭代次数的增加而迅速降低，并在迭代次数增加到 1000 次时已经趋近于稳定，而对应的参数 k_h 、 k_{cl} 、 k_e 、 β 、 α_1 、 α_2 和 α_3 分别收敛于 229.38、0.01、0.11、2.66、7.32、-1.18 和 -1.18。因此，模型的求解结果为：

$$P = 229.38 \cdot f \cdot B_m^{2.66} \cdot T^{7.32} + 0.01 \cdot f^2 \cdot B_m^2 \cdot T^{-1.18} + 0.11 \cdot f^{1.5} \cdot B_m^{1.5} \cdot T^{-1.18} \quad (5.87)$$

为了更客观地分析 TDM 模型的预测性能，表 5.28 给出了模型在不同材料下的指标值。从表 5.28 中可以看出，模型的拟合优度在训练集和验证集上都能达到 0.9 以上，但是在验证集上的相对误差 MRE 达到了 1.0+，说明模型的预测性能很差，精度低。

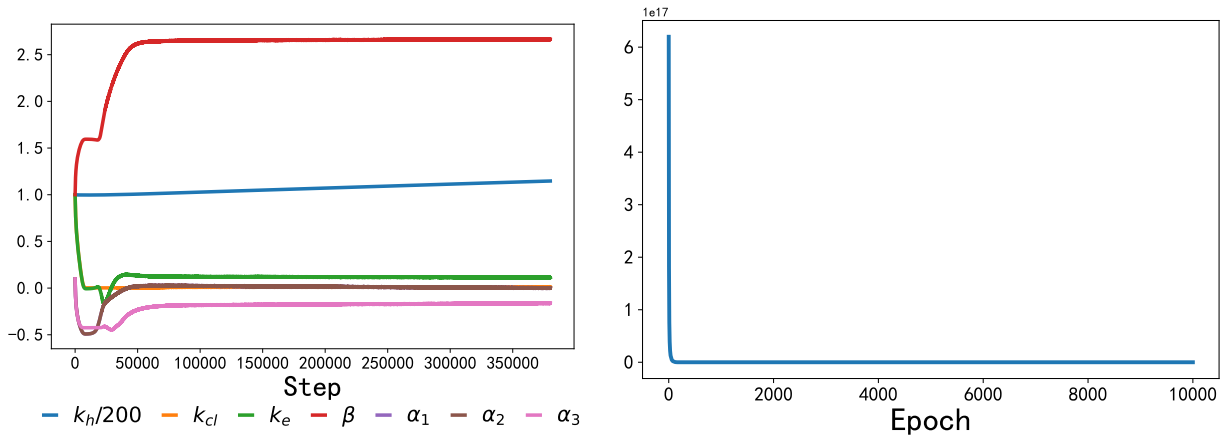


图 5.19 使用 NAdam 求解分离模型 TDM 时参数值（左）和损失函数值（右）变化趋势

表 5.28 TDM 模型的预测性能

材料	数据集	$R^2 \uparrow$	$MRE \downarrow$	$MAE \downarrow$	$MSE \downarrow$	$RMSE \downarrow$	$MSLE \downarrow$	$EVS \uparrow$
材料 1	训练集	0.924	1.336	6.001E+04	8.573E+09	9.259E+04	0.761	0.940
	验证集	0.928	1.284	6.083E+04	9.241E+09	9.613E+04	0.725	0.942
材料 2	训练集	0.969	1.156	4.850E+04	5.449E+09	7.382E+04	0.652	0.973
	验证集	0.959	1.167	4.763E+04	5.407E+09	7.354E+04	0.658	0.962
材料 3	训练集	0.961	0.953	4.870E+04	8.224E+09	9.069E+04	0.526	0.962
	验证集	0.965	0.868	5.039E+04	8.110E+09	9.006E+04	0.469	0.965
材料 4	训练集	0.948	1.038	2.754E+04	2.451E+09	4.951E+04	0.585	0.949
	验证集	0.951	1.048	2.516E+04	1.930E+09	4.393E+04	0.592	0.951
训练集均值		0.955	1.126	4.697E+04	6.345E+09	7.965E+04	0.634	0.957
验证集均值		0.954	1.095	4.689E+04	6.371E+09	7.982E+04	0.612	0.956

5.4.4 经验模型改进 1: LMSE — 基于 $\ln$ 的修正 SE 方程

(1) 模型的建立

在斯坦麦茨方程的基础上, [27] 提出了修正的斯坦麦茨方程 (Modified Steinmetz-Equation, MSE), 将磁通密度变率进行加权求和后, 就可以用来计算任意波形下的磁芯损耗, 并且不增加额外的参数。其修正的频率可表示为:

$$f_{\sin.eq} = \frac{2}{\Delta B^2 \cdot \pi^2} \int_0^T \left(\frac{dB}{dt} \right)^2 dt \quad (5.88)$$

然后利用等效正弦波频率来计算非正弦波下的磁芯损耗:

$$P = f \cdot (k_2 \cdot f_{\sin.eq}^{\alpha_2-1} \cdot B_m^{\beta_2}) \quad (5.89)$$

其中, f 是非正弦波频率, k_2 、 α_2 和 β_2 是等效频率下 SE 方程的系数 (为了方便表示, 在后文中同样的参数都形式化地去掉脚标)。那么其损失函数可定义为:

$$\begin{aligned}\mathcal{L} &= \mathbb{E}\{(P - \hat{P})^2\} \\ &= \mathbb{E}\{(P - f \cdot (k \cdot f_{\sin.eq}^{\alpha-1} \cdot B_m^\beta))^2\} \\ &= \mathbb{E}\{(P - f \cdot (k \cdot \left\{ \frac{2}{\Delta B^2 \cdot \pi^2} \int_0^T \left(\frac{dB}{dt}\right)^2 dt \right\}^{\alpha-1} \cdot B_m^\beta))^2\}\end{aligned}\quad (5.90)$$

但是, 这种方式的定义使优化目标变得非常复杂, 因此本文将采用模型二的指数形式进行线性结构的表示, 即:

$$\ln P = \ln k + \ln f + (\alpha - 1) \cdot \ln f_{\sin.eq} + \beta \cdot B_m \quad (5.91)$$

本文称这种修正后 MSE 方程为基于 $\ln$ 的修正斯坦麦茨方程 ($\ln$ -Based Modified Steinmetz-Equation, LMSE), 但是需要注意的是, 上式中含有一项 $\ln f$ 的系数恒为 1, 不能直接使用多元线性回归的方式进行求解, 因此同样需要使用 NAdam 算法进行求解。其改进后的损失可表示为:

$$\begin{aligned}\mathcal{L} &= \mathbb{E}\{(\ln P - \ln \hat{P})^2\} \\ &= \mathbb{E}\{(\ln P - \ln k - \ln f - (\alpha - 1) \cdot \ln f_{\sin.eq} - \beta \cdot B_m)^2\}\end{aligned}\quad (5.92)$$

(2) 模型的求解与分析

A. 实验环境说明

该模型的实验环境与前一节相同。

B. 参数设置

本文以附件一材料 1~4 中的正弦波形共 12400 个样本为研究对象, 将其随机分为 80% 的训练集和 20% 的验证集。并且给定三种未知参数 k 、 α 和 β 的初始值均为 1, 初始学习率设置为 0.001, 批处理大小设置为 256, 迭代次数设置为 5000。

C. 结果与分析

通过 NAdam 求解参数的参数值变化曲线和损失函数值曲线如图 5.20 所示。图 5.20 中横轴 Epoch 与 Step 分别代表迭代次数与其对应的批处理数目, 从图中可以看出, 损失函数随着迭代次数的增加而迅速降低, 并在迭代次数增加到 1000 次时已经趋近于稳定, 而对应的参数 k 、 α 和 β 最终分别趋近于 11.44、1.94 和 2.15。此外, 参数最终并不是一个固定的数值, 而是在某个均值处震荡, 这是因为模型有对输入数据自适应调整参数的能力。

为了更客观地分析 LMSE 模型的预测性能, 表 5.29 给出了模型在不同材料下的指标值。从表 5.29 中可以看出, 模型的拟合优度在训练集和验证集上都只有 0.75 左右, 说明公式拟合的程度一般, 但是 LMSE 模型模型的性能仍然优于传统的 DM 模型。

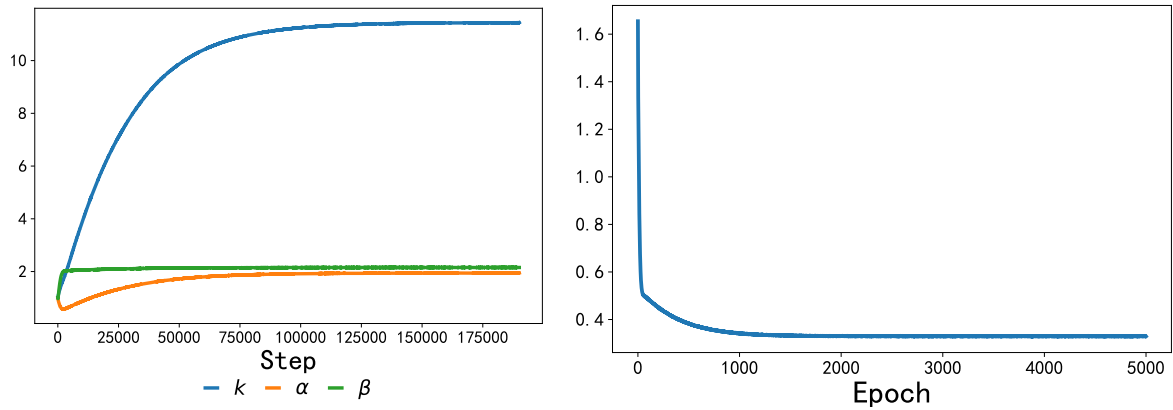


图 5.20 使用 NAdam 求解经验模型 LMSE 时参数值（左）和损失函数值（右）变化趋势

表 5.29 LMSE 模型的预测性能

材料	数据集	$R^2 \uparrow$	$MRE \downarrow$	$MAE \downarrow$	$MSE \downarrow$	$RMSE \downarrow$	$MSLE \downarrow$	$EVS \uparrow$
材料 1	训练集	0.769	0.706	6.186E+04	2.589E+10	1.609E+05	0.379	0.769
	验证集	0.778	0.659	6.689E+04	2.834E+10	1.683E+05	0.352	0.779
材料 2	训练集	0.758	0.496	8.679E+04	4.251E+10	2.062E+05	0.329	0.788
	验证集	0.784	0.488	7.190E+04	2.851E+10	1.689E+05	0.332	0.809
材料 3	训练集	0.712	0.423	1.047E+05	6.126E+10	2.475E+05	0.306	0.757
	验证集	0.724	0.422	1.066E+05	6.383E+10	2.526E+05	0.311	0.768
材料 4	训练集	0.762	0.472	3.861E+04	1.123E+10	1.060E+05	0.298	0.778
	验证集	0.755	0.465	3.416E+04	9.731E+09	9.864E+04	0.295	0.771
训练集均值		0.749	0.529	7.370E+04	3.573E+10	1.890E+05	0.330	0.766
验证集均值		0.761	0.512	7.096E+04	3.334E+10	1.826E+05	0.324	0.778

5.4.5 经验模型改进 2: $LMSE_{SDR}$ — 基于深度自表示的 LMSE 方程

(1) 模型的建立

与模型二和模型 DM_{SDR} 类似，常规的经验模型 SE、MSE 和所提的 LMSE 把待估参数设置为确定的数值，最终可能导致模型的泛化能力差，预测的精度低。因此，本文考虑将模型改进为基于深度自表示的 LMSE 模型（ Ln -Based Modified Steinmetz-Equation with Deep Self-Reputation, $LMSE_{SDR}$ ），用神经网络（其拓扑结构如图 5.14 所示）来提取各个属性之间的深度特征，这种深度学习的方法可以自适应数据属性内部间的分布规律，从而动态地调整系数，达到更精确的预测性能。 $LMSE_{SDR}$ 的形式可表示为：

$$\ln P = \ln \ddot{k} + \ln f + (\ddot{\alpha} - 1) \cdot \ln f_{\sin.eq} + \ddot{\beta} \cdot \ln B_m \quad (5.93)$$

其中， $\ddot{k}$ 、 $\ddot{\alpha}$ 和 $\ddot{\beta}$ 是通过深度全连接网络得到的，可以近似地表示为：

$$\ddot{k} = w_{k2} \cdot \text{Tanh}(w_{k1} \cdot \text{Concat}(\ln f, \ln f_{\sin.eq}, \ln B_m)) \quad (5.94)$$

$$\ddot{\alpha} = w_{\alpha2} \cdot \text{Tanh}(w_{\alpha1} \cdot \text{Concat}(\ln f, \ln f_{\sin.eq}, \ln B_m)) \quad (5.95)$$

$$\ddot{\beta} = w_{\beta2} \cdot \text{Tanh}(w_{\beta1} \cdot \text{Concat}(\ln f, \ln f_{\sin.eq}, \ln B_m)) \quad (5.96)$$

其中， $\text{Concat}(\ln f, \ln f_{\sin.eq}, \ln B_m)^T \in \mathbb{R}^{3 \times 1}$ 表示将 3 个元素相拼接， $w_1 \in \mathbb{R}^{Latent \times 3}$ 和 $w_2 \in \mathbb{R}^{1 \times Latent}$ 是可学习的参数， $Latent$ 为隐藏层的维度， Tanh 为激活函数。通过这种方式计算得到的系数是来自于 f 、 $f_{\sin.eq}$ 和 B_m 自身。因此，模型的损失函数可定义为：

$$\begin{aligned} \mathcal{L} &= \mathbb{E}\{(\ln P - \ln \hat{P})^2\} \\ &= \mathbb{E}\{(\ln P - \ln \ddot{k} - \ln f - (\ddot{\alpha} - 1) \cdot \ln f_{\sin.eq} - \ddot{\beta} \cdot \ln B_m)^2\} \end{aligned} \quad (5.97)$$

(2) 模型的求解与分析

A. 实验环境说明

该模型的实验环境与前一节相同。

B. 参数设置

本文以附件一材料 1~4 中的正弦波形共 12400 个样本为研究对象，将其随机分为 80% 的训练集和 20% 的验证集。并且给定初始学习率设置为 0.001，隐藏层维度 $Latent = 128$ ，批处理大小设置为 256，迭代次数设置为 2000。

C. 结果与分析

通过 NAdam 求解参数的参数值变化曲线和损失函数值曲线如图 5.21 所示。图 5.21 中

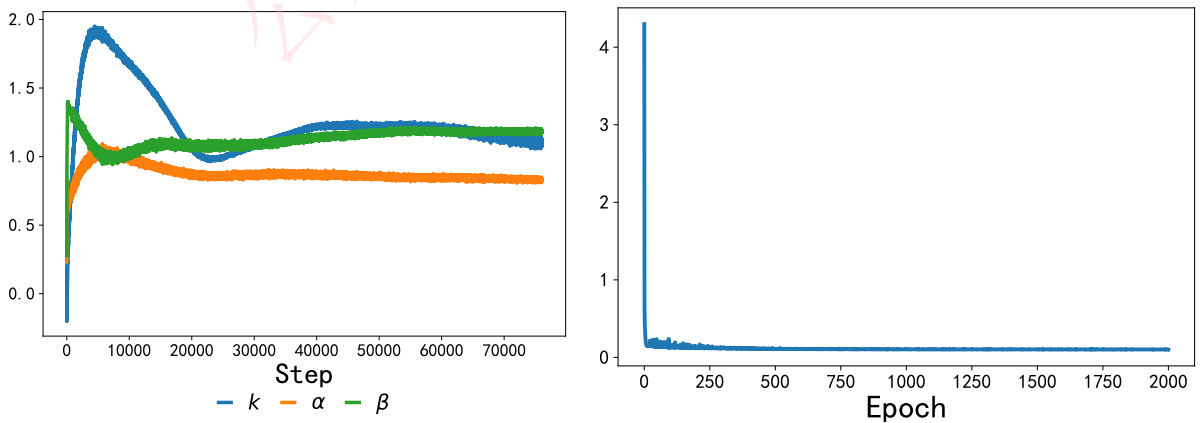


图 5.21 用 NAdam 求解经验模型 $LMSE_{SDR}$ 时参数值（左）和损失函数值（右）变化趋势

横轴 Epoch 与 Step 分别代表迭代次数与其对应的批处理数目，从图中可以看出，损失函数随着迭代次数的增加而迅速降低，并在迭代次数增加到 1000 次时已经趋近于稳定，而对

应的参数 k 、 α 和 β 最终分别趋近于 1.11、0.83 和 1.18。此外，参数最终并不是一个固定的数值，而是在某个均值处震荡，这是因为模型有对输入数据自适应调整参数的能力。

为了更客观地分析 LMSE_{SDR} 模型的预测性能，表5.30 给出了模型在不同材料下的指标值。从表5.30 中可以看出，模型的拟合优度在训练集和验证集上都有 0.9 左右，优于

表 5.30 LMSE_{SDR} 模型的预测性能

材料	数据集	$R^2 \uparrow$	$MRE \downarrow$	$MAE \downarrow$	$MSE \downarrow$	$RMSE \downarrow$	$MSLE \downarrow$	$EVS \uparrow$
材料 1	训练集	0.897	0.388	4.820E+04	1.159E+10	1.076E+05	0.138	0.912
	验证集	0.906	0.364	4.651E+04	1.195E+10	1.093E+05	0.127	0.920
材料 2	训练集	0.971	0.246	3.566E+04	5.177E+09	7.195E+04	0.089	0.971
	验证集	0.970	0.239	3.136E+04	3.930E+09	6.269E+04	0.088	0.970
材料 3	训练集	0.964	0.187	3.569E+04	7.668E+09	8.757E+04	0.075	0.967
	验证集	0.961	0.178	3.874E+04	9.089E+09	9.534E+04	0.069	0.964
材料 4	训练集	0.924	0.207	2.405E+04	3.567E+09	5.973E+04	0.099	0.936
	验证集	0.922	0.198	2.058E+04	3.099E+09	5.567E+04	0.091	0.932
训练集均值		0.949	0.261	3.649E+04	7.214E+09	8.493E+04	0.101	0.949
验证集均值		0.948	0.248	3.498E+04	7.273E+09	8.528E+04	0.095	0.948

LMSE 模型。此外， LMSE_{SDR} 在验证集上的相对误差为 0.248，优于上述的模型。

5.4.6 经验模型改进 3: TLMSE_{SDR} — 考虑温度的 TLMSE_{SDR} 方程

(1) 模型的建立

由模型二和模型 TDM 可以看出，磁芯损耗对温度是较为敏感的，因此本文考虑将温度引入模型 LMSE_{SDR} ，其形式可表示为：

$$\ln P = \ln \ddot{k} + \ln f + (\ddot{\alpha} - 1) \cdot \ln f_{\sin.eq} + \ddot{\beta} \cdot \ln B_m + \ddot{\gamma} \cdot \ln T \quad (5.98)$$

其中， T 表示温度， $\ddot{\gamma}$ 等参数表示对应深度全连接网络得到的待估参数。本文称这个模型为 TLMSE_{SDR} (Ln -Based Modified Steinmetz-Equation with Deep Self-Reputation and Temperature)，可表示为：

$$\ddot{k} = w_{k2} \cdot \text{Tanh}(w_{k1} \cdot \text{Concat}(\ln f, \ln f_{\sin.eq}, \ln B_m, \ln T)) \quad (5.99)$$

$$\ddot{\alpha} = w_{\alpha2} \cdot \text{Tanh}(w_{\alpha1} \cdot \text{Concat}(\ln f, \ln f_{\sin.eq}, \ln B_m, \ln T)) \quad (5.100)$$

$$\ddot{\beta} = w_{\beta2} \cdot \text{Tanh}(w_{\beta1} \cdot \text{Concat}(\ln f, \ln f_{\sin.eq}, \ln B_m, \ln T)) \quad (5.101)$$

$$\ddot{\gamma} = w_{\gamma2} \cdot \text{Tanh}(w_{\gamma1} \cdot \text{Concat}(\ln f, \ln f_{\sin.eq}, \ln B_m, \ln T)) \quad (5.102)$$

其中， $\text{Concat}(\ln f, \ln f_{\sin.eq}, \ln B_m, \ln T)^T \in \mathbb{R}^{4 \times 1}$ 表示将 4 个元素相拼接， $w_1 \in \mathbb{R}^{\text{Latent} \times 4}$ 和 $w_2 \in \mathbb{R}^{1 \times \text{Latent}}$ 是可学习的参数， Latent 为隐藏层的维度， Tanh 为激活函数。

在求解方程时，损失函数定义为：

$$\begin{aligned}\mathcal{L} &= \mathbb{E}\{(\ln P - \ln \hat{P})^2\} \\ &= \mathbb{E}\{(\ln P - \ln \ddot{k} - \ln f - (\ddot{\alpha} - 1) \cdot \ln f_{\sin.eq} - \ddot{\beta} \cdot \ln B_m - \ddot{\gamma} \cdot \ln T)^2\}\end{aligned}\quad (5.103)$$

(2) 模型的求解与分析

A. 实验环境说明

该模型的实验环境与前一节相同。

B. 参数设置

本文以附件一材料 1~4 中的正弦波形共 12400 个样本为研究对象，将其随机分为 80% 的训练集和 20% 的验证集。并且给定初始学习率设置为 0.001，隐藏层维度 $Latent = 128$ ，批处理大小设置为 256，迭代次数设置为 2000。

C. 结果与分析

通过 NAdam 求解参数的参数值变化曲线和损失函数值曲线如图 5.22 所示。图 5.22 中

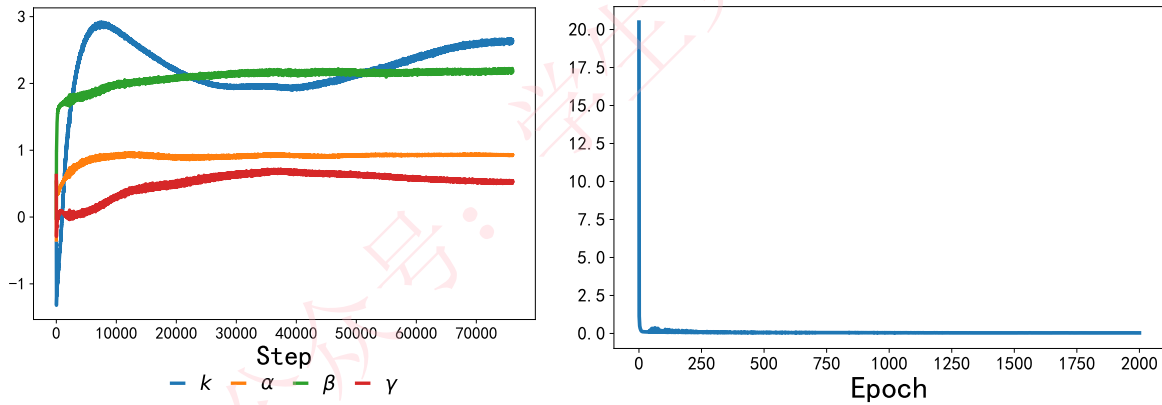


图 5.22 NAdam 求解经验模型 $TLMSE_{SDR}$ 时参数值（左）和损失函数值（右）变化趋势

横轴 Epoch 与 Step 分别代表迭代次数与其对应的批处理数目，从图中可以看出，损失函数随着迭代次数的增加而迅速降低，并在迭代次数增加到 1000 次时已经趋近于稳定，而对应的参数 k 、 α 、 β 和 γ 最终分别趋近于 2.63、0.93、2.20 和 0.53。此外，参数最终并不是一个固定的数值，而是在某个均值处震荡，这是因为模型有对输入数据自适应调整参数的能力。

为了更客观地分析 $TLMSE_{SDR}$ 模型的预测性能，表 5.31 给出了模型在不同材料下的指标值。从表 5.31 中可以看出，模型的拟合优度在训练集和验证集上都有 0.973 左右，优于 $LMSE_{SDR}$ 模型。说明将温度信息引入到模型当中可以有效地提升预测性能。此外， $TLMSE_{SDR}$ 在验证集上的相对误差为 0.152，相较于 $LMSE_{SDR}$ 提升了 0.096。

表 5.31 TLMSE_{SDR} 模型的预测性能

材料	数据集	$R^2 \uparrow$	$MRE \downarrow$	$MAE \downarrow$	$MSE \downarrow$	$RMSE \downarrow$	$MSLE \downarrow$	$EVS \uparrow$
材料 1	训练集	0.961	0.232	3.369E+04	4.325E+09	6.577E+04	0.048	0.972
	验证集	0.964	0.224	3.391E+04	4.598E+09	6.781E+04	0.045	0.973
材料 2	训练集	0.994	0.074	1.436E+04	1.112E+09	3.335E+04	0.009	0.994
	验证集	0.993	0.077	1.337E+04	9.122E+08	3.020E+04	0.009	0.993
材料 3	训练集	0.969	0.114	3.589E+04	6.662E+09	8.162E+04	0.020	0.975
	验证集	0.969	0.110	3.626E+04	7.121E+09	8.438E+04	0.019	0.975
材料 4	训练集	0.936	0.197	2.557E+04	3.036E+09	5.510E+04	0.063	0.949
	验证集	0.944	0.194	2.159E+04	2.237E+09	4.729E+04	0.061	0.955
训练集均值		0.973	0.155	2.775E+04	3.860E+09	6.213E+04	0.035	0.973
验证集均值		0.973	0.152	2.676E+04	3.824E+09	6.184E+04	0.033	0.973

5.4.7 经验模型改进 4: MTLMSE_{SDR} — 考虑材料类型的 TLMSE_{SDR} 方程

(1) 模型的建立

从分离模型可以看出,材料的固有特性对模型的影响是比较大的,不同类型材料的属性会对磁芯损耗的预测产生较大差异的结果。因此,为了解决这一问题,本文在 TLMSE_{SDR} 方程的基础上引入了材料类型信息,并将这种模型称之为 MTLMSE_{SDR} (Ln-Based Modified Steinmetz-Equation with Deep Self-Reputation, Temperature, and Material), 其形式可表示为:

$$\ln P = \ln \ddot{k} + \ln f + (\ddot{\alpha} - 1) \cdot \ln f_{\sin.eq} + \ddot{\beta} \cdot \ln B_m + \ddot{\gamma} \cdot \ln T + \ddot{\zeta} \cdot Emb(M) \quad (5.104)$$

其中, $\ddot{k}$ 、 $\ddot{\alpha}$ 、 $\ddot{\beta}$ 、 $\ddot{\gamma}$ 和 $\ddot{\zeta}$ 表示通过深度全连接网络得到的待估参数, $Emb()$ 表示词嵌入, 目的是将离散的材料类型变量 M 嵌入至连续的特征空间。式中的参数可表示为:

$$\ddot{k} = w_{k2} \cdot Tanh(w_{k1} \cdot Concat(\ln f, \ln f_{\sin.eq}, \ln B_m, \ln T, Emb(M))) \quad (5.105)$$

$$\ddot{\alpha} = w_{\alpha2} \cdot Tanh(w_{\alpha1} \cdot Concat(\ln f, \ln f_{\sin.eq}, \ln B_m, \ln T, Emb(M))) \quad (5.106)$$

$$\ddot{\beta} = w_{\beta2} \cdot Tanh(w_{\beta1} \cdot Concat(\ln f, \ln f_{\sin.eq}, \ln B_m, \ln T, Emb(M))) \quad (5.107)$$

$$\ddot{\gamma} = w_{\gamma2} \cdot Tanh(w_{\gamma1} \cdot Concat(\ln f, \ln f_{\sin.eq}, \ln B_m, \ln T, Emb(M))) \quad (5.108)$$

其中, $Concat(\ln f, \ln f_{\sin.eq}, \ln B_m, \ln T, Emb(M))^T \in \mathbb{R}^{5 \times 1}$ 表示将 5 个元素相拼接, $w_1 \in \mathbb{R}^{Latent \times 5}$ 和 $w_2 \in \mathbb{R}^{1 \times Latent}$ 是可学习的参数, $Latent$ 为隐藏层的维度, $Tanh$ 为激活函数。

在求解方程时，损失函数定义为：

$$\begin{aligned}\mathcal{L} &= \mathbb{E}\{(\ln P - \ln \hat{P})^2\} \\ &= \mathbb{E}\{(\ln P - \ln \ddot{k} - \ln f - (\ddot{\alpha} - 1) \cdot \ln f_{\sin.eq} - \ddot{\beta} \cdot \ln B_m \\ &\quad - \ddot{\gamma} \cdot \ln T - \ddot{\zeta} \cdot Emb(M))^2\}\end{aligned}\quad (5.109)$$

(2) 模型的求解与分析

A. 实验环境说明

该模型的实验环境与前一节相同。

B. 参数设置

本文以附件一材料 1~4 中的正弦波形共 12400 个样本为研究对象，将其随机分为 80% 的训练集和 20% 的验证集。并且给定初始学习率设置为 0.001，隐藏层维度 $Latent = 128$ ，批处理大小设置为 256，迭代次数设置为 4000。

C. 结果与分析

通过 NAdam 求解参数的参数值变化曲线和损失函数值曲线如图 5.23 所示。图 5.23 中

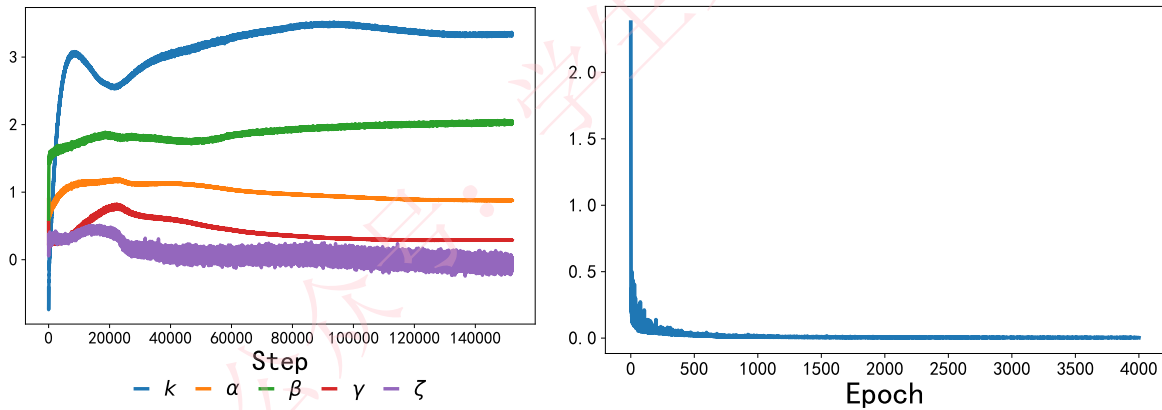


图 5.23 NAdam 求解经验模型 $MTLMSE_{SDR}$ 时参数值（左）和损失函数值（右）变化趋势

横轴 Epoch 与 Step 分别代表迭代次数与其对应的批处理数目，从图中可以看出，损失函数随着迭代次数的增加而迅速降低，并在迭代次数增加到 1000 次时已经趋近于稳定，而对应的参数 k 、 α 、 β 、 γ 和 ζ 最终分别趋近于 3.34、0.88、2.30、0.29、和 -0.05。此外，参数最终并不是一个固定的数值，而是在某个均值处震荡，这是因为模型有对输入数据自适应调整参数的能力。

为了更客观地分析 $MTLMSE_{SDR}$ 模型的预测性能，表 5.32 给出了模型在不同材料下的指标值。从表 5.32 中可以看出，模型的拟合优度在训练集和验证集上都有 0.998 左右，优于 $TLMSE_{SDR}$ 模型。说明将磁芯材料信息引入到模型当中可以有效地提升预测性能。此外， $MTLMSE_{SDR}$ 在验证集上的相对误差为 0.042，相较于 $TLMSE_{SDR}$ 提升了 0.11。

表 5.32 MTLMSE_{SDR} 模型的预测性能

材料	数据集	$R^2 \uparrow$	$MRE \downarrow$	$MAE \downarrow$	$MSE \downarrow$	$RMSE \downarrow$	$MSLE \downarrow$	$EVS \uparrow$
材料 1	训练集	0.998	0.032	5.379E+03	1.693E+08	1.301E+04	0.002	0.999
	验证集	0.998	0.032	5.380E+03	1.954E+08	1.398E+04	0.002	0.998
材料 2	训练集	0.998	0.045	8.770E+03	3.551E+08	1.885E+04	0.003	0.998
	验证集	0.998	0.048	7.433E+03	2.702E+08	1.644E+04	0.004	0.998
材料 3	训练集	0.997	0.045	1.009E+04	5.597E+08	2.366E+04	0.003	0.997
	验证集	0.997	0.047	1.104E+04	6.968E+08	2.640E+04	0.004	0.997
材料 4	训练集	0.998	0.041	3.378E+03	1.018E+08	1.009E+04	0.003	0.998
	验证集	0.998	0.042	3.088E+03	8.426E+07	9.180E+03	0.003	0.998
训练集均值		0.998	0.041	6.962E+03	2.998E+08	1.731E+04	0.003	0.998
验证集均值		0.998	0.042	6.819E+03	3.178E+08	1.783E+04	0.003	0.998

5.4.8 经验模型改进 5: WMTLMSE_{SDR} — 考虑波形的 MTLMSE_{SDR} 方程

(1) 模型的建立

MSE 方程针对非正弦励磁波形的频率进行了修正，但是这种修正方法也可能存在一定的缺陷，不能完全地表征不同励磁波形间频率的差异。因此，本文在 MTLMSE_{SDR} 方程的基础上引入了波形类别信息，并将这种模型称之为 WMTLMSE_{SDR} (Ln-Based Modified Steinmetz-Equation with Deep Self-Reputation, Temperature, Material, and Waveform)，其形式可表示为：

$$\begin{aligned} \ln P = & \ln \ddot{k} + \ln f + (\ddot{\alpha} - 1) \cdot \ln f_{\sin.eq} + \ddot{\beta} \cdot \ln B_m \\ & + \ddot{\gamma} \cdot \ln T + \ddot{\zeta} \cdot Emb(M) + \ddot{\eta} \cdot Emb(W) \end{aligned} \quad (5.110)$$

其中， $\ddot{k}$ 、 $\ddot{\alpha}$ 、 $\ddot{\beta}$ 、 $\ddot{\gamma}$ 、 $\ddot{\zeta}$ 和 $\ddot{\eta}$ 表示通过深度全连接网络（其拓扑结构如图5.14 所示）得到的待估参数， $Emb()$ 表示词嵌入，目的是将离散的材料类型变量 M 和离散的励磁波形类型变量 W 嵌入至连续的特征空间。式中的参数可表示为：

$$\ddot{k} = w_{k2} \cdot Tanh(w_{k1} \cdot Concat(\ln f, \ln f_{\sin.eq}, \ln B_m, \ln T, Emb(M), Emb(W))) \quad (5.111)$$

$$\ddot{\alpha} = w_{\alpha2} \cdot Tanh(w_{\alpha1} \cdot Concat(\ln f, \ln f_{\sin.eq}, \ln B_m, \ln T, Emb(M), Emb(W))) \quad (5.112)$$

$$\ddot{\beta} = w_{\beta2} \cdot Tanh(w_{\beta1} \cdot Concat(\ln f, \ln f_{\sin.eq}, \ln B_m, \ln T, Emb(M), Emb(W))) \quad (5.113)$$

$$\ddot{\gamma} = w_{\gamma2} \cdot Tanh(w_{\gamma1} \cdot Concat(\ln f, \ln f_{\sin.eq}, \ln B_m, \ln T, Emb(M), Emb(W))) \quad (5.114)$$

$$\ddot{\eta} = w_{\eta2} \cdot Tanh(w_{\eta1} \cdot Concat(\ln f, \ln f_{\sin.eq}, \ln B_m, \ln T, Emb(M), Emb(W))) \quad (5.115)$$

其中， $Concat(\ln f, \ln f_{\sin.eq}, \ln B_m, \ln T, Emb(M), Emb(W))^T \in \mathbb{R}^{6 \times 1}$ 表示将 6 个元素相拼接， $w_1 \in \mathbb{R}^{Latent \times 5}$ 和 $w_2 \in \mathbb{R}^{1 \times Latent}$ 是可学习的参数， $Latent$ 为隐藏层的维度， $Tanh$ 为激活函数。

在求解方程时，损失函数定义为：

$$\begin{aligned}\mathcal{L} &= \mathbb{E}\{(\ln P - \ln \hat{P})^2\} \\ &= \mathbb{E}\{(\ln P - \ln \ddot{k} - \ln f - (\ddot{\alpha} - 1) \cdot \ln f_{\sin.eq} - \ddot{\beta} \cdot \ln B_m \\ &\quad - \ddot{\gamma} \cdot \ln T - \ddot{\zeta} \cdot Emb(M) - \ddot{\eta} \cdot Emb(W))^2\}\end{aligned}\quad (5.116)$$

(2) 模型的求解与分析

A. 实验环境说明

该模型的实验环境与前一节相同。

B. 参数设置

本文以附件一材料 1~4 中的正弦波形共 12400 个样本为研究对象，将其随机分为 80% 的训练集和 20% 的验证集。并且给定初始学习率设置为 0.001，隐藏层维度 $Latent = 128$ ，批处理大小设置为 256，迭代次数设置为 10000。

C. 结果与分析

通过 NAdam 求解参数的参数值变化曲线和损失函数值曲线如图 5.24 所示。图 5.24 中

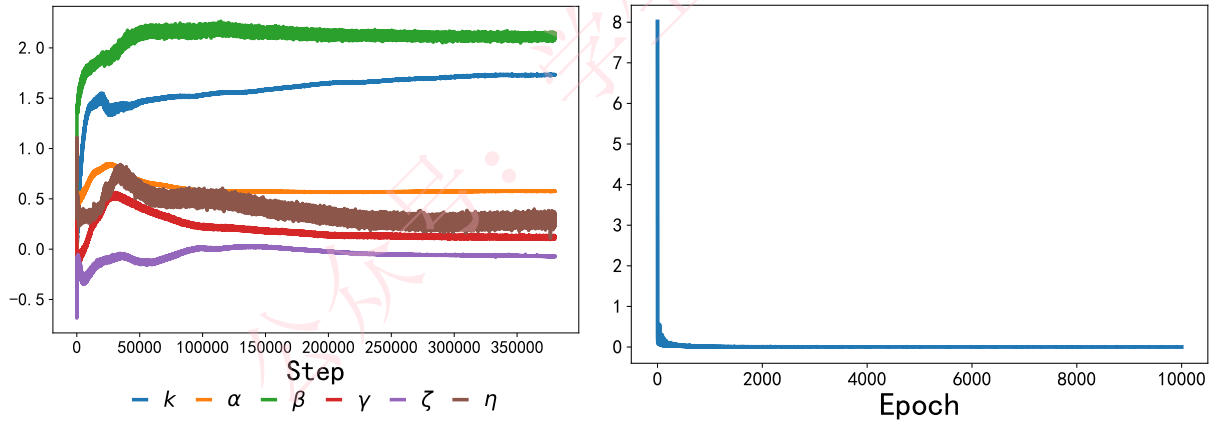


图 5.24 求解经验模型 $WMTLMSE_{SDR}$ 时参数值（左）和损失函数值（右）变化趋势

横轴 Epoch 与 Step 分别代表迭代次数与其对应的批处理数目，从图中可以看出，损失函数随着迭代次数的增加而迅速降低，并在迭代次数增加到 1000 次时已经趋近于稳定，而对应的参数 k 、 α 、 β 、 γ 、 ζ 和 η 最终分别趋近于 1.74、0.58、2.11、0.12、-0.07 和 0.29。此外，参数最终并不是一个固定的数值，而是在某个均值处震荡，这是因为模型有对输入数据自适应调整参数的能力。

为了更客观地分析 $WMTLMSE_{SDR}$ 模型的预测性能，表 5.33 给出了模型在不同材料下的指标值。从表 5.33 中可以看出，模型的拟合优度在训练集和验证集上都为 1 左右，优于 $MTLMSE_{SDR}$ 模型。说明将励磁波形信息引入到模型当中可以有效地提升预测性能。此外， $WMTLMSE_{SDR}$ 在验证集上的相对误差为 0.016，相较于 $MTLMSE_{SDR}$ 提升了 0.026。

表 5.33 WMTLMSE_{SDR} 模型的预测性能

材料	数据集	$R^2 \uparrow$	$MRE \downarrow$	$MAE \downarrow$	$MSE \downarrow$	$RMSE \downarrow$	$MSLE \downarrow$	$EVS \uparrow$
材料 1	训练集	1.000	0.013	2.083E+03	3.805E+07	6.168E+03	0.000	1.000
	验证集	1.000	0.013	2.064E+03	2.664E+07	5.162E+03	0.000	1.000
材料 2	训练集	0.999	0.016	3.741E+03	8.761E+07	9.360E+03	0.000	0.999
	验证集	1.000	0.017	3.350E+03	6.122E+07	7.824E+03	0.000	1.000
材料 3	训练集	0.999	0.017	4.149E+03	1.262E+08	1.123E+04	0.001	0.999
	验证集	0.999	0.017	4.273E+03	1.644E+08	1.282E+04	0.001	0.999
材料 4	训练集	1.000	0.015	1.595E+03	1.892E+07	4.349E+03	0.000	1.000
	验证集	1.000	0.016	1.448E+03	1.373E+07	3.705E+03	0.000	1.000
训练集均值		1.000	0.015	2.908E+03	6.852E+07	8.278E+03	0.000	1.000
验证集均值		1.000	0.016	2.800E+03	6.706E+07	8.189E+03	0.000	1.000

5.4.9 探讨：基于深度学习和 IGSE 方程的模型

在本文中，对于已有模型的改进和其他模型的设计主要依赖于深度学习方法（基于深度自表示的修正模型），那么就会带来一个问题，是否数据驱动的深度学习模型一定优于传统模型？另一方面，[28] 给出了一种修正的斯坦麦茨方程（Improved Generalized Steinmetz equation, IGSE），为何本文没有在这个模型的基础上加以改进？本节将针对这两个问题进行探讨和说明。

(1) 基于深度学习的预测模型

在模型二中，本文就引入了深度学习模块，目的是提取各特征之间的相互关系，自适应地调整待估参数，提升模型的预测性能。考虑到深度学习方法强大的拟合能力，本文设计了一种完全基于深度学习的磁芯损耗预测模型，其结构如图5.25 所示（其中编码器的结构见图5.14）。

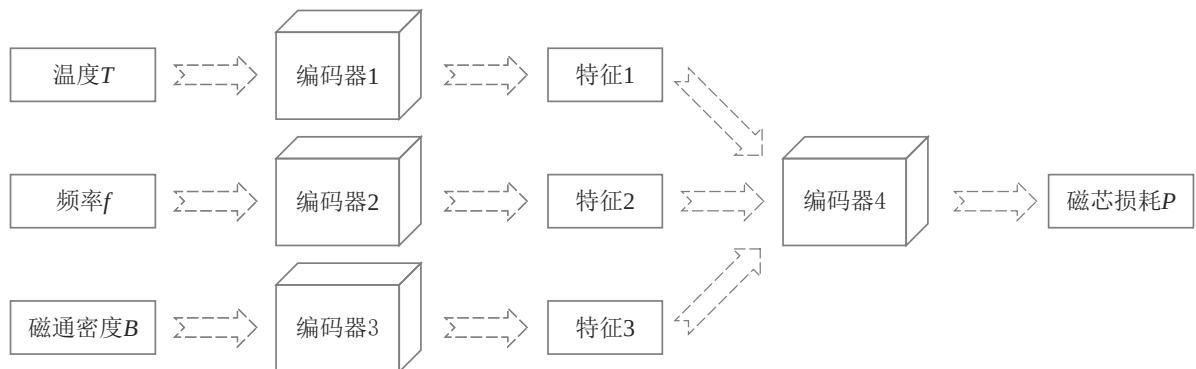


图 5.25 深度学习模型结构图

该深度学习算法在附件一数据集上的识别性能如表5.33 所示：从表5.34 中可以看出，

表 5.34 深度学习模型性能

数据集	$R^2 \uparrow$	$MRE \downarrow$	$MAE \downarrow$	$MSE \downarrow$	$RMSE \downarrow$	$MSLE \downarrow$	$EVS \uparrow$
训练集	1.000	0.419	5.192E+03	3.830E+09	6.189E+04	0.117	1.000
验证集	0.972	0.784	2.263E+04	9.022E+09	9.498E+04	0.279	0.972

深度学习模型发生了过拟合现象^[4]，即在训练集上的相对误差很小，但是在验证集上的精度很低。因此本文不再深入地研究不同模型结构和参数对预测性能的影响，而是采用在深度学习模型中引入先验知识（例如频域信息）的方式去对已有模型做改进 [28]。

(2) 基于 IGSE 方程的预测模型

[29] 提出了 IGSE 方程用于预测任意波形下的磁芯损耗，其公式如下：

$$P = \frac{k_i(\Delta B)^{\beta-\alpha}}{T} \int_0^T \left| \frac{dB}{dt} \right|^\alpha dt \quad (5.117)$$

$$k_i = \frac{k}{(2\pi)^{\alpha-1} \int_0^{2\pi} |\cos \theta|^\alpha 2^{\beta-\alpha} dt} \quad (5.118)$$

其中 ΔB 是磁通密度在一个周期内的峰峰值（周期内最大值与最小值的差）， k 、 α 和 β 是等效频率下 SE 方程的系数。由于方程中的 k_i 过于复杂，[29] 给出了一种简单的近似计算：

$$k_i = \frac{k}{2^{\beta+1} \alpha^{\alpha-1} (0.2761 + \frac{1.7061}{\alpha+1.354})} \quad (5.119)$$

但是在求解的过程中，上式存在一项 $\alpha^{\alpha-1}$ 是不能计算偏导数的，也就是说 NAdam 等优化算法无法正常工作，因此本文未考虑 IGSE 及其改进模型。

5.4.10 总结

在模型四的建立与求解中，本文研究了两类模型：损耗分离模型和经验计算模型，并探究了深度学习模型的 IGSE 模型的可行性。在分离模型中，本文把剩余损耗项中的系数抽象为待估参数，定义了 DM 模型。然后，本文设计了一种深度自表示的方案去动态调整参数，以达到更好的效果，即 DM_{DSR} ，同时设计了加入温度因素后的 TDM 模型。在经验模型中，本文首先研究了修正 SE 方程（MSE）的线性化结构模型 LMSE，然后同样采用深度自表示的方法去对参数做动态修正，即 $LMSE_{DSR}$ 模型。随后，在 $LMSE_{DSR}$ 模型的基础上引入了温度（T）、磁芯材料类型（M）和励磁波形（W）等信息，提出了 $TLMSE_{DSR}$ 、 $MTLMSE_{DSR}$ 和 $WMTLMSE_{DSR}$ 模型。各个模型的预测性能如表5.35 所示。

从表5.35 可以看出，分离模型由于设计粗糙，性能通常不及经验模型。而本文提出的深度自表示方法能有效地将深度学习模型与传统模型相结合，在深度学习中引入了先验知

表 5.35 模型的预测性能总结

数据集	模型	$R^2 \uparrow$	$MRE \downarrow$	$MAE \downarrow$	$MSE \downarrow$	$RMSE \downarrow$	$MSLE \downarrow$	$EVS \uparrow$
训练集	DM	0.940	0.812	4.737E+04	8.593E+09	9.270E+04	0.434	0.940
	DM _{DSR}	0.932	0.470	4.636E+04	9.668E+09	9.832E+04	0.245	0.933
	TDM	0.955	1.126	4.697E+04	6.345E+09	7.965E+04	0.634	0.957
	LMSE	0.749	0.529	7.370E+04	3.573E+10	1.890E+05	0.330	0.766
	LMSE _{DSR}	0.949	0.261	3.649E+04	7.214E+09	8.493E+04	0.101	0.949
	TLMSE _{DSR}	0.973	0.155	2.775E+04	3.860E+09	6.213E+04	0.035	0.973
	MTLMSE _{DSR}	0.998	0.041	6.962E+03	2.998E+08	1.731E+04	0.003	0.998
	WMTLMSE _{DSR}	1.000	0.015	2.908E+03	6.852E+07	8.278E+03	0.000	1.000
验证集	DM	0.935	0.784	4.682E+04	9.022E+09	9.498E+04	0.416	0.935
	DM _{DSR}	0.927	0.458	4.582E+04	1.013E+10	1.006E+05	0.242	0.929
	TDM	0.954	1.095	4.689E+04	6.371E+09	7.982E+04	0.612	0.956
	LMSE	0.761	0.512	7.096E+04	3.334E+10	1.826E+05	0.324	0.778
	LMSE _{DSR}	0.948	0.248	3.498E+04	7.273E+09	8.528E+04	0.095	0.948
	TLMSE _{DSR}	0.973	0.152	2.676E+04	3.824E+09	6.184E+04	0.033	0.973
	MTLMSE _{DSR}	0.998	0.042	6.819E+03	3.178E+08	1.783E+04	0.003	0.998
	WMTLMSE _{DSR}	1.000	0.016	2.800E+03	6.706E+07	8.189E+03	0.000	1.000

识。此外，额外引入温度、磁芯材料类型和励磁波形等信息也能显著提升模型的预测精度。例如，引入温度后，相对误差降低了 0.096；引入磁芯材料类型信息后，相对误差降低了 0.11；引入励磁波形信息后，相对误差降低了 0.026。因此，本文将用模型预测附件三（测试集）中的数据并提交，其中部分数据的磁芯损耗预测结果如表 5.36 所示：

表 5.36 附件三部分预测结果

序号	16	76	98	126	168
预测值	967.1788	1591303	11104.23	1758.888	93744.56
序号	230	271	338	348	379
预测值	67590.55	1731214	13595.36	931271	1376.272

5.5 问题五的模型建立与求解

5.5.1 多目标优化模型

传统的线性规划只能解决一组线性约束条件下，一个目标的最大或最小值的问题。在本文研究中，要考虑磁芯损耗和传输磁能两个目标，所以建立多目标优化模型。由于多个

目标之间的矛盾性和不可公度性，无法要求使所有目标均达到最优解，因此多目标优化问题往往只是求其有效解 (非劣解)。求解非劣解，通常需要将多目标优化问题转化为单目标规划问题处理^[30]。

(1) 目标函数

根据任务要求，目标函数设定包含磁芯损耗和传输磁能，通过问题四的模型建立与求解，磁芯损耗预测模型跨越不同材料类型与工况条件，受到温度、频率、波形、磁通密度峰值及磁芯材料的影响，定义为 $P(f, f_{\sin.eq}, B_m, M, T, W)$ ，其中 f 是频率， $f_{\sin.eq}$ 是等效正弦波频率， B_m 是磁通密度峰值， M 是磁芯材料， T 是温度， W 是波形。传输磁能定义为 $\mathcal{P}(f, B_m) = f \times B_m$ 。

优化目标为磁芯损耗尽可能小，传输磁能尽可能大，即：

$$\min P(f, f_{\sin.eq}, B_m, M, T, W) \quad (5.120)$$

$$\max \mathcal{P}(f, B_m) \quad (5.121)$$

考虑到磁芯损耗与传输性能均 > 0 ：

$$\max \mathcal{P}(f, B_m) \Leftrightarrow \min(-\mathcal{P}(f, B_m)) \quad (5.122)$$

$$\max \mathcal{P}(f, B_m) \Leftrightarrow \frac{1}{\min(-\mathcal{P}(f, B_m))} \quad (5.123)$$

由此等价关系可将此多目标优化模型的目标函数的转化为单目标，考虑以下两种转化方式分别利用上述两种等价关系：

A. 减法

$$\mathcal{L}_1 = \min(P(f, f_{\sin.eq}, B_m, M, T, W) - \lambda \mathcal{P}(f, B_m)) \quad (5.124)$$

λ 为惩罚系数，调整二者的影响程度， λ 取值为 0.01, 0.1, 1, 10, 100。

B. 除法

$$\mathcal{L}_2 = \frac{P(f, f_{\sin.eq}, B_m, M, T, W)}{\mathcal{P}(f, B_m)} \quad (5.125)$$

通过公式 5.126 和公式 5.128 两种转化方式得到目标函数 $\mathcal{L}_1$ 和 $\mathcal{L}_2$ 。

(2) 约束条件

为了确保优化模型的求解结果与实际应用相契合，在定义目标函数的同时，明确各输入变量的合理取值范围，给定目标函数的约束条件。这些输入变量包括频率 f ，等效正弦

波频率 $f_{\sin.eq}$ 是, 磁通密度峰值 B_m , 磁芯材料 M , 温度 T , 波形 W 。其中部分输入变量是连续变量难以处理或优化, 因此将其转换为离散变量来进行研究。

频率 f : $f \in [50000, 500000]\text{Hz}$, 因为频率值为连续的, 所以转变为离散变量, 在求解 f 时仅考虑为整数的情况。

等效正弦波频率 $f_{\sin.eq}$: $f_{\sin.eq} \in [8.25E-4, 2.20E-3]\text{Hz}$, 因为频率值为连续的且较小, 所以转变为离散变量, 在求解 $f_{\sin.eq}$ 时考虑数据精度为 $1E-6$ 。

磁通密度峰值 B_m : $B_m \in [B_{mmin}, B_{mmax}]$, B_{mmax} 为所有样本磁通密度峰值的最大值, B_{mmin} 为所有样本磁通密度峰值的最小值, 计算了所有样本磁通密度峰值, 最终确定范围 $B_m \in [9.64E-3, 3.13E-1]$, 在求解 B_m 时考虑数据精度为 $E-6$ 。

磁芯材料 M : $M \in \text{材料 1, 材料 2, 材料 3, 材料 4}$, 总共有四种磁芯材料, 在模型处理中用 0, 1, 2, 3 数值化表示, 即 $M \in [0, 1, 2, 3]$

温度 T : $T \in [25, 90]^\circ\text{C}$, 且只能取四个值: 25°C 、 50°C 、 70°C 、 90°C 。

波形 W : $W \in \text{正弦波, 三角波, 梯形波}$, 总共有三类励磁波形, 在模型处理中用 0, 1, 2 数值化表示, 即 $W \in [0, 1, 2]$ 。

(3) 多目标优化模型

问题五建立的优化模型总结如下:

A. 减法模型

$$\mathcal{L}_1 = P(f, f_{\sin.eq}, B_m, M, T, W) - \lambda \mathcal{D}(f, B_m) \quad (5.126)$$

$$\text{s.t.} \begin{cases} 50000 \leq f \leq 500000, f \in N \\ 8.25E-4 \leq f_{\sin.eq} \leq 2.20E-3 \\ 9.64E-3 \leq B_m \leq 3.13E-1 \\ 0 \leq M \leq 3, M \in N \\ 25 \leq T \leq 90 \\ 0 \leq W \leq 3, W \in N \end{cases} \quad (5.127)$$

B. 除法模型

$$\mathcal{L}_2 = \frac{P(f, f_{\sin.eq}, B_m, M, T, W)}{\mathcal{D}(f, B_m)} \quad (5.128)$$

$$\text{s.t.} \begin{cases} 50000 \leq f \leq 500000, f \in N \\ 8.25E - 4 \leq f_{\sin.eq} \leq 2.20E - 3 \\ 9.64E - 3 \leq B_m \leq 3.13E - 1 \\ 0 \leq M \leq 3, M \in N \\ 25 \leq T \leq 90 \\ 0 \leq W \leq 3, W \in N \end{cases} \quad (5.129)$$

接下来考虑对此模型求解,由于在第四问中所建立的磁芯损耗预测模型 WMTLMSE_{DSR} 中的参数是由网络学到的,高度非凸的,而传统的算法中,需要进行多次的运行才能得到多次的解,无法得到 Pareto 最优解集,整体上的效率并不高。具体来说, Pareto 边界/最优解集是指没有多个目标整体改进余地的解所组成的集合,改进一个目标会降低另一个目标。而智能优化算法能在一次运行过程中找到 Pareto 最优解集中的多个解,且不限于 Parato 前沿解集的形状和连续性。常用的智能搜索算法包括遗传算法 (Genetic Algorithm, GA) 和粒子群优化算法 (Particle swarm optimization, PSO)。

5.5.2 利用遗传算法求解优化模型

(1) 遗传算法

遗传算法^[30]是一种基于自然选择和遗传学原理的全局优化技术。其基本思想是模拟生物进化过程中的自然选择、遗传和突变等机制,通过一系列操作在解空间内搜索最优或近似最优解。遗传算法的核心在于编码、适应度评估、选择、交叉(重组)以及变异等操作,在遗传算法中,将问题空间的决策变量通过一定的编码方法转换为遗传空间中的个体。将目标函数转化成为一定的适应值,此适应值将作为评价个体优劣的依据。上述步骤共同驱动着算法朝着目标解的方向进化。目前遗传算法结合实数编码能够很好的应对一些整数规划类问题,并以能够较好地规避局部最优解地陷阱这一特点在智能算法领域优势显著。

具体而言首先,将问题的解表示为染色体(Chromosome),通常由一串基因(Gene)组成,基因可以是实数、整数或是其他类型的变量,取决于问题的性质。染色体的编码方式决定了解空间的表示,常见的编码方法有二进制编码、格雷码、实数编码等。编码的选择直接影响算法的性能和效率,因此需根据具体问题选择合适的编码方案。

适应度函数(Fitness Function)的设计至关重要,它决定了哪些个体(解)更适合生存下来并遗传给下一代。适应度函数应当能够准确地反映解的优劣程度,并且尽可能地简化计算量,以保证算法运行的高效性。适应度函数的形式依据具体问题而定,可以是目标函数本身,也可以是经过某种转换后形成的函数。

选择(Selection)是遗传算法中用来确定哪些个体将参与下一步遗传操作的过程。常用的选择策略有轮盘赌选择、锦标赛选择等。这些策略确保了适应度高的个体有更多的机

会被选中，从而增加优秀基因传递到下一代的概率。

交叉（Crossover）操作是遗传算法区别于传统随机搜索方法的关键所在。通过将两个或多个父代个体的部分基因片段互换，生成新的后代个体。交叉操作不仅增加了种群的多样性，还促进了优秀基因的重新组合，有助于探索解空间中的新区域。

变异（Mutate）则是为了保持种群的多样性而设计的操作，通过以很小的概率改变某些个体的基因值，防止算法过早地收敛到局部最优。变异操作通常发生在交叉之后，其概率设置得较低，以免破坏已经通过交叉获得的良好解。

种群（population）是解集的抽象化名词。个体（individual），即对模型的一个解。遗传算法是一种群体优化算法，需要初始生成一定大小的种群（一定数量的有效解构成的解集）来进行后续的寻优。遗传算法的迭代过程从初始化种群开始，经历多轮选择、交叉、变异操作，直至达到预定的停止标准。

遗传算法性能依赖于参数设置，合理的种群规模对于算法性能至关重要。种群规模的大小直接影响算法的探索能力和收敛速度。较大的种群规模能够更好地覆盖解空间，有助于避免局部最优解；然而，这也意味着更高的计算成本。相反，较小的种群规模虽然可以快速收敛，但由于探索不足，可能导致算法过早地陷入局部最优。因此，选择一个合适的种群规模是非常重要的。根据经验，种群规模通常设置为决策变量数量的 4 至 6 倍，并且一般在 20 到 100 之间选取^[31]。另一个关键参数是交叉概率，它决定了个体间发生交叉操作的几率。交叉操作通过混合两个或多个个体的部分特征来创造新的后代。较高的交叉概率有助于增加种群的多样性，但可能会导致解的不稳定；反之，较低的概率则可能导致收敛速度减慢。交叉概率通常设定在一个合理的区间内，大约在 0.4 到 0.99 之间。变异概率则是控制个体是否进行变异操作的标准。变异通过随机改变个体的一部分特征来增强种群的多样性，从而帮助算法逃离局部最优。然而，过高的变异概率可能会导致产生大量无效解，而过低的概率则可能不足以促进种群的多样性。变异概率通常建议设置在 0.0001 到 0.5 之间。最大迭代次数是遗传算法执行的一个终止条件。当算法达到预设的最大迭代次数时，无论是否找到满意解，都将结束运行。这一参数有助于控制算法的执行时间和资源消耗，同时也作为算法未能在限定次数内找到合适解时的退出机制。

（2）模型求解及分析

A. 实验环境说明

本文实验的硬件环境为 Intel(R) Core(TM) i7-13700K@3.40 GHz，64G DDR5 内存及 GeForce RTX 4090 D（24GB 显存）的 64 位计算机。软件平台为 Python 3.11 及 Pytorch 2.0.1+cu118。

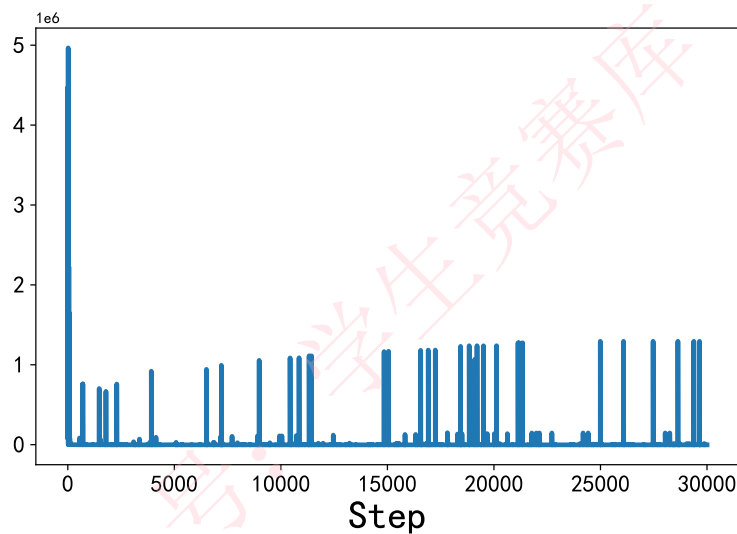
B. 参数设置

遗传算法中的种群数量为 30，迭代次数为 1000。

C. 结果与分析

表 5.37 遗传算法求解优化模型

λ	减法			除法		
	P	$\mathcal{P}$	$\mathcal{L}$	P	$\mathcal{P}$	$\mathcal{L}$
0.01	1.53	48.19	1.05	1.80	57.40	3.13
0.1	41.01	621.61	-21.15	1.80	57.40	0.31
1	1449.52	3664.84	-2215.32	1.80	57.40	0.03
10	4710097.00	106556.35	-5945538.16	1.80	57.40	0.00
100	79740.54	18659.54	-106854.87	1.80	57.40	0.00

图 5.26 GA 求解减法模型 $\lambda = 10$

利用遗传算法对优化问题的减法模型和除法模型分别进行求解， $\lambda = 10$ 时目标函数随迭代次数增加的变化图像如图 5.26 所示，可以发现目标函数振荡但振荡幅度逐渐保持一致，此时遗传算法已经搜到了最优参数组合。具体求解结果见表 5.37，考虑我们最终的优化得到的结果要符合现实，所以磁芯损耗和传输磁能均应该在之前样本数据的范围内。即 $P \in [415.61, 3616132.50]$, $\mathcal{P} \in [1122.15, 77608.98]$ 。故除法模型所得结果均不符合要求，仅考虑减法模型，在该限定范围内查找上表中不同 λ 值所对应的最优结果，并对应检索参数。在 $\lambda = 10$ 时能达到最小的磁芯损耗和最大的传输磁能的最优参数组合具体为：频率 $f = 402092.0$ ，等效正弦波频率 $f_{\sin.eq} = 8.25E - 4$ ，磁通密度峰值 $B_m = 4.64E - 2$ ，磁芯材料 $M = 0.0$ 即材料一，温度 $T = 90$ ，波形 $W = 0.0$ 即正弦波时磁芯损耗最小，传输磁能最大： $P = 71942.29$, $\mathcal{P} = 17038.21$ 。

5.5.3 利用粒子群优化算法求解优化模型

(1) 粒子群优化算法

粒子群优化算法^[7]受到鸟类群体飞行行为的启发,通过模拟粒子在解空间中的运动来寻找最优解。每个粒子代表解空间中的一个候选解,粒子通过更新自己的位置和速度来搜索最优解。从随机解出发,通过迭代寻找最优解,通过适应度来评价解的品质,但粒子群优化算法相较遗传算法规则更为简单,它没有遗传算法的“交叉”(Crossover)和“变异”(Mutation)操作,通过追随当前搜索到的最优值来寻找全局最优。

假设正在寻找 d -维空间中的最优解,则每个粒子的位置可以用 d -维向量来表示:

$$\mathbf{x}_i(t) = (x_{i1}(t), x_{i2}(t), \dots, x_{id}(t)) \quad (5.130)$$

这里的 $\mathbf{x}_i(t)$ 表示在第 t 次迭代时,第 i 个粒子的位置。同样,每个粒子也有一个对应的 d -维速度向量:

$$\mathbf{v}_i(t) = (v_{i1}(t), v_{i2}(t), \dots, v_{id}(t)) \quad (5.131)$$

这里的 $\mathbf{v}_i(t)$ 表示在第 t 次迭代时,第 i 个粒子的速度。 $\mathbf{pbest}_i(t)$ 表示粒子 i 到达过的最好位置; $\mathbf{gbest}(t)$ 表示整个粒子群到达过的最好位置。

粒子 i 在第 $t+1$ 次迭代时,速度更新公式为:

$$\mathbf{v}_i(t+1) = \omega \cdot \mathbf{v}_i(t) + c_1 \cdot r_1(t) \cdot (\mathbf{pbest}_i(t) - \mathbf{x}_i(t)) + c_2 \cdot r_2(t) \cdot (\mathbf{gbest}(t) - \mathbf{x}_i(t)) \quad (5.132)$$

ω 是惯性权重,控制粒子对过去速度的依赖程度;

c_1 和 c_2 分别称为认知因子和社会因子,分别反映了粒子对 $\mathbf{pbest}_i(t)$ 和 $\mathbf{gbest}(t)$ 的信赖程度;

$r_1(t)$ 和 $r_2(t)$ 是在 $[0, 1]$ 范围内的随机数,引入随机性以避免过早收敛。

粒子 i 在 $t+1$ 时刻的位置更新公式为:

$$\mathbf{x}_i(t+1) = \mathbf{x}_i(t) + \mathbf{v}_i(t+1) \quad (5.133)$$

如果达到预定的终止条件(如最大迭代次数、适应度值不再显著变化等),则停止迭代。适应度函数 f 用来衡量解的好坏。适应度函数将解映射到一个标量值,这个值越大,说明解越好。

$$f(\mathbf{x}) : \mathbb{R}^d \rightarrow \mathbb{R} \quad (5.134)$$

粒子群优化算法能够在解空间中有效地搜索最优解。合理设置算法参数(如种群规模、惯性权重、认知因子和社会因子等),可以显著提高算法的性能。

(2) 模型求解及分析

A. 实验环境说明

该模型的实验环境说明与前一节相同。

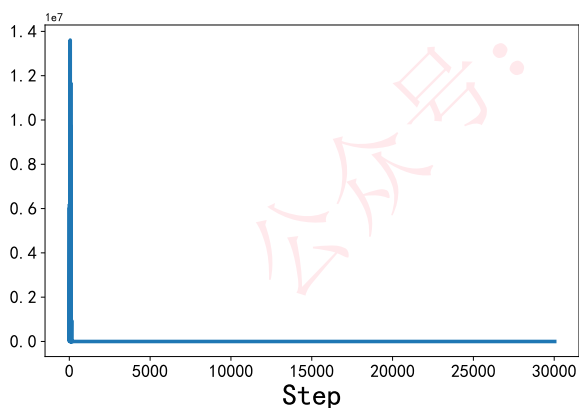
B. 参数设置

粒子群优化算法中的种群数量为，粒子群大小初始化为 30，迭代次数为 1000， $\omega = 0.8$ ， $c_1 = c_2 = 0.5$ 。

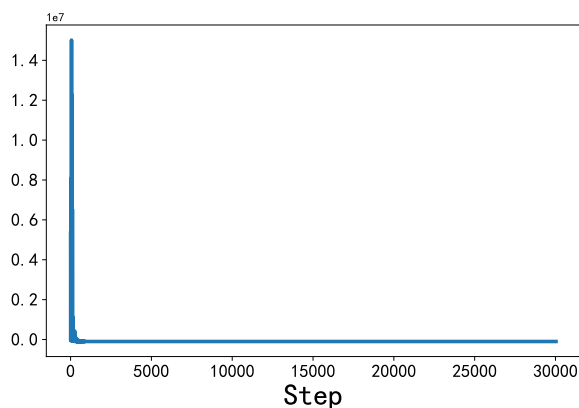
C. 结果与分析

表 5.38 粒子群算法求解优化模型

λ	减法			除法		
	P	$\mathcal{P}$	$\mathcal{L}$	P	$\mathcal{P}$	$\mathcal{L}$
0.01	1.53	48.19	1.05	1.80	57.40	3.13
0.1	41.01	621.61	-21.15	1.80	57.40	0.31
1	1449.52	3664.84	-2215.32	1.80	57.40	0.03
10	4710097.00	106556.35	-5945538.16	1.80	57.40	0.00
100	79740.54	18659.54	-106854.87	1.80	57.40	0.00



(a) PSO 求解减法模型 1



(b) PSO 求解减法模型 2

图 5.27 PSO 求解减法模型

利用粒子群算法对优化问题的减法模型和除法模型分别进行求解，目标函数随迭代次数增加的变化图像如图 5.27 所示，可以发现 $\lambda = 1$ 和 $\lambda = 10$ 时目标函数均快速收敛，此时粒子群算法已经搜到了最优参数组合。具体求解结果见表 5.38，考虑我们最终的优化得到的结果要符合现实，所以磁芯损耗和传输磁能均应该在之前样本数据的范围内。即 $P \in [415.61, 3616132.50]$ ， $\mathcal{P} \in [1122.15, 77608.98]$ 。故除法模型所得结果均不符合要求，仅考虑减法模型，在该限定范围内查找上表中不同 λ 值所对应的最优结果，并对应检索参

数。在 $\lambda = 1$ 时能达到最小的磁芯损耗和最大的传输磁能的最优参数组合具体为：频率 $f = 7862.85$ ，等效正弦波频率 $f_{\sin.eq} = 2.2E - 3$ ，磁通密度峰值 $B_m = 9.6E - 3$ ，磁芯材料 $M = 0.0$ 即材料一，温度 $T = 90$ ，波形 $W = 0.0$ 即正弦波时磁芯损耗最小，传输磁能最大： $P = 1449.52$ ， $\mathcal{P} = 3664.84$ 。在 $\lambda = 10$ 时能达到最小的磁芯损耗和最大的传输磁能的最优参数组合具体为：频率 $f = 50000$ ，等效正弦波频率 $f_{\sin.eq} = 8.25E - 4$ ，磁通密度峰值 $B_m = 1.99E - 1$ ，磁芯材料 $M = 1$ 即材料二，温度 $T = 90$ ，波形 $W = 0.0$ 即正弦波时磁芯损耗最小，传输磁能最大： $P = 79740.54$ ， $\mathcal{P} = 18659.54$ 。

上述研究同时考虑磁芯损耗和传输磁能两个指标，利用遗传算法和粒子群算法求解此优化问题，均取得较好的结果。同时对比两种算法求解结果共同点，温度为 90，波形为正弦波时呈现磁芯损耗更小，传输磁能更大的特性。这对磁性元件的设计与优化领域作出有力的贡献，可以指导磁性元件的设计方向、优化其性能表现，具有重要的理论及实践意义。

6 模型评价与推广

6.1 模型优点

- (1) 本文的模型逐层递进，从模型和算法本身的价值而言，算法与模型契合度良好，求解速度快、优度高，有希望能够为实际磁性元件的设计与优化领域提供想法。
- (2) 本文在提取特征时采用多域融合特征，并且不局限性题目的理想环境，对于有噪环境的分类性能同样卓越。在方程修正和磁芯损耗预测中本文创新性地探究深度自表示方法利用将传统的经验模型与深度学习模型融合，从而利用先验知识对模型训练。这种融合方式的可操作性大，如果后续有更多考虑因素，可以通过这种方式快速简单的融合。
- (3) 在某些特殊场景下，本文所提模型可以自适应地动态调整参数，从而相较固定参数具备更好的泛化性能。

6.2 模型不足

在建立模型时考虑的因素不足，后续拓展分析时可以考虑更多影响因素，让模型更能适用于真实环境。

6.3 模型推广

本文所做改进主要是将传统的经验模型与深度学习模型进行简单的融合，这种融合方式相对简单，且使用的深度学习模型结构基础，虽然能达到一定的效果，但也值得进一步提升。后续希望将这两种模型进行深度融合，使用更复杂的深度学习网络结构，能在更多的场景下达到更好的预测性能。

参考文献

- [1] 颜冲, 何俊, 郭晓东. 不同激励条件对铁氧体磁心损耗的影响 [J]. 磁性材料及器件, 2011, 42(02): 37-42.
- [2] 张伟, 李鑫. 基于多元统计方法的高校教育教学评估研究 [J/OL]. 吉林大学学报 (信息科学版): 1-13.
- [3] 张继国, 辛格. 信息熵: 理论与应用 [M]. 北京水利水电出版社, 2012.
- [4] 周志华, 王珏. 机器学习及其应用 2009 [M]. 清华大学出版社, 2009.
- [5] 统计学习方法 [M]. 清华大学出版社, 2019.
- [6] WANG Xiaohan, YAN Yi, FAN Yanan, LI Xue, MOU Jiao. Space emitter fine feature identification based on multi-domain fusion [J]. Chinese Space Science and Technology, 2023, 43(4): 126-136.
- [7] 何遵文, 侯帅, 张万成, 张焱. 通信特定辐射源识别的多特征融合分类方法 [J]. 通信学报, 2021, 42(02): 103-112.
- [8] 王晓晗, 闫毅, 范亚楠, 李雪, 牟娇. 基于多域融合的空间辐射源细微特征识别 [J/OL]. 中国空间科学技术, 2023, 43(04): 126-136.
- [9] 韩洁, 张涛, 王欢欢, 等. 基于 3D-Hibert 能量谱和多尺度分形特征的通信辐射源个体识别 [J]. 通信学报, 2017, 38(04): 99-109.
- [10] Laurens V D M, Hinton G. Visualizing Data using t-SNE [J]. Journal of Machine Learning Research, 2008, 9(2605): 2579-2605.
- [11] 唐蒙, 张义民, 张凯. Nadam 算法优化卷积神经网络的滚动轴承多故障耦合诊断 [J/OL]. 机械设计与制造: 1-5.
- [12] 黑新宏, 高苗, 张宽, 费蓉, 邱原, 姬文江. 基于 Nadam-TimeGAN 和 XGBoost 的时序信号故障诊断方法 [J]. 通信学报, 2024, 45(04): 185-200.
- [13] 段运会. 图神经网络中的随机梯度下降优化算法研究 [D]. 太原师范学院: 2024.
- [14] 姜文翰, 姜志侠, 孙雪莲. 一种修正学习率的梯度下降算法 [J]. 长春理工大学学报 (自然科学版), 2023, 46(06): 112-120.
- [15] 刘欢, 徐德勃, 张涵涛, 王皓月, 靳洪旭. 基于 Adam 算法的 LMS 优化 [J]. 电子制作, 2024, 32(12): 114-116+113.
- [16] 陇盛, 陶蔚, 张泽东, 陶卿. 基于 AdaGrad 的自适应 NAG 方法及其最优个体收敛性 [J]. 软件学报, 2022, 33(04): 1231-1243.
- [17] 刘锦兰. 深度学习中非凸随机优化算法的收敛性分析 [D]. 东北师范大学: 2023.
- [18] 汪东华. 多元线性分析与 SPSS 应用 [M]. 华东理工大学出版社, 2010.
- [19] 韩骁, 陈浩. 离心泵运行特性的多项式最小二乘拟合应用 [J]. 船电技术, 2024, 44(06): 70-72+78.

- [20] 施艳艳, 李玉珠, 王萌, 郑硕, 付峰. 基于全连接神经网络的颅脑电阻抗成像参考电压预测方法 [J]. 电工技术学报, 2024, 39(14): 4317-4327.
- [21] 梁云辉, 甘舰文, 陈艳, 周芑, 杜亮. 基于图滤波与自表示的无监督特征选择算法 [J]. 吉林大学学报 (理学版), 2024, 62(03): 655-664.
- [22] 鲍全. 基于多因素方差分析法的外窗隔声性能分析与研究 [J]. 工程质量, 2020, 38(02): 42-46+57.
- [23] 罗岚, 何清华, 杨德磊, 谢坚勋. 复杂建设项目的复杂性差异特征分析 [J]. 科技管理研究, 2017, 37(22): 199-207.
- [24] 苏荣娟, 范永辉. 高维数据下两个正态总体均值的逐段检验 [J]. 哈尔滨商业大学学报 (自然科学版), 2023, 39(01): 102-109.
- [25] 李剑锋, 张强, 陈晓宏, 江涛. 考虑水文变异的黄河干流河道内生态需水研究 [J]. 地理学报, 2011, 66(01): 99-110.
- [26] 李家盛, 张正艺, 华宏星. 弹性桨-轴系统建模与计算方法研究 [J]. 振动与冲击, 2023, 42(20): 30-39. DOI:10.13465/j.cnki.jvs.2023.20.004.
- [27] Rudy Severns. HF core loss for non-sinsoidal waveforms. Proc. HFPC' 91, 1991: 140-148.
- [28] Shang, S., Shan, Z., Liu, G., Wang, L., Wang, X., Zhang, Z., and Zhang, J. ResDiff: Combining CNN and Diffusion Model for Image Super-resolution. Proceedings of the AAAI Conference on Artificial Intelligence, 2024. 38(8), 8975-8983.
- [29] Venkatachalam K, Sullivan C R, Abdallah T, et al. Accurate prediction of ferrite core loss with nonsinusoidal waveforms using only Steinmetz parameters[C]// IEEE Workshop on Computers in Power Electronics. Mayaguez, Puerto Rico, USA, 2002: 36-41.
- [30] 肖晓伟, 肖迪, 林锦国, 等. 多目标优化问题的研究概述 [J]. 计算机应用研究, 2011, 28(3): 5. DOI:10.3969/j.issn.1001-3695.2011.03.002.
- [31] 刘晓霞. 种群规模对遗传算法性能影响的研究 [D]. 华北电力大学; 华北电力大学 (保定), 2010.
- [32] 苏守宝. 粒子群优化算法的理论研究与应用 [D]. 安徽大学, 2009.

附录 A MATLAB 源程序

A.1 问题 1: 计算多域融合特征

Listing 1: Feature_Extractor.m

```
1 % Feature_Extractor
2 clear all;
3 clc;
4 close all;
5 %% 读取数据
6 data = load('D:\Pycharm Data\2024研究生数学建模竞赛\问题1\Datasets\data_t1.mat');
7 data = data.data_t1;
8 [M, N] = size(data);
9 %% 数据标准化
10 data_min = min(data, [], 2);
11 data_max = max(data, [], 2);
12 a = (data - data_min);
13 b = (data_max - data_min);
14 data = (data - data_min) ./ (data_max - data_min);
15 %% 特征提取
16 res = ones(M, 10);
17 for i = 1:M
18     sig = data(i, :);
19     %% HHT
20     imf = emd(sig); % 经验模态分解
21     [A, f, t] = hhspectrum(imf); % imf各个分量分别进行希尔伯特变换, 然后分别做fft, 此处的f是归一化的频率
22     [E, t, Cenf] = toimage(A, f); % 合成HHT谱图 (类似语谱图)
23     HHT_margin = sum(E, 2); % HHT边谱图
24     % --- 计算特征
25     HHT_margin_mean = mean(HHT_margin); % 均值
26     HHT_margin_var = var(HHT_margin); % 方差
27     HHT_margin_kurtosis = kurtosis(HHT_margin); % 峰度
28     HHT_margin_skewness = skewness(HHT_margin); % 偏度
29     HHT_margin_entropy = -1*sum(HHT_margin.*... % 信息熵
30         log(HHT_margin + 1e-6)); % 加上1e-n是为了避免log(0)
31     %% Bispecd
32     [bis, waxis] = bispecd(sig);
33     bis_cof = sum(abs(bis)); % 双谱系数
34     % --- 计算特征
35     bis_cof_mean = mean(bis_cof); % 均值
36     bis_cof_var = var(bis_cof); % 方差
37     bis_cof_kurtosis = kurtosis(bis_cof); % 峰度
38     bis_cof_skewness = skewness(bis_cof); % 偏度
39     bis_cof_entropy = -1*sum(bis_cof.*... % 信息熵
40         log(bis_cof + 1e-6)); % 加上1e-n是为了避免log(0)
```

```
41     %% 保存
42     res(i, :) = [HHT_margin_mean, HHT_margin_var, HHT_margin_kurtosis, HHT_margin_skewness,
43                 HHT_margin_entropy ...
44                 bis_cof_mean, bis_cof_var, bis_cof_kurtosis, bis_cof_skewness, bis_cof_entropy];
45     % fprintf('i = %d, %g\n', i)
46     disp(i)
47 end
48 save('D:\Pycharm Data\2024研究生数学建模竞赛\问题1\Datasets\data_10_t1.mat', 'res')
```

附录 B Python 源程序

B.1 问题 1: 基于多域特征的分类

Listing 2: Main_Statistical_characteristics.py

```
1 import gzip
2 import os.path
3 import scipy
4 import hdf5storage
5 import scipy.stats as st
6 import numpy as np
7 import pandas as pd
8 from time import time
9 from tqdm import tqdm
10 from collections import Counter
11 from sklearn.manifold import MDS
12 from scipy.spatial import distance
13 from sklearn import svm
14 import matplotlib.font_manager as fm
15 from sklearn.manifold import TSNE
16 from sklearn.model_selection import train_test_split
17 from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix
18 num_seed = 252618
19 np.random.seed(num_seed)
20 import matplotlib
21 matplotlib.use('TKAgg')
22 import matplotlib.pyplot as plt
23 plt.rcParams['font.sans-serif'] = ['SimHei']
24 plt.rcParams['axes.unicode_minus'] = False
25 colors = np.array(plt.rcParams['axes.prop_cycle']).by_key()['color'] + ['gold', 'deeppink'])
26 print()
27 def draw_matx(label, predict, class_list, savefile):
28     plt.figure(figsize=(4, 4), dpi=150)
29     matx = confusion_matrix(y_true=label, y_pred=predict)
30     matx = matx / np.sum(matx, axis=1, keepdims=True) * 100
31     im = plt.imshow(matx, cmap=plt.cm.Blues)
32     thresh = matx.max() / 2 # 数值颜色阈值, 如果数值超过这个, 就颜色加深。
33     for x in range(num_classes):
34         for y in range(num_classes):
35             # 注意这里的matrix[y, x]不是matrix[x, y]
36             if matx[y, x] == 100:
37                 info = int(matx[y, x])
38             else:
39                 info = round(matx[y, x], 2)
```

```

40         plt.text(x, y, info, verticalalignment='center', horizontalalignment='center',
41                 color="white" if info > thresh else "black", fontsize=18)
42     font = fm.FontProperties(size=18)
43     plt.yticks( range(num_classes), class_list, fontproperties=font)
44     plt.xticks( range(num_classes), class_list, fontproperties=font)
45     plt.savefig(savefile+".png", bbox_inches='tight')
46     plt.savefig(savefile+".pdf", bbox_inches='tight')
47     plt.close('all') # 避免内存泄漏
48 def get_res(data, label):
49     # SVM分类
50     train_data, val_data, train_label, val_label = \
51         train_test_split(data, label, test_size=0.2, random_state=num_seed, stratify=label)
52     print("Train.shape: ", train_data.shape, Counter(train_label))
53     print("val.shape: ", val_data.shape, Counter(val_label))
54     print()
55     # rbf
56     model_rbf = svm.SVC(kernel='rbf', random_state=num_seed, C=2)
57     model_rbf.fit(train_data, train_label)
58     predict_train = model_rbf.predict(train_data)
59     predict_val = model_rbf.predict(val_data)
60     acc_train = round(accuracy_score(train_label, predict_train), 4)
61     acc_val = round(accuracy_score(val_label, predict_val), 4)
62     precision_train = round(precision_score(train_label, predict_train, average='weighted'), 4)
63     precision_val = round(precision_score(val_label, predict_val, average='weighted'), 4)
64     recall_train = round(recall_score(train_label, predict_train, average='weighted'), 4)
65     recall_val = round(recall_score(val_label, predict_val, average='weighted'), 4)
66     f1_train = round(f1_score(train_label, predict_train, average='weighted'), 4)
67     f1_val = round(f1_score(val_label, predict_val, average='weighted'), 4)
68     txt_rbf = "rbf: Accuracy = ({}, {}), Precision = ({}, {}), Recall = ({}, {}), F1 = ({}, {})\n"
69     .format(
70         acc_train, acc_val, precision_train, precision_val, recall_train, recall_val, f1_train,
71         f1_val)
72     print(txt_rbf)
73     predict_rbf = predict_val
74     # linear
75     model_linear = svm.SVC(kernel='linear', random_state=num_seed)
76     model_linear.fit(train_data, train_label)
77     predict_train = model_linear.predict(train_data)
78     predict_val = model_linear.predict(val_data)
79     acc_train = round(accuracy_score(train_label, predict_train), 4)
80     acc_val = round(accuracy_score(val_label, predict_val), 4)
81     precision_train = round(precision_score(train_label, predict_train, average='weighted'), 4)
82     precision_val = round(precision_score(val_label, predict_val, average='weighted'), 4)
83     recall_train = round(recall_score(train_label, predict_train, average='weighted'), 4)
84     recall_val = round(recall_score(val_label, predict_val, average='weighted'), 4)

```



```

83 f1_train = round(f1_score(train_label, predict_train, average='weighted'), 4)
84 f1_val = round(f1_score(val_label, predict_val, average='weighted'), 4)
85 txt_linear = "linear: Accuracy = ({}, {}), Precision = ({}, {}), Recall = ({}, {}), F1 = ({},
    {})\n".
    format(
86     acc_train, acc_val, precision_train, precision_val, recall_train, recall_val, f1_train,
        f1_val)
87 print(txt_linear)
88 predict_linear = predict_val
89 # sigmoid
90 model_sigmoid = svm.SVC(kernel='sigmoid', random_state=num_seed)
91 model_sigmoid.fit(train_data, train_label)
92 predict_train = model_sigmoid.predict(train_data)
93 predict_val = model_sigmoid.predict(val_data)
94 acc_train = round(accuracy_score(train_label, predict_train), 4)
95 acc_val = round(accuracy_score(val_label, predict_val), 4)
96 precision_train = round(precision_score(train_label, predict_train, average='weighted'), 4)
97 precision_val = round(precision_score(val_label, predict_val, average='weighted'), 4)
98 recall_train = round(recall_score(train_label, predict_train, average='weighted'), 4)
99 recall_val = round(recall_score(val_label, predict_val, average='weighted'), 4)
100 f1_train = round(f1_score(train_label, predict_train, average='weighted'), 4)
101 f1_val = round(f1_score(val_label, predict_val, average='weighted'), 4)
102 txt_sigmoid = "sigmoid: Accuracy = ({}, {}), Precision = ({}, {}), Recall = ({}, {}), F1 =
    ({}, {})\n".
    format(
103     acc_train, acc_val, precision_train, precision_val, recall_train, recall_val, f1_train,
        f1_val)
104 print(txt_sigmoid)
105 predict_sigmoid = predict_val
106 # poly
107 model_poly = svm.SVC(kernel='poly', random_state=num_seed)
108 model_poly.fit(train_data, train_label)
109 predict_train = model_poly.predict(train_data)
110 predict_val = model_poly.predict(val_data)
111 acc_train = round(accuracy_score(train_label, predict_train), 4)
112 acc_val = round(accuracy_score(val_label, predict_val), 4)
113 precision_train = round(precision_score(train_label, predict_train, average='weighted'), 4)
114 precision_val = round(precision_score(val_label, predict_val, average='weighted'), 4)
115 recall_train = round(recall_score(train_label, predict_train, average='weighted'), 4)
116 recall_val = round(recall_score(val_label, predict_val, average='weighted'), 4)
117 f1_train = round(f1_score(train_label, predict_train, average='weighted'), 4)
118 f1_val = round(f1_score(val_label, predict_val, average='weighted'), 4)
119 txt_poly = "poly: Accuracy = ({}, {}), Precision = ({}, {}), Recall = ({}, {}), F1 = ({}, {})\
    n".
    format(
120     acc_train, acc_val, precision_train, precision_val, recall_train, recall_val, f1_train,
        f1_val)

```

```

121     print(txt_poly)
122     predict_poly = predict_val
123     return txt_rbf, txt_linear, txt_sigmoid, txt_poly, predict_rbf, predict_linear,
        predict_sigmoid, predict_poly, val_label
124 def get_feature(data):
125     data_mean = np.mean(data, axis=-1, keepdims=True)
126     data_var = np.std(data, axis=-1, keepdims=True) ** 2
127     data_kurt = st.kurtosis(data, axis=-1, keepdims=True)
128     data_skew = st.skew(data, axis=-1, keepdims=True)
129     data_ent = st.entropy(data, axis=-1).reshape(-1, 1)
130     data_sc = np.concatenate((data_mean, data_var, data_kurt, data_skew, data_ent), axis=-1) #
131     data_min = np. min(data_sc, axis=0, keepdims=True)
132     data_max = np. max(data_sc, axis=0, keepdims=True)
133     data_sc = (data_sc - data_min) / (data_max - data_min)
134     return data_sc
135 # 读取数据
136 data = hdf5storage.loadmat('./Datasets/data.mat')['data']
137 data_10 = hdf5storage.loadmat('./Datasets/data_10.mat')['res']
138 label = hdf5storage.loadmat('./Datasets/label.mat')['label'].flatten()
139 mate = hdf5storage.loadmat('./Datasets/mate.mat')['mate'].flatten()
140 num_classes = 3
141 # 数据加噪
142 SNR = 5
143 Ps = np.mean(data**2, axis=-1, keepdims=True)
144 Pn = Ps / (10**(SNR/10))
145 noise = np.sqrt(Pn)*np.random.randn(data.shape[0], 1024)
146 data_snr = data + noise
147 # plt.plot(data[0])
148 # 保存数据 --- 计算特征提取的结果
149 scipy.io.savemat('./Datasets/data_snr.mat', {'data': data_snr})
150 # 读取提取的特征
151 data_snr_10 = hdf5storage.loadmat('./Datasets/data_10_snr.mat')['res']
152 # 数据标准化
153 data_min = np. min(data, axis=-1, keepdims=True)
154 data_max = np. max(data, axis=-1, keepdims=True)
155 data = (data - data_min) / (data_max - data_min)
156 data_min = np. min(data_snr, axis=-1, keepdims=True)
157 data_max = np. max(data_snr, axis=-1, keepdims=True)
158 data_snr = (data_snr - data_min) / (data_max - data_min)
159 data_min = np. min(data_10, axis=0, keepdims=True)
160 data_max = np. max(data_10, axis=0, keepdims=True)
161 data_10 = (data_10 - data_min) / (data_max - data_min)
162 data_min = np. min(data_snr_10, axis=0, keepdims=True)
163 data_max = np. max(data_snr_10, axis=0, keepdims=True)
164 data_snr_10 = (data_snr_10 - data_min) / (data_max - data_min)
165 # 提取统计特征(对原始数据)

```

```

166 data = get_feature(data)
167 data_snr = get_feature(data_snr)
168 # 计算准确率
169 class_list = ['正弦波', '三角波', '梯形波']
170 txt_rbf, txt_linear, txt_sigmoid, txt_poly, predict_rbf, predict_linear, predict_sigmoid,
    predict_poly, val_label = get_res(data, label)
171 with open('./统计特征.txt', 'w') as f:
172     f.write(txt_rbf)
173     f.write(txt_linear)
174     f.write(txt_sigmoid)
175     f.write(txt_poly)
176 draw_matx(val_label, predict_rbf, class_list, savefile='./Plots/混淆矩阵_统计特征_rbf')
177 draw_matx(val_label, predict_linear, class_list, savefile='./Plots/混淆矩阵_统计特征_linear')
178 draw_matx(val_label, predict_sigmoid, class_list, savefile='./Plots/混淆矩阵_统计特征_sigmoid')
179 draw_matx(val_label, predict_poly, class_list, savefile='./Plots/混淆矩阵_统计特征_poly')
180 txt_rbf, txt_linear, txt_sigmoid, txt_poly, predict_rbf, predict_linear, predict_sigmoid,
    predict_poly, val_label = get_res(data_snr, label)
181 with open('./统计特征SNR.txt', 'w') as f:
182     f.write(txt_rbf)
183     f.write(txt_linear)
184     f.write(txt_sigmoid)
185     f.write(txt_poly)
186 draw_matx(val_label, predict_rbf, class_list, savefile='./Plots/混淆矩阵_统计特征SNR_rbf')
187 draw_matx(val_label, predict_linear, class_list, savefile='./Plots/混淆矩阵_统计特征SNR_linear')
188 draw_matx(val_label, predict_sigmoid, class_list, savefile='./Plots/混淆矩阵_统计特征SNR_sigmoid')
189 draw_matx(val_label, predict_poly, class_list, savefile='./Plots/混淆矩阵_统计特征SNR_poly')
190 txt_rbf, txt_linear, txt_sigmoid, txt_poly, predict_rbf, predict_linear, predict_sigmoid,
    predict_poly, val_label = get_res(data_10, label)
191 with open('./SEI统计特征.txt', 'w') as f:
192     f.write(txt_rbf)
193     f.write(txt_linear)
194     f.write(txt_sigmoid)
195     f.write(txt_poly)
196 draw_matx(val_label, predict_rbf, class_list, savefile='./Plots/混淆矩阵_SEI统计特征_rbf')
197 draw_matx(val_label, predict_linear, class_list, savefile='./Plots/混淆矩阵_SEI统计特征_linear')
198 draw_matx(val_label, predict_sigmoid, class_list, savefile='./Plots/混淆矩阵_SEI统计特征_sigmoid')
199 draw_matx(val_label, predict_poly, class_list, savefile='./Plots/混淆矩阵_SEI统计特征_poly')
200 txt_rbf, txt_linear, txt_sigmoid, txt_poly, predict_rbf, predict_linear, predict_sigmoid,
    predict_poly, val_label = get_res(data_snr_10, label)
201 with open('./SEI统计特征SNR.txt', 'w') as f:
202     f.write(txt_rbf)
203     f.write(txt_linear)
204     f.write(txt_sigmoid)
205     f.write(txt_poly)
206 draw_matx(val_label, predict_rbf, class_list, savefile='./Plots/混淆矩阵_SEI统计特征SNR_rbf')
207 draw_matx(val_label, predict_linear, class_list, savefile='./Plots/混淆矩阵_SEI统计特征SNR_linear')

```

```

    )
208 drow_matx(val_label, predict_sigmoid, class_list, savefile='./Plots/混淆矩阵_SEI统计特征
    SNR_sigmoid')
209 drow_matx(val_label, predict_poly, class_list, savefile='./Plots/混淆矩阵_SEI统计特征SNR_poly')
210 print()

```

B.2 问题 2: $TLSE_{DSR}$ 方程求解

Listing 3: Main_NAdam_for_TSElogpro.py

```

1 import gzip
2 import os.path
3 import hdf5storage
4 import torch
5 import torch.nn as nn
6 import numpy as np
7 import pandas as pd
8 from time import time
9 from tqdm import tqdm
10 from collections import Counter
11 from sklearn.manifold import MDS
12 from scipy.spatial import distance
13 from sklearn import svm
14 from sklearn.manifold import TSNE
15 from torch.optim.lr_scheduler import CosineAnnealingLR
16 from torch.utils.data import DataLoader, TensorDataset
17 from sklearn.model_selection import train_test_split
18 from sklearn.metrics import mean_squared_error, mean_squared_log_error, mean_absolute_error,
    explained_variance_score, r2_score
19 import statsmodels.api as sm
20 num_seed = 252618
21 np.random.seed(num_seed)
22 torch.manual_seed(num_seed)
23 import matplotlib
24 matplotlib.use('Agg')
25 import matplotlib.pyplot as plt
26 from mpl_toolkits.mplot3d import Axes3D
27 plt.rcParams['font.sans-serif'] = ['SimHei']
28 plt.rcParams['axes.unicode_minus'] = False
29 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
30 colors = np.array(plt.rcParams['axes.prop_cycle'].by_key()['color'] + ['gold', 'deeppink'])
31 print()
32 # =====
33 batch_size = 32
34 learning_rate = 1e-3
35 num_epochs = 3000

```

```

36 S_T = 100
37 loss_list = []
38 k_list = []
39 alpha_list = []
40 beta_list = []
41 gamma_list = []
42 mre_train_list = []
43 mre_test_list = []
44 mre_epoch_list = []
45 mre_test_min = 999
46 mre_min_epoch = 0
47 def get_res(y, predict):
48     # 计算误差
49     mre = np.mean(np. abs(y - predict) / y)
50     mse = mean_squared_error(y, predict)
51     rmse = np.sqrt(mse)
52     msle = mean_squared_log_error(y, predict)
53     mae = mean_absolute_error(y, predict)
54     evs = explained_variance_score(y, predict)
55     r2s = r2_score(y, predict)
56     txt = 'MRE = {}, \nMSE = {}, \nRMSE = {}, \nMSLE = {}, \nMAE = {}, \nEVS = {}, \nR2-score = {}
        '.
        format(
57         mre, mse, rmse, msle, mae, evs, r2s
58     )
59     return txt, mre
60 class Net(nn.Module):
61     def __init__(self, latent=128):
62         super().__init__()
63         self.k = nn.Sequential(
64             nn.Linear(3, latent),
65             nn.GroupNorm(1, latent),
66             nn.Tanh(),
67             nn.Linear(latent, 1)
68         )
69         self.alpha = nn.Sequential(
70             nn.Linear(3, latent),
71             nn.GroupNorm(1, latent),
72             nn.Tanh(),
73             nn.Linear(latent, 1)
74         )
75         self.beta = nn.Sequential(
76             nn.Linear(3, latent),
77             nn.GroupNorm(1, latent),
78             nn.Tanh(),
79             nn.Linear(latent, 1)

```

```

80         )
81         self.gamma = nn.Sequential(
82             nn.Linear(3, latent),
83             nn.GroupNorm(1, latent),
84             nn.Tanh(),
85             nn.Linear(latent, 1)
86         )
87     def forward(self, T, f, Bm):
88         data_tmp = torch.concatenate((f.unsqueeze(-1), Bm.unsqueeze(-1), T.unsqueeze(-1)), dim=-1)
89         k = self.k(data_tmp).flatten()
90         alpha = self.alpha(data_tmp).flatten()
91         beta = self.beta(data_tmp).flatten()
92         gamma = self.gamma(data_tmp).flatten()
93         P = k + alpha * f + beta * Bm + gamma * T
94         return k, alpha, beta, gamma, P
95 def read_data():
96     # 读取数据
97     data = hdf5storage.loadmat('./Datasets/data.mat')['data']
98     label = hdf5storage.loadmat('./Datasets/label.mat')['label'].flatten()
99     # 划分训练与测试集
100     train_data, test_data, train_label, test_label = \
101         train_test_split(data, label, test_size=0.2, random_state=num_seed)
102     print("Train.shape: ", train_data.shape, Counter(train_label))
103     print("Test.shape: ", test_data.shape, Counter(test_label))
104     # 生成迭代器
105     # 修改数据格式
106     train_data = torch.tensor(train_data, dtype=torch.float32)
107     test_data = torch.tensor(test_data, dtype=torch.float32)
108     train_data = torch.log(train_data)
109     test_data = torch.log(test_data)
110     train_ds = TensorDataset(train_data, torch.log(torch.tensor(train_label, dtype=torch.float32)))
111     train_dl = DataLoader(train_ds, batch_size=batch_size, shuffle=True, drop_last=True)
112     return train_dl, train_data, test_data, train_label, test_label
113 if __name__ == '__main__':
114     # 读取数据
115     train_dl, train_data, test_data, train_label, test_label = read_data()
116     # 初始化网络
117     net = Net().to(device)
118     # 定义损失
119     loss_fn = nn.MSELoss()
120     # 定义优化器
121     optimizer = torch.optim.NAdam(net.parameters(), lr=learning_rate)
122     # 开始训练
123     for epoch in range(num_epochs+1):
124         loss_list_temp = []

```



```

125 net.train()
126 torch.cuda.empty_cache() # 清理无关显
127 data_loader_tqdm = tqdm(train_dl)
128 for step, (batch_x, batch_y) in enumerate(data_loader_tqdm):
129     batch_x = batch_x.to(device)
130     batch_y = batch_y.to(device)
131     k, alpha, beta, gamma, P = net(batch_x[:, 0], batch_x[:, 1], batch_x[:, 2])
132     loss = loss_fn(P, batch_y)
133     optimizer.zero_grad()
134     loss.backward()
135     optimizer.step()
136     loss_list_temp.append(loss.item())
137     k_list.append(k.mean().item())
138     alpha_list.append(alpha.mean().item())
139     beta_list.append(beta.mean().item())
140     gamma_list.append(gamma.mean().item())
141     # 打印输出
142     # 设置tqdm显示
143     data_loader_tqdm.set_description('epoch: {}'.format(epoch))
144     data_loader_tqdm.set_postfix({'lr': optimizer.state_dict()['param_groups'][0]['lr'],
145                                   'loss': loss.cpu().item(),
146                                   'k': k.mean().item(),
147                                   'alpha': alpha.mean().item(),
148                                   'beta': beta.mean().item(),
149                                   'gamma': gamma.mean().item(),
150                                   })
151 loss_list.append(np.mean(loss_list_temp))
152 # 模型测试
153 if epoch % S_T == 0:
154     net.eval()
155     # 计算误差等
156     mre_epoch_list.append(epoch)
157     # 训练集
158     with torch.no_grad():
159         k, alpha, beta, gamma, predict = net(train_data[:, 0].to(device), train_data[:,
160                                             1].to(device), train_data[:, 2].to(device))
161         predict = torch.exp(predict)
162         txt1, mre_train = get_res(train_label, predict.cpu().numpy())
163         print(txt1)
164         mre_train_list.append(mre_train)
165     # 测试集
166     with torch.no_grad():
167         _, _, _, _, predict = net(test_data[:, 0].to(device), test_data[:, 1].to(device),
168                                   test_data[:, 2].to(device))
169         predict = torch.exp(predict)
170         txt2, mre_test = get_res(test_label, predict.cpu().numpy())

```

```

169 print(txt2)
170 mre_test_list.append(mre_test)
171 if mre_test_min > mre_test:
172     mre_test_min = mre_test
173     mre_min_epoch = epoch
174     txt3 = '\n\nk = {}, alpha = {}, beta = {}, gamma = {}'.format(np.exp(k.mean().
        item()), alpha.mean().item(), beta.mean().item(), gamma.mean().item())
175     with open('./NAdam_for_TSElogpro.txt', 'w') as f:
176         f.write('Epoch={}\n\n'.format(mre_min_epoch))
177         f.write('训练集: \n'+ str(txt1))
178         f.write('\n\n测试集: \n'+ str(txt2))
179         f.write(txt3)
180     torch.save(net, './Argus/TSElogpro')
181 try:
182     # 画loss曲线
183     plt.figure(figsize=(8, 5), dpi=150)
184     plt.plot(loss_list, linewidth=3, markersize='3')
185     plt.tick_params(labelsize=15)
186     plt.xlabel('Epoch', fontsize=25)
187     plt.savefig("./Plots/TSElogpro_MSELoss.png", bbox_inches='tight')
188     plt.savefig("./Plots/TSElogpro_MSELoss.pdf", bbox_inches='tight')
189     plt.close('all') # 避免内存泄漏
190     # 画MRE曲线
191     plt.figure(figsize=(8, 5), dpi=150)
192     plt.plot(mre_epoch_list, mre_train_list, linewidth=3, markersize='3')
193     plt.plot(mre_epoch_list, mre_test_list, linewidth=3, markersize='3')
194     plt.tick_params(labelsize=15)
195     plt.xlabel('Epoch', fontsize=25)
196     plt.legend(['MRE_train', 'MRE_val'], prop={'size': 20}, markerscale=5.)
197     plt.savefig("./Plots/TSElogpro_MRE.png", bbox_inches='tight')
198     plt.savefig("./Plots/TSElogpro_MRE.pdf", bbox_inches='tight')
199     plt.close('all') # 避免内存泄漏
200     # 画参数变化曲线
201     plt.figure(figsize=(8, 5), dpi=150)
202     plt.plot(k_list, linewidth=3, markersize='3')
203     plt.plot(alpha_list, linewidth=3, markersize='3')
204     plt.plot(beta_list, linewidth=3, markersize='3')
205     plt.plot(gamma_list, linewidth=3, markersize='3')
206     plt.tick_params(labelsize=15)
207     plt.xlabel('Step', fontsize=25)
208     plt.legend([r'$k$', r'$\alpha$', r'$\beta$', r'$\gamma$'], prop={'size': 20},
        markerscale=5.)
209     plt.savefig("./Plots/TSElogpro_Param.png", bbox_inches='tight')
210     plt.savefig("./Plots/TSElogpro_Param.pdf", bbox_inches='tight')
211     plt.close('all') # 避免内存泄漏
212 except:

```

```
213         pass
214     print()
```

B.3 问题 4: DM_{DSR} 方程求解

Listing 4: Main_ 分离模型 pro.py

```
1  import gzip
2  import os.path
3  import hdf5storage
4  import torch
5  import torch.nn as nn
6  import numpy as np
7  import pandas as pd
8  from time import time
9  from tqdm import tqdm
10 from collections import Counter
11 from sklearn.manifold import MDS
12 from scipy.spatial import distance
13 from sklearn import svm
14 from sklearn.manifold import TSNE
15 from torch.optim.lr_scheduler import CosineAnnealingLR
16 from torch.utils.data import DataLoader, TensorDataset
17 from sklearn.model_selection import train_test_split
18 from sklearn.metrics import mean_squared_error, mean_squared_log_error, mean_absolute_error,
    explained_variance_score, r2_score
19 import statsmodels.api as sm
20 num_seed = 252618
21 np.random.seed(num_seed)
22 torch.manual_seed(num_seed)
23 import matplotlib
24 matplotlib.use('Agg')
25 import matplotlib.pyplot as plt
26 from mpl_toolkits.mplot3d import Axes3D
27 plt.rcParams['font.sans-serif'] = ['SimHei']
28 plt.rcParams['axes.unicode_minus'] = False
29 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
30 colors = np.array(plt.rcParams['axes.prop_cycle'].by_key()['color'] + ['gold', 'deeppink'])
31 print()
32 # =====
33 batch_size = 256
34 learning_rate = 1e-3
35 num_epochs = 10000
36 S_T = 500
37 loss_list = []
38 kh_list = []
```

```

39 kcl_list = []
40 ke_list = []
41 beta_list = []
42 def get_res(y, predict):
43     # 计算误差
44     mre = np.mean(np. abs(y - predict) / y)
45     mse = mean_squared_error(y, predict)
46     rmse = np.sqrt(mse)
47     msle = mean_squared_log_error(y, predict)
48     mae = mean_absolute_error(y, predict)
49     evs = explained_variance_score(y, predict)
50     r2s = r2_score(y, predict)
51     txt = 'MRE = {}, \nMSE = {}, \nRMSE = {}, \nMSLE = {}, \nMAE = {}, \nEVS = {}, \nR2-score = {}
        '.format(
52         mre, mse, rmse, msle, mae, evs, r2s
53     )
54     return txt, [r2s, mre, mae, mse, rmse, msle, evs]
55 class Net(nn.Module):
56     def __init__(self, latent=128):
57         super().__init__()
58         self.kh = nn.Sequential(
59             nn.Linear(2, latent),
60             nn.GroupNorm(1, latent),
61             nn.Tanh(),
62             nn.Linear(latent, 1)
63         )
64         self.kcl = nn.Sequential(
65             nn.Linear(2, latent),
66             nn.GroupNorm(1, latent),
67             nn.Tanh(),
68             nn.Linear(latent, 1)
69         )
70         self.ke = nn.Sequential(
71             nn.Linear(2, latent),
72             nn.GroupNorm(1, latent),
73             nn.Tanh(),
74             nn.Linear(latent, 1)
75         )
76         self.beta = nn.Parameter(torch.ones(1))
77     def forward(self, f, Bm):
78         data_tmp = torch.concatenate((f.unsqueeze(-1), Bm.unsqueeze(-1)), dim=-1)
79         kh = self.kh(data_tmp).flatten()
80         kcl = self.kcl(data_tmp).flatten()
81         ke = self.ke(data_tmp).flatten()
82         Ph = kh * f * (Bm**self.beta)

```

```

83         Pcl = kcl * (f ** 2) * (Bm ** 2)
84         Pe = ke * (f ** 1.5) * (Bm ** 1.5)
85         P = Ph + Pcl + Pe
86         return kh, kcl, ke, P
87 def read_data():
88     # 读取数据
89     data = np.load('./Datasets/data.npy', allow_pickle=True)
90     label = np.load('./Datasets/label.npy', allow_pickle=True)
91     mate = np.load('./Datasets/mate.npy', allow_pickle=True)
92     # 计算磁通密度峰值
93     Bm = data[:, 2:].max(-1, keepdims=True)
94     data = np.concatenate((data[:, :2], Bm), axis=-1)
95     # 去除温度
96     data = data[:, 1:]
97     # label打包
98     label = np.concatenate((label.reshape(-1, 1), mate.reshape(-1, 1)), axis=-1)
99     # 划分训练与测试集
100    train_data, test_data, train_label, test_label = \
101        train_test_split(data, label, test_size=0.2, random_state=num_seed, stratify=mate)
102    print("Train.shape: ", train_data.shape, Counter(train_label[:, 1]))
103    print("Test.shape: ", test_data.shape, Counter(test_label[:, 1]))
104    # label解包
105    train_mate = train_label[:, 1]
106    test_mate = test_label[:, 1]
107    train_label = train_label[:, 0]
108    test_label = test_label[:, 0]
109    # 生成迭代器
110    train_data = torch.tensor(train_data, dtype=torch.float32)
111    test_data = torch.tensor(test_data, dtype=torch.float32)
112    train_ds = TensorDataset(train_data, torch.tensor(train_label, dtype=torch.float32))
113    train_dl = DataLoader(train_ds, batch_size=batch_size, shuffle=True, drop_last=True)
114    return train_dl, train_data, test_data, train_label, test_label, train_mate, test_mate
115 if __name__ == '__main__':
116     # 读取数据
117     train_dl, train_data, test_data, train_label, test_label, train_mate, test_mate = read_data()
118     # 初始化网络
119     net = Net().to(device)
120     # 定义损失
121     loss_fn = nn.MSELoss()
122     # 定义优化器
123     optimizer = torch.optim.NAdam(net.parameters(), lr=learning_rate)
124     # 开始训练
125     for epoch in range(num_epochs+1):
126         loss_list_temp = []
127         net.train()
128         torch.cuda.empty_cache() # 清理无关显

```

```

129 data_loader_tqdm = tqdm(train_dl)
130 for step, (batch_x, batch_y) in enumerate(data_loader_tqdm):
131     batch_x = batch_x.to(device)
132     batch_y = batch_y.to(device)
133     kh, kcl, ke, P = net(batch_x[:, 0], batch_x[:, 1])
134     loss = loss_fn(P, batch_y)
135     optimizer.zero_grad()
136     loss.backward()
137     optimizer.step()
138     loss_list_temp.append(loss.item())
139     kh_list.append(kh.mean().item())
140     kcl_list.append(kcl.mean().item())
141     ke_list.append(ke.mean().item())
142     beta_list.append(net.beta.item())
143     # 打印输出
144     # 设置tqdm显示
145     data_loader_tqdm.set_description('epoch: {}'.format(epoch))
146     data_loader_tqdm.set_postfix({'lr': optimizer.state_dict()['param_groups'][0]['lr'],
147                                   'loss': loss.cpu().item(),
148                                   'kh': kh.mean().item(),
149                                   'kcl': kcl.mean().item(),
150                                   'ke': ke.mean().item(),
151                                   'beta': net.beta.item(),
152                                   })
153     loss_list.append(np.mean(loss_list_temp))
154     # 模型测试
155     if epoch % S_T == 0:
156         net.eval()
157         try:
158             # 画loss曲线
159             plt.figure(figsize=(8, 5), dpi=150)
160             plt.plot(loss_list, linewidth=3, markersize='3')
161             plt.tick_params(labelsize=15)
162             plt.xlabel('Epoch', fontsize=25)
163             plt.savefig("./Plots/分离模型pro_Loss.png", bbox_inches='tight')
164             plt.savefig("./Plots/分离模型pro_Loss.pdf", bbox_inches='tight')
165             plt.close('all') # 避免内存泄漏
166             # 画参数变化曲线
167             plt.figure(figsize=(8, 5), dpi=150)
168             plt.plot(np.array(kh_list) / 200, linewidth=3, markersize='3') #
169             plt.plot(kcl_list, linewidth=3, markersize='3')
170             plt.plot(ke_list, linewidth=3, markersize='3')
171             plt.plot(beta_list, linewidth=3, markersize='3')
172             plt.tick_params(labelsize=15)
173             plt.xlabel('Step', fontsize=25)
174             plt.legend([r'$k_h/200$', r'$k_{cl}$', r'$k_e$', r'$\beta$'], prop={'size': 18},

```



```

175         markerscale=3., bbox_to_anchor=(0.5, -0.3), loc=8, ncol=7, handlelength
            =0.6, frameon=False, columnspacing=1)
176     plt.savefig("./Plots/分离模型pro.png", bbox_inches='tight')
177     plt.savefig("./Plots/分离模型pro.pdf", bbox_inches='tight')
178     plt.close('all') # 避免内存泄漏
179     except:
180         pass
181     # 计算误差等
182     res_matx = np.zeros([10, 7])
183     net. eval()
184     # 训练集
185     with torch.no_grad():
186         kh, kcl, ke, predict = net(train_data[:, 0].to(device), train_data[:, 1].to(device))
187         txt1, res_list = get_res(train_label, predict.cpu().numpy())
188         res_matx[-2, :] = res_list
189         # 计算每一种材料的结果:
190         for i in range(4):
191             idx = train_mate == i
192             with torch.no_grad(): _, _, _, predict = net(train_data[idx, 0].to(device), train_data[idx
                , 1].to(device))
193             txt_tmp, res_list = get_res(train_label[idx], predict.cpu().numpy())
194             txt1 = txt1 + '\n材料{}: \n'. format(i+1) + txt_tmp
195             res_matx[i * 2, :] = res_list
196         print(txt1)
197         # 测试集
198         with torch.no_grad():
199             _, _, _, predict = net(test_data[:, 0].to(device), test_data[:, 1].to(device))
200             txt2, res_list = get_res(test_label, predict.cpu().numpy())
201             res_matx[-1, :] = res_list
202             # 计算每一种材料的结果:
203             for i in range(4):
204                 idx = test_mate == i
205                 with torch.no_grad(): _, _, _, predict = net(test_data[idx, 0].to(device), test_data[idx,
                    1].to(device))
206                 txt_tmp, res_list = get_res(test_label[idx], predict.cpu().numpy())
207                 txt2 = txt2 + '\n材料{}: \n'. format(i+1) + txt_tmp
208                 res_matx[i * 2 + 1, :] = res_list
209             print(txt2)
210             txt3 = '\n\nkh = {}, kcl = {}, ke = {}, beta = {}'. format(
211                 kh.mean().item(), kcl.mean().item(), ke.mean().item(), net.beta.item()
212             )
213             with open('./分离模型pro.txt', 'w', encoding='utf-8') as f:
214                 f.write('训练集: \n'+ str(txt1))
215                 f.write('\n\n测试集: \n'+ str(txt2))
216                 f.write(txt3)
217             res_pd = pd.DataFrame(res_matx)

```

```

218 writer = pd.ExcelWriter('./Reslut/FIMXpro.xlsx')
219 res_pd.to_excel(writer, 'page_1', float_format='%.5f')
220 writer.close()
221 torch.save(net, './Argus/FIMXpro')
222 print()

```

B.4 问题 4: WMTLMSE_{DSR} 方程求解

Listing 5: Main_经验模型 WMTMSEpro.py

```

1 import gzip
2 import os.path
3 import sympy
4 import hdf5storage
5 import torch
6 import torch.nn as nn
7 import numpy as np
8 import pandas as pd
9 from time import time
10 from tqdm import tqdm
11 from collections import Counter
12 from sklearn.manifold import MDS
13 from scipy.spatial import distance
14 from sklearn import svm
15 from sklearn.manifold import TSNE
16 from torch.optim.lr_scheduler import CosineAnnealingLR
17 from torch.utils.data import DataLoader, TensorDataset
18 from sklearn.model_selection import train_test_split
19 from sklearn.metrics import mean_squared_error, mean_squared_log_error, mean_absolute_error,
    explained_variance_score, r2_score
20 import statsmodels.api as sm
21 num_seed = 252618
22 np.random.seed(num_seed)
23 torch.manual_seed(num_seed)
24 import matplotlib
25 matplotlib.use('Agg')
26 import matplotlib.pyplot as plt
27 from mpl_toolkits.mplot3d import Axes3D
28 plt.rcParams['font.sans-serif'] = ['SimHei']
29 plt.rcParams['axes.unicode_minus'] = False
30 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
31 colors = np.array(plt.rcParams['axes.prop_cycle'].by_key()['color'] + ['gold', 'deeppink'])
32 print()
33 # =====
34 batch_size = 256
35 learning_rate = 1e-3

```

```

36 num_epochs = 10000
37 S_T = 100
38 loss_list = []
39 k_list = []
40 alpha_list = []
41 beta_list = []
42 gamma_list = []
43 zeta_list = []
44 eta_list = []
45 mre_train_list = []
46 mre_test_list = []
47 mre_epoch_list = []
48 mre_test_min = 999
49 mre_min_epoch = 0
50 def get_res(y, predict):
51     # 计算误差
52     mre = np.mean(np. abs(y - predict) / y)
53     mse = mean_squared_error(y, predict)
54     rmse = np.sqrt(mse)
55     msle = mean_squared_log_error(y, predict)
56     mae = mean_absolute_error(y, predict)
57     evs = explained_variance_score(y, predict)
58     r2s = r2_score(y, predict)
59     txt = 'MRE = {}, \nMSE = {}, \nRMSE = {}, \nMSLE = {}, \nMAE = {}, \nEVS = {}, \nR2-score = {}
        '.
        format(
60         mre, mse, rmse, msle, mae, evs, r2s
61     )
62     return txt, mre, [r2s, mre, mae, mse, rmse, msle, evs]
63 class Net(nn.Module):
64     def __init__(self, latent=128):
65         super().__init__()
66         self.k = nn.Sequential(
67             nn.Linear(6, latent),
68             nn.GroupNorm(1, latent),
69             nn.Tanh(),
70             nn.Linear(latent, 1)
71         )
72         self.alpha = nn.Sequential(
73             nn.Linear(6, latent),
74             nn.GroupNorm(1, latent),
75             nn.Tanh(),
76             nn.Linear(latent, 1)
77         )
78         self.beta = nn.Sequential(
79             nn.Linear(6, latent),

```

```

80         nn.GroupNorm(1, latent),
81         nn.Tanh(),
82         nn.Linear(latent, 1)
83     )
84     self.gamma = nn.Sequential(
85         nn.Linear(6, latent),
86         nn.GroupNorm(1, latent),
87         nn.Tanh(),
88         nn.Linear(latent, 1)
89     )
90     self.zeta = nn.Sequential(
91         nn.Linear(6, latent),
92         nn.GroupNorm(1, latent),
93         nn.Tanh(),
94         nn.Linear(latent, 1)
95     )
96     self.eta = nn.Sequential(
97         nn.Linear(6, latent),
98         nn.GroupNorm(1, latent),
99         nn.Tanh(),
100         nn.Linear(latent, 1)
101     )
102     self.M_emb = nn.Embedding(4, 1)
103     self.W_emb = nn.Embedding(3, 1)
104     def forward(self, f, fs, Bm, T, M, W):
105         M = self.M_emb(M. type(torch.int32))
106         W = self.W_emb(W. type(torch.int32))
107         data_tmp = torch.concatenate((f.unsqueeze(-1), fs.unsqueeze(-1), Bm.unsqueeze(-1), T.
            unsqueeze(-1), M, W), dim=-1)
108         k = self.k(data_tmp).flatten()
109         alpha = self.alpha(data_tmp).flatten()
110         beta = self.beta(data_tmp).flatten()
111         gamma = self.gamma(data_tmp).flatten()
112         zeta = self.zeta(data_tmp).flatten()
113         eta = self.eta(data_tmp).flatten()
114         P = k + f + (alpha-1) * fs + beta * Bm + gamma * T + zeta * M.squeeze() + eta * W.squeeze
            ()
115         return k, alpha, beta, gamma, zeta, eta, P
116     def read_data():
117         # 读取数据
118         data = np.load('./Datasets/data.npy', allow_pickle=True)
119         label = np.load('./Datasets/label.npy', allow_pickle=True)
120         mate = np.load('./Datasets/mate.npy', allow_pickle=True)
121         wave = np.load('./Datasets/wave.npy', allow_pickle=True)
122         f = data[:, 1].reshape(-1, 1)
123         # 修正的频率

```

```

124 B = data[:, 2:]
125 Delta_B = B. max(-1, keepdims=True) - B. min(-1, keepdims=True)
126 f_dB_dt = np. sum(np.diff(B)**2, axis=-1, keepdims=True)
127 f_sin = 2/(Delta_B**2 * np.pi**2)*f_dB_dt
128 # 温度
129 T = data[:, 0].reshape(-1, 1)
130 # 计算磁通密度峰值
131 Bm = data[:, 2:]. max(-1, keepdims=True)
132 # # 材料特征嵌入
133 # mate_emb = nn.Embedding(4, 1)(torch.tensor(mate)).detach().numpy()
134 # 数据拼接
135 data = np.concatenate((f, f_sin, Bm, T), axis=-1) # , mate.reshape(-1, 1)
136 # label打包
137 label = np.concatenate((label.reshape(-1, 1), mate.reshape(-1, 1), wave.reshape(-1, 1)), axis
    =-1)
138 # 划分训练与测试集
139 train_data, test_data, train_label, test_label = \
140     train_test_split(data, label, test_size=0.2, random_state=num_seed)
141 print("Train.shape: ", train_data.shape)
142 print("Test.shape: ", test_data.shape)
143 # label解包
144 train_mate = train_label[:, 1]
145 test_mate = test_label[:, 1]
146 train_wave = train_label[:, 2]
147 test_wave = test_label[:, 2]
148 train_label = train_label[:, 0]
149 test_label = test_label[:, 0]
150 train_data = np.log(train_data)
151 test_data = np.log(test_data)
152 train_data = np.concatenate((train_data, train_mate.reshape(-1, 1), train_wave.reshape(-1, 1))
    , axis=-1)
153 test_data = np.concatenate((test_data, test_mate.reshape(-1, 1), test_wave.reshape(-1, 1)),
    axis=-1)
154 # 生成迭代器
155 train_data = torch.tensor(train_data, dtype=torch.float32)
156 test_data = torch.tensor(test_data, dtype=torch.float32)
157 train_ds = TensorDataset(train_data, torch.log(torch.tensor(train_label, dtype=torch.float32))
    )
158 train_dl = DataLoader(train_ds, batch_size=batch_size, shuffle=True, drop_last=True)
159 return train_dl, train_data, test_data, train_label, test_label, train_mate, test_mate,
    train_wave, test_wave
160 if __name__ == '__main__':
161     # 读取数据
162     train_dl, train_data, test_data, train_label, test_label, train_mate, test_mate, train_wave,
        test_wave = read_data()
163     # 初始化网络

```

```

164 net = Net().to(device)
165 # 定义损失
166 loss_fn = nn.MSELoss()
167 # 定义优化器
168 optimizer = torch.optim.NAdam(net.parameters(), lr=learning_rate)
169 # 开始训练
170 for epoch in range(num_epochs+1):
171     loss_list_temp = []
172     net.train()
173     torch.cuda.empty_cache() # 清理无关显
174     data_loader_tqdm = tqdm(train_dl)
175     for step, (batch_x, batch_y) in enumerate(data_loader_tqdm):
176         batch_x = batch_x.to(device)
177         batch_y = batch_y.to(device)
178         k, alpha, beta, gamma, zeta, eta, P = net(batch_x[:, 0], batch_x[:, 1], batch_x[:, 2],
179             batch_x[:, 3], batch_x[:, 4], batch_x[:, 5])
180         loss = loss_fn(P, batch_y)
181         optimizer.zero_grad()
182         loss.backward()
183         optimizer.step()
184         loss_list_temp.append(loss.item())
185         k_list.append(k.mean().item())
186         alpha_list.append(alpha.mean().item())
187         beta_list.append(beta.mean().item())
188         gamma_list.append(gamma.mean().item())
189         zeta_list.append(zeta.mean().item())
190         eta_list.append(eta.mean().item())
191     # 打印输出
192     # 设置tqdm显示
193     data_loader_tqdm.set_description('epoch: {}'.format(epoch))
194     data_loader_tqdm.set_postfix({'lr': optimizer.state_dict()['param_groups'][0]['lr'],
195         'loss': loss.cpu().item(),
196         'k': k.mean().item(),
197         'alpha': alpha.mean().item(),
198         'beta': beta.mean().item(),
199         'gamma': gamma.mean().item(),
200         'zeta': zeta.mean().item(),
201         'eta': eta.mean().item(),
202     })
203     loss_list.append(np.mean(loss_list_temp))
204 # 模型测试
205 if epoch % S_T == 0:
206     res_matx = np.zeros([10, 7])
207     net.eval()
208     # 计算误差等
209     mre_epoch_list.append(epoch)

```

```

209     # 训练集
210     with torch.no_grad():
211         k, alpha, beta, gamma, zeta, eta, predict = net(train_data[:, 0].to(device),
212                 train_data[:, 1].to(device), train_data[:, 2].to(device), train_data[:, 3].to(
213                     device), train_data[:, 4].to(device), train_data[:, 5].to(device))
214         predict = torch.exp(predict)
215         txt1, mre_train, res_list = get_res(train_label, predict.cpu().numpy())
216         res_matx[-2, :] = res_list
217     # 计算每一种材料的结果:
218     for i in range(4):
219         idx = train_mate == i
220         with torch.no_grad():
221             _, _, _, _, _, predict = net(train_data[idx, 0].to(device), train_data[idx,
222                 1].to(device), train_data[idx, 2].to(device), train_data[idx, 3].to(
223                     device), train_data[idx, 4].to(device), train_data[idx, 5].to(device))
224             predict = torch.exp(predict)
225             txt_tmp, _, res_list = get_res(train_label[idx], predict.cpu().numpy())
226             txt1 = txt1 + '\n材料{}: \n'.format(i + 1) + txt_tmp
227             res_matx[i * 2, :] = res_list
228         print(txt1)
229         mre_train_list.append(mre_train)
230     # 测试集
231     with torch.no_grad():
232         _, _, _, _, _, predict = net(test_data[:, 0].to(device), test_data[:, 1].to(
233             device), test_data[:, 2].to(device), test_data[:, 3].to(device), test_data[:,
234                 4].to(device), test_data[:, 5].to(device))
235         predict = torch.exp(predict)
236         txt2, mre_test, res_list = get_res(test_label, predict.cpu().numpy())
237         res_matx[-1, :] = res_list
238     # 计算每一种材料的结果:
239     for i in range(4):
240         idx = test_mate == i
241         with torch.no_grad():
242             _, _, _, _, _, predict = net(test_data[idx, 0].to(device), test_data[idx,
243                 1].to(device), test_data[idx, 2].to(device), test_data[idx, 3].to(device),
244                 test_data[idx, 4].to(device), test_data[idx, 5].to(device))
245             predict = torch.exp(predict)
246             txt_tmp, _, res_list = get_res(test_label[idx], predict.cpu().numpy())
247             txt2 = txt2 + '\n材料{}: \n'.format(i + 1) + txt_tmp
248             res_matx[i * 2 + 1, :] = res_list
249         print(txt2)
250         mre_test_list.append(mre_test)
251     if mre_test_min > mre_test:
252         mre_test_min = mre_test
253         mre_min_epoch = epoch
254     txt3 = '\n\nk = {}, alpha = {}, beta = {}, gamma = {}'.format(np.exp(k.mean()).

```



```

        item()), alpha.mean().item(), beta.mean().item(), gamma.mean().item())
247 with open('./经验模型WMTMSEpro.txt', 'w', encoding='utf-8') as f:
248     f.write('Epoch={}\n\n'.format(mre_min_epoch))
249     f.write('训练集: \n'+ str(txt1))
250     f.write('\n\n测试集: \n'+ str(txt2))
251     f.write(txt3)
252 res_pd = pd.DataFrame(res_matx)
253 writer = pd.ExcelWriter('./Reslut/WMTMSEpro.xlsx')
254 res_pd.to_excel(writer, 'page_1', float_format='%.5f')
255 writer.close()
256 torch.save(net, './Argus/WMTMSEpro')
257 try:
258     # 画loss曲线
259     plt.figure(figsize=(8, 5), dpi=150)
260     plt.plot(loss_list, linewidth=3, markersize='3')
261     plt.tick_params(labelsize=15)
262     plt.xlabel('Epoch', fontsize=25)
263     plt.savefig('./Plots/经验模型WMTMSEpro_Loss.png', bbox_inches='tight')
264     plt.savefig('./Plots/经验模型WMTMSEpro_Loss.pdf', bbox_inches='tight')
265     plt.close('all') # 避免内存泄漏
266     # 画MRE曲线
267     plt.figure(figsize=(8, 5), dpi=150)
268     plt.plot(mre_epoch_list, mre_train_list, linewidth=3, markersize='3')
269     plt.plot(mre_epoch_list, mre_test_list, linewidth=3, markersize='3')
270     plt.tick_params(labelsize=15)
271     plt.xlabel('Epoch', fontsize=25)
272     plt.legend(['MRE_train', 'MRE_val'], prop={'size': 18},
273               markerscale=3., bbox_to_anchor=(0.5, -0.3), loc=8, ncol=7, handlelength
               =0.6, frameon=False, columnspacing=1)
274     plt.savefig('./Plots/经验模型WMTMSEpro_MRE.png', bbox_inches='tight')
275     plt.savefig('./Plots/经验模型WMTMSEpro_MRE.pdf', bbox_inches='tight')
276     plt.close('all') # 避免内存泄漏
277     # 画参数变化曲线
278     plt.figure(figsize=(8, 5), dpi=150)
279     plt.plot(k_list, linewidth=3, markersize='3')
280     plt.plot(alpha_list, linewidth=3, markersize='3')
281     plt.plot(beta_list, linewidth=3, markersize='3')
282     plt.plot(gamma_list, linewidth=3, markersize='3')
283     plt.plot(zeta_list, linewidth=3, markersize='3')
284     plt.plot(eta_list, linewidth=3, markersize='3')
285     plt.tick_params(labelsize=15)
286     plt.xlabel('Step', fontsize=25)
287     plt.legend([r'$k$', r'$\alpha$', r'$\beta$', r'$\gamma$', r'$\zeta$', r'$\eta$'],
                prop={'size': 18},
288               markerscale=3., bbox_to_anchor=(0.5, -0.3), loc=8, ncol=7, handlelength
               =0.6, frameon=False, columnspacing=1)

```

```

289         plt.savefig("./Plots/经验模型WMTMSEpro.png", bbox_inches='tight')
290         plt.savefig("./Plots/经验模型WMTMSEpro.pdf", bbox_inches='tight')
291         plt.close('all') # 避免内存泄漏
292     except:
293         pass
294 print()

```

B.5 问题 5: 遗传算法-减法

Listing 6: Main_GA 减法.py

```

1  import gzip
2  import os.path
3  import sympy
4  import hdf5storage
5  import torch
6  import torch.nn as nn
7  import numpy as np
8  import pandas as pd
9  from time import time
10 from tqdm import tqdm
11 from collections import Counter
12 from sklearn.manifold import MDS
13 from scipy.spatial import distance
14 from sklearn import svm
15 from sklearn.manifold import TSNE
16 from torch.optim.lr_scheduler import CosineAnnealingLR
17 from torch.utils.data import DataLoader, TensorDataset
18 from sklearn.model_selection import train_test_split
19 from sklearn.metrics import mean_squared_error, mean_squared_log_error, mean_absolute_error,
    explained_variance_score, r2_score
20 import statsmodels.api as sm
21 from sko.GA import GA
22 num_seed = 252618
23 np.random.seed(num_seed)
24 torch.manual_seed(num_seed)
25 import matplotlib
26 matplotlib.use('Agg')
27 import matplotlib.pyplot as plt
28 from mpl_toolkits.mplot3d import Axes3D
29 plt.rcParams['font.sans-serif'] = ['SimHei']
30 plt.rcParams['axes.unicode_minus'] = False
31 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
32 colors = np.array(plt.rcParams['axes.prop_cycle'].by_key()['color'] + ['gold', 'deeppink'])
33 print()
34 # =====

```

```

35 P_list = []
36 MMP_list = []
37 L_list = []
38 Lambda = [0.01, 0.1, 1, 10, 100][4]
39 name_list = 'f, fs, Bm, T, M, W'.split(',')
40 class Net(nn.Module):
41     def __init__(self, latent=128):
42         super().__init__()
43         self.k = nn.Sequential(
44             nn.Linear(6, latent),
45             nn.GroupNorm(1, latent),
46             nn.Tanh(),
47             nn.Linear(latent, 1)
48         )
49         self.alpha = nn.Sequential(
50             nn.Linear(6, latent),
51             nn.GroupNorm(1, latent),
52             nn.Tanh(),
53             nn.Linear(latent, 1)
54         )
55         self.beta = nn.Sequential(
56             nn.Linear(6, latent),
57             nn.GroupNorm(1, latent),
58             nn.Tanh(),
59             nn.Linear(latent, 1)
60         )
61         self.gamma = nn.Sequential(
62             nn.Linear(6, latent),
63             nn.GroupNorm(1, latent),
64             nn.Tanh(),
65             nn.Linear(latent, 1)
66         )
67         self.zeta = nn.Sequential(
68             nn.Linear(6, latent),
69             nn.GroupNorm(1, latent),
70             nn.Tanh(),
71             nn.Linear(latent, 1)
72         )
73         self.eta = nn.Sequential(
74             nn.Linear(6, latent),
75             nn.GroupNorm(1, latent),
76             nn.Tanh(),
77             nn.Linear(latent, 1)
78         )
79         self.M_emb = nn.Embedding(4, 1)
80         self.W_emb = nn.Embedding(3, 1)

```

```

81 def forward(self, f, fs, Bm, T, M, W):
82     M = self.M_emb(M. type(torch.int32))
83     W = self.W_emb(W. type(torch.int32))
84     data_tmp = torch.concatenate((f.unsqueeze(-1), fs.unsqueeze(-1), Bm.unsqueeze(-1), T.
        unsqueeze(-1), M, W), dim=-1)
85     k = self.k(data_tmp).flatten()
86     alpha = self.alpha(data_tmp).flatten()
87     beta = self.beta(data_tmp).flatten()
88     gamma = self.gamma(data_tmp).flatten()
89     zeta = self.zeta(data_tmp).flatten()
90     eta = self.eta(data_tmp).flatten()
91     P = k + f + (alpha-1) * fs + beta * Bm + gamma * T + zeta * M.squeeze() + eta * W.squeeze
        ()
92     return k, alpha, beta, gamma, zeta, eta, P
93 # 读取模型
94 net = torch.load('D:/Pycharm Data/2024研究生数学建模竞赛/问题4/Argus/WMIMSEpro').cpu()
95 net. eval()
96 def func(x):
97     x = torch.tensor(x, dtype=torch.float32)
98     f, fs, Bm, T, M, W = x
99     M = M % 4
100    W = W % 3
101    print(f.item(), fs.item(), Bm.item(), T.item(), M.item(), W.item(), end='')
102    f = torch.log(f).view(1)
103    fs = torch.log(fs).view(1)
104    Bm = torch.log(Bm).view(1)
105    T = torch.log(T).view(1)
106    M = M.view(1)
107    W = W.view(1)
108    with torch.no_grad():
109        k, alpha, beta, gamma, zeta, eta, P = net(f, fs, Bm, T, M, W)
110        P = torch.exp(P)
111        MMP = torch.exp(f)*torch.exp(Bm)
112        L = P.item() - Lambda*MMP.item()
113        print(' P={}, MMP={}, L={}'. format(P.item(), MMP.item(), L))
114        P_list.append(P.item())
115        MMP_list.append(MMP.item())
116        L_list.append(L)
117    return L
118 def get_res(x):
119     x = torch.tensor(x, dtype=torch.float32)
120     f, fs, Bm, T, M, W = x
121     M = M % 4
122     W = W % 3
123     print(f.item(), fs.item(), Bm.item(), T.item(), M.item(), W.item(), end='')
124     f = torch.log(f).view(1)

```

```

125     fs = torch.log(fs).view(1)
126     Bm = torch.log(Bm).view(1)
127     T = torch.log(T).view(1)
128     M = M.view(1)
129     W = W.view(1)
130     with torch.no_grad():
131         k, alpha, beta, gamma, zeta, eta, P = net(f, fs, Bm, T, M, W)
132     P = torch.exp(P)
133     MMP = torch.exp(f) * torch.exp(Bm)
134     L = P.item() - Lambda * MMP.item()
135     print(' P={}, MMP={}, L={}'.format(P.item(), MMP.item(), L))
136     P_list.append(P.item())
137     MMP_list.append(MMP.item())
138     L_list.append(L)
139     return P.item(), MMP.item(), L
140 if __name__ == '__main__':
141     ga = GA(func=func,
142             n_dim=6,
143             size_pop=30,
144             max_iter=1000,
145             lb=[5000, 0.00082535174, 0.00963815, 25, 0, 0],
146             ub=[500000, 0.0021981855, 0.31328446, 90, 3, 2],
147             precision=[1, 0.000001, 0.000001, 1, 1, 1]
148             )
149     best_x, best_y = ga.run()
150     P, MMP, L = get_res(best_x)
151     txt1 = 'P = {}\nMMP = {}\nL={}\n'.format(P, MMP, L)
152     print(txt1)
153     plt.figure(figsize=(8, 5), dpi=150)
154     plt.plot(P_list, linewidth=3, markersize='3')
155     plt.tick_params(labelsize=15)
156     plt.xlabel('Step', fontsize=25)
157     plt.savefig("./Plots/GA减法_P({}).png".format(Lambda), bbox_inches='tight')
158     plt.savefig("./Plots/GA减法_P({}).pdf".format(Lambda), bbox_inches='tight')
159     plt.close('all') # 避免内存泄漏
160     plt.figure(figsize=(8, 5), dpi=150)
161     plt.plot(MMP_list, linewidth=3, markersize='3')
162     plt.tick_params(labelsize=15)
163     plt.xlabel('Step', fontsize=25)
164     plt.savefig("./Plots/GA减法_MMP({}).png".format(Lambda), bbox_inches='tight')
165     plt.savefig("./Plots/GA减法_MMP({}).pdf".format(Lambda), bbox_inches='tight')
166     plt.close('all') # 避免内存泄漏
167     plt.figure(figsize=(8, 5), dpi=150)
168     plt.plot(L_list, linewidth=3, markersize='3')
169     plt.tick_params(labelsize=15)
170     plt.xlabel('Step', fontsize=25)

```

```

171 plt.savefig("./Plots/GA减法_L({}).png".format(Lambda), bbox_inches='tight')
172 plt.savefig("./Plots/GA减法_L({}).pdf".format(Lambda), bbox_inches='tight')
173 plt.close('all') # 避免内存泄漏
174 txt2 = '\n'
175 for i in range(6):
176     txt2 = txt2 + '{} = {} \n\n'.format(name_list[i], best_x[i])
177 print(txt2)
178 with open('./Results/GA减法_({}).txt'.format(Lambda), 'w', encoding='utf-8') as f:
179     f.write(txt1)
180     f.write(txt2)
181 print()

```

B.6 问题 5: 粒子群优化-减法

Listing 7: Main_PSO 减法.py

```

1 import gzip
2 import os.path
3 import sympy
4 import hdf5storage
5 import torch
6 import torch.nn as nn
7 import numpy as np
8 import pandas as pd
9 from time import time
10 from tqdm import tqdm
11 from collections import Counter
12 from sklearn.manifold import MDS
13 from scipy.spatial import distance
14 from sklearn import svm
15 from sklearn.manifold import TSNE
16 from torch.optim.lr_scheduler import CosineAnnealingLR
17 from torch.utils.data import DataLoader, TensorDataset
18 from sklearn.model_selection import train_test_split
19 from sklearn.metrics import mean_squared_error, mean_squared_log_error, mean_absolute_error,
    explained_variance_score, \
20     r2_score
21 import statsmodels.api as sm
22 from sko.PSO import PSO
23 num_seed = 252618
24 np.random.seed(num_seed)
25 torch.manual_seed(num_seed)
26 import matplotlib
27 matplotlib.use('Agg')
28 import matplotlib.pyplot as plt
29 from mpl_toolkits.mplot3d import Axes3D

```

```

30 plt.rcParams['font.sans-serif'] = ['SimHei']
31 plt.rcParams['axes.unicode_minus'] = False
32 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
33 colors = np.array(plt.rcParams['axes.prop_cycle'].by_key()['color'] + ['gold', 'deeppink'])
34 print()
35 # =====
36 P_list = []
37 MMP_list = []
38 L_list = []
39 Lambda = [0.01, 0.1, 1, 10, 100][4]
40 name_list = 'f, fs, Bm, T, M, W'.split(',')
41 class Net(nn.Module):
42     def __init__(self, latent=128):
43         super().__init__()
44         self.k = nn.Sequential(
45             nn.Linear(6, latent),
46             nn.GroupNorm(1, latent),
47             nn.Tanh(),
48             nn.Linear(latent, 1)
49         )
50         self.alpha = nn.Sequential(
51             nn.Linear(6, latent),
52             nn.GroupNorm(1, latent),
53             nn.Tanh(),
54             nn.Linear(latent, 1)
55         )
56         self.beta = nn.Sequential(
57             nn.Linear(6, latent),
58             nn.GroupNorm(1, latent),
59             nn.Tanh(),
60             nn.Linear(latent, 1)
61         )
62         self.gamma = nn.Sequential(
63             nn.Linear(6, latent),
64             nn.GroupNorm(1, latent),
65             nn.Tanh(),
66             nn.Linear(latent, 1)
67         )
68         self.zeta = nn.Sequential(
69             nn.Linear(6, latent),
70             nn.GroupNorm(1, latent),
71             nn.Tanh(),
72             nn.Linear(latent, 1)
73         )
74         self.eta = nn.Sequential(
75             nn.Linear(6, latent),

```



```

76         nn.GroupNorm(1, latent),
77         nn.Tanh(),
78         nn.Linear(latent, 1)
79     )
80     self.M_emb = nn.Embedding(4, 1)
81     self.W_emb = nn.Embedding(3, 1)
82     def forward(self, f, fs, Bm, T, M, W):
83         M = self.M_emb(M. type(torch.int32))
84         W = self.W_emb(W. type(torch.int32))
85         data_tmp = torch.concatenate((f.unsqueeze(-1), fs.unsqueeze(-1), Bm.unsqueeze(-1), T.
            unsqueeze(-1), M, W),
86                                     dim=-1)
87         k = self.k(data_tmp).flatten()
88         alpha = self.alpha(data_tmp).flatten()
89         beta = self.beta(data_tmp).flatten()
90         gamma = self.gamma(data_tmp).flatten()
91         zeta = self.zeta(data_tmp).flatten()
92         eta = self.eta(data_tmp).flatten()
93         P = k + f + (alpha - 1) * fs + beta * Bm + gamma * T + zeta * M.squeeze() + eta * W.
            squeeze()
94         return k, alpha, beta, gamma, zeta, eta, P
95 # 读取模型
96 net = torch.load('D:/Pycharm Data/2024研究生数学建模竞赛/问题4/Argus/WMTMSEpro').cpu()
97 net. eval()
98 def func(x):
99     x = torch.tensor(x, dtype=torch.float32)
100     f, fs, Bm, T, M, W = x
101     M = M % 4
102     W = W % 3
103     print(f.item(), fs.item(), Bm.item(), T.item(), M.item(), W.item(), end='')
104     f = torch.log(f).view(1)
105     fs = torch.log(fs).view(1)
106     Bm = torch.log(Bm).view(1)
107     T = torch.log(T).view(1)
108     M = M.view(1)
109     W = W.view(1)
110     with torch.no_grad():
111         k, alpha, beta, gamma, zeta, eta, P = net(f, fs, Bm, T, M, W)
112         P = torch.exp(P)
113         MMP = torch.exp(f) * torch.exp(Bm)
114         L = P.item() - Lambda * MMP.item()
115         print(' P={}, MMP={}, L={}'. format(P.item(), MMP.item(), L))
116         P_list.append(P.item())
117         MMP_list.append(MMP.item())
118         L_list.append(L)
119     return L

```

```

120 def get_res(x):
121     x = torch.tensor(x, dtype=torch.float32)
122     f, fs, Bm, T, M, W = x
123     M = M % 4
124     W = W % 3
125     print(f.item(), fs.item(), Bm.item(), T.item(), M.item(), W.item(), end='')
126     f = torch.log(f).view(1)
127     fs = torch.log(fs).view(1)
128     Bm = torch.log(Bm).view(1)
129     T = torch.log(T).view(1)
130     M = M.view(1)
131     W = W.view(1)
132     with torch.no_grad():
133         k, alpha, beta, gamma, zeta, eta, P = net(f, fs, Bm, T, M, W)
134         P = torch.exp(P)
135         MMP = torch.exp(f) * torch.exp(Bm)
136         L = P.item() - Lambda * MMP.item()
137         print(' P={}, MMP={}, L={}'.format(P.item(), MMP.item(), L))
138         P_list.append(P.item())
139         MMP_list.append(MMP.item())
140         L_list.append(L)
141     return P.item(), MMP.item(), L
142 if __name__ == '__main__':
143     pso = PSO(func=func,
144               dim=6,
145               pop=30,
146               max_iter=1000,
147               lb=[5000, 0.00082535174, 0.00963815, 25, 0, 0],
148               ub=[500000, 0.0021981855, 0.31328446, 90, 3, 2]
149               )
150     pso.run()
151     best_x, best_y = pso.gbest_x, pso.gbest_y
152     best_x[-1] = best_x[-1] % 3
153     best_x[-2] = best_x[-2] % 4
154     P, MMP, L = get_res(best_x)
155     txt1 = 'P = {}\nMMP = {}\nL={}\n'.format(P, MMP, L)
156     print(txt1)
157     plt.figure(figsize=(8, 5), dpi=150)
158     plt.plot(P_list, linewidth=3, markersize='3')
159     plt.tick_params(labels=15)
160     plt.xlabel('Step', fontsize=25)
161     plt.savefig("./Plots/PSO减法_P({}).png".format(Lambda), bbox_inches='tight')
162     plt.savefig("./Plots/PSO减法_P({}).pdf".format(Lambda), bbox_inches='tight')
163     plt.close('all') # 避免内存泄漏
164     plt.figure(figsize=(8, 5), dpi=150)
165     plt.plot(MMP_list, linewidth=3, markersize='3')

```

```

166 plt.tick_params(labelsize=15)
167 plt.xlabel('Step', fontsize=25)
168 plt.savefig("./Plots/PSO减法_MMP({}).png".format(Lambda), bbox_inches='tight')
169 plt.savefig("./Plots/PSO减法_MMP({}).pdf".format(Lambda), bbox_inches='tight')
170 plt.close('all') # 避免内存泄漏
171 plt.figure(figsize=(8, 5), dpi=150)
172 plt.plot(L_list, linewidth=3, markersize='3')
173 plt.tick_params(labelsize=15)
174 plt.xlabel('Step', fontsize=25)
175 plt.savefig("./Plots/PSO减法_L({}).png".format(Lambda), bbox_inches='tight')
176 plt.savefig("./Plots/PSO减法_L({}).pdf".format(Lambda), bbox_inches='tight')
177 plt.close('all') # 避免内存泄漏
178 txt2 = '\n'
179 for i in range(6):
180     txt2 = txt2 + '{} = {} \n\n'.format(name_list[i], best_x[i])
181 print(txt2)
182 with open("./Results/PSO减法_({}).txt".format(Lambda), 'w', encoding='utf-8') as f:
183     f.write(txt1)
184     f.write(txt2)
185 print()

```